



République Tunisienne
Ministère de l'Enseignement Supérieur
et de la Recherche Scientifique

École Supérieure Privée de Technologies et d'Ingénierie
TEK-UP



Rapport de Projet de Fin d'Études

Présenté en vue de l'obtention du

Diplôme National d'Ingénieur en Sciences Appliquées et Technologiques
Spécialité : Sécurité des Systèmes Informatiques et Réseaux (SSIR)

Élaboré par :

Aymen AZIZI

Conception et Développement d'une Plateforme Web de Reconnaissance de Surface d'Attaque avec Génération de Rapports Assistée par IA et Intégration DevSecOps

Encadrant Professionnel : **M. Ali DORBOZ**

Encadrante Universitaire : **Mme Sonia BEN AISSA**

Directeur Technique
(CTO)

Encadrante Universitaire

J'autorise l'étudiant à déposer son rapport de projet de fin d'études en vue de la soutenance.

Encadrant Professionnel, **M. Ali DORBOZ**

Signature et cachet

J'autorise l'étudiant à déposer son rapport de projet de fin d'études en vue de la soutenance.

Encadrante Universitaire, **Mme Sonia BEN AISSA**

Signature

Dédicaces

À mes très chers parents :

Votre amour inconditionnel, votre soutien indéfectible et vos sacrifices continus ont été ma source d'inspiration et de persévérance tout au long de mon parcours. Sans vos encouragements et vos prières, rien de tout cela n'aurait été possible. Que ce travail soit le témoignage de ma profonde gratitude et de mon éternel respect.

À mes frères et sœurs :

Merci pour vos encouragements constants, votre présence chaleureuse et votre motivation inébranlable dans les moments les plus exigeants de ce projet. Vous avez été un véritable pilier de force et d'inspiration.

À toute ma famille et à mes fidèles amis :

Je vous exprime ma plus sincère reconnaissance pour votre soutien moral, vos précieux conseils et vos encouragements tout au long de mes années d'études d'ingénieur. Chacun de vous a contribué à sa manière à l'aboutissement de cette réalisation.

Aymen AZIZI

Remerciements

C'est avec un immense plaisir et une profonde gratitude que j'adresse mes sincères remerciements à toutes les personnes qui ont contribué, de près ou de loin, à la réalisation et à la réussite de ce travail.

Tout d'abord, j'adresse mes plus vifs remerciements aux membres du jury pour l'honneur qu'ils me font en acceptant d'évaluer ce projet de fin d'études. Leur expertise, leur temps et leurs précieuses remarques constructives sont pour moi du plus grand intérêt.

*J'exprime ma profonde reconnaissance à mon encadrante universitaire à l'École Supérieure Privée de Technologies et d'Ingénierie **TEK-UP**, Madame **Sonia Ben Aissa**, pour son encadrement rigoureux, sa disponibilité constante et ses orientations méthodologiques avisées tout au long de ce projet.*

*J'adresse également mes chaleureux remerciements à mon encadrant professionnel au sein de l'entreprise d'accueil **Designet Web Agency**, Monsieur **Ali DORBOZ**, Directeur Technique (CTO), pour son accompagnement technique de haut niveau, sa bienveillance et ses précieux conseils durant toute la période du stage.*

*Je tiens également à témoigner ma gratitude à toute l'équipe de Designet Web Agency, et en particulier à Monsieur **Raed Ben Khelifa**, pour son accueil chaleureux ainsi que pour les excellentes conditions de travail offertes tout au long de mon stage.*

Enfin, je remercie chaleureusement l'ensemble du corps professoral et administratif de l'école

***TEK-UP** pour la qualité de la formation d'ingénieur dispensée tout au long de notre cursus*

universitaire.

Résumé

Ce projet de fin d'études porte sur la conception et le développement d'une plateforme web semi-agentique de reconnaissance de surface d'attaque, intégrant une modélisation par graphe de connaissances (PostgreSQL 16 / Apache AGE), une IA locale contrainte (Ollama / `qwen2.5-coder`) pour la production de *Remediation-as-Code*, et une chaîne DevSecOps automatisée (SAST, SCA, SBOM CycloneDX, Trivy, Cosign). Réalisée au sein de Designet Web Agency avec un back-end Laravel 11 et cinq microservices Python Flask orchestrés via Redis Streams, la plateforme a validé 140 tests automatisés de bout en bout et soutenu une charge nominale de 50 utilisateurs simultanés sous un cadre légal strict (Proof of Authorization).

Mots-clés : Cybersécurité, Surface d'attaque, Graphe de connaissances, Remediation-as-Code, DevSecOps, Redis Streams, PostgreSQL, Apache AGE, NetworkX, Ollama, Laravel 11, Flask, SBOM, Cosign

Abstract

This graduation project focuses on the design and implementation of a semi-agentic web platform for attack surface reconnaissance, integrating a knowledge graph (PostgreSQL 16 / Apache AGE), a constrained local AI (Ollama / qwen2.5-coder) for Remediation-as-Code generation, and an automated DevSecOps pipeline (SAST, SCA, CycloneDX SBOM, Trivy, Cosign). Developed at Designet Web Agency with a Laravel 11 back-end and five Python Flask microservices orchestrated via Redis Streams, the platform validated 140 automated tests and sustained a benchmark load of 50 concurrent users under a mandatory Proof of Authorization (PoA) governance.

Keywords : Cybersecurity, Attack Surface Reconnaissance, Knowledge Graph, Remediation-as-Code, DevSecOps, Redis Streams, PostgreSQL, Apache AGE, NetworkX, Ollama, Laravel 11, Flask, SBOM, Cosign

Table des matières

Résumé	iv
Abstract	v
Liste des Abréviations	xiv
Introduction Générale	1
1 Cadre Général du Projet	3
1.1 Présentation de l'Organisme d'Accueil	3
1.1.1 Identité et Domaines d'Activité	3
1.1.2 Organigramme de l'Entreprise	4
1.1.3 Cadre Institutionnel du Stage	5
1.2 Contexte Industriel et Motivations	5
1.3 Étude de l'Existant et Analyse Critique	5
1.3.1 Procédure d'Audit Traditionnelle	5
1.3.2 Limites de la Démarche Traditionnelle	6
1.3.3 Comparatif des Solutions Existantes sur le Marché	6
1.4 Problématique	7
1.5 Solution Proposée	7
1.6 Objectifs du Projet	7
1.6.1 Objectif Général	7
1.6.2 Objectifs Spécifiques	8
1.7 Périmètre du Projet (Scope)	8
1.7.1 Périmètre Inclus	8
1.7.2 Périmètre Exclu (par conception)	9
1.8 Méthodologie de Travail et Conduite du Projet	9
1.8.1 Justification de la Démarche Agile	9
1.8.2 Le Cadre Méthodologique Scrum	9
1.8.3 Planification des Sprints et Calendrier Global	10
2 État de l'Art et Étude Comparative	11
2.1 Fondements de la Cybersécurité	11

2.1.1	Définition et Enjeux Contemporains	11
2.1.2	Piliers Fondamentaux de la Sécurité	11
2.2	Le Référentiel OWASP Top 10	12
2.3	Gestion Continue de la Surface d’Attaque (ASM)	14
2.3.1	Définition et Principes	14
2.3.2	Cycle de Vie du Processus ASM	14
2.4	Reconnaissance et Outils d’Audit de Sécurité	15
2.4.1	Rôle et Typologie de la Reconnaissance	15
2.4.2	Outils d’Audit Intégrés dans la Plateforme	15
2.5	DevSecOps et Sécurité de la Chaîne Logicielle	16
2.5.1	Origine et Philosophie Shift-Left	16
2.5.2	Sécurité de la Chaîne d’Approvisionnement Logicielle (Software Supply Chain)	16
2.6	Intelligence Artificielle Locale et Grands Modèles de Langage (LLM)	17
2.6.1	Apport des LLM dans l’Audit de Sécurité	17
2.6.2	Impératif de Confidentialité et Modèles Contraints	17
2.7	Architectures Microservices et Systèmes Événementiels	18
2.8	Étude Comparative et Justification des Choix Technologiques	18
2.8.1	Comparatif des Frameworks Backend Web	19
2.8.2	Comparatif des Frameworks de Microservices Python	19
2.8.3	Comparatif des Moteurs d’IA Locale (LLM)	20
2.8.4	Comparatif de la Base de Données et du Moteur de Graphe	20
2.8.5	Comparatif du Message Broker pour Traitement Asynchrone	21
2.8.6	Synthèse Globale des Choix Technologiques	21
3	Analyse des Besoins et Conception du Système	22
3.1	Analyse des Besoins	22
3.1.1	Identification des Acteurs	22
3.1.2	Besoins Fonctionnels	23
3.1.3	Besoins Non Fonctionnels	24
3.1.4	Contraintes et Exigences de Sécurité	24
3.1.5	Contraintes Légales et Preuve d’Autorisation (Proof of Authorization – PoA)	24
3.2	Modélisation Fonctionnelle UML	24
3.2.1	Diagramme Global des Cas d’Utilisation	24
3.2.2	Cas d’Utilisation de l’Analyste de Sécurité	24

3.2.3	Cas d'Utilisation de l'Administrateur Système	25
3.2.4	Description Textuelle Détaillée des Cas d'Utilisation Clés	25
3.3	Architecture Globale du Système	32
3.3.1	Vue d'Ensemble du Système	32
3.3.2	Composants Applicatifs Métier (7 Composants Logiques)	32
3.3.3	Conteneurs d'Infrastructure Docker (12 Services Conteneurisés)	33
3.3.4	Communication Inter-Services et Flux Asynchrone	33
3.3.5	Communication Synchrones REST vs Asynchrone par Événements	35
3.4	Conception des Données et Dynamique du Système	35
3.4.1	Modèle Entité-Association (ERD)	35
3.4.2	Diagramme de Classes du Modèle de Données	36
3.4.3	Modèle du Graphe de Connaissances et Projection Apache AGE	36
3.4.4	Diagrammes de Séquence du Système	36
3.4.5	Machine à États d'un Scan	38
3.4.6	Diagramme de Flux de Données (DFD)	39
3.5	Architecture de Sécurité Multicouche	39
3.6	Modélisation Formelle des Menaces (Threat Modeling STRIDE)	39
3.6.1	Méthodologie et Frontières de Confiance	39
3.6.2	Matrice des Menaces STRIDE et Contre-Mesures Appliquées	40
3.7	Architecture DevSecOps Détaillée	40
3.7.1	Pipeline CI/CD sous GitHub Actions	40
3.7.2	Description des Portes de Sécurité (Security Gates)	40
4	Réalisation et Validation Expérimentale	42
4.1	Environnement de Développement et Outillage	42
4.2	Implémentation du Back-end Applicatif (Laravel 11)	43
4.2.1	Architecture Interne en Couches du Back-end	43
4.2.2	Authentification, RBAC et Middlewares de Sécurité	43
4.2.3	Orchestration Asynchrone des Scans (Jobs & Queues)	44
4.3	Implémentation des Microservices Python Flask	44
4.3.1	Microservice de Reconnaissance (Port 5000)	44
4.3.2	Microservice de Sécurité Offensive et Bac à Sable (Port 5001)	44
4.3.3	Microservice OSINT (Port 5002)	44
4.3.4	Microservice d'IA Contrainte (Port 5003)	45

4.3.5	Architecture du Framework de Scanners (BaseScannerService)	45
4.3.6	Pipeline de Normalisation des Données (ResultParser)	46
4.4	Spécification et Conception de l'API RESTful	46
4.4.1	Exemple de Requête et Réponse JSON Normalisée	47
4.5	Interfaces Utilisateurs de la Plateforme	47
4.5.1	Authentification et Enregistrement Sécurisé	47
4.5.2	Tableau de Bord Principal (Dashboard ASM)	48
4.5.3	Gestion des Projets et Cibles d'Audit	49
4.5.4	Lancement et Suivi des Scans de Reconnaissance	50
4.5.5	Restitution des Résultats et Preuves Brutes	52
4.5.6	Fiche de Constat et Remediation-as-Code Assistée par IA	53
4.5.7	Visualisation Interactive du Graphe de Connaissances	53
4.5.8	Module de Renseignement Passif (OSINT)	54
4.5.9	Gestionnaire du Bac à Sable (Sandbox Isolée)	55
4.5.10	Alertes de Sécurité et Surveillance Continue	56
4.5.11	Co-pilote Conversationnel de Sécurité (AI Assistant)	57
4.5.12	Administration des Utilisateurs et Rôles RBAC	57
4.5.13	Journalisation d'Audit Immuable (Audit Logs)	57
4.5.14	Surveillance de la Santé des Microservices (System Health)	58
4.5.15	Génération et Consultation des Rapports d'Expertise	59
4.6	Conception et Implémentation Avancée du Graphe de Connaissances	60
4.6.1	Modèle de Données en Graphe (Nœuds et Arêtes)	60
4.6.2	Pipeline de Construction et Requêtage openCypher	60
4.6.3	Algorithme de Calcul du Rayon d'Impact (Blast Radius)	61
4.7	Moteur d'Intelligence Artificielle Contrainte et Remediation-as-Code	62
4.7.1	Pipeline de Traitement IA	62
4.7.2	Schéma de Sortie Strict et Réduction des Hallucinations	62
4.8	Déploiement en Production et Durcissement VPS	63
4.9	Exécution du Pipeline DevSecOps	63
4.10	Difficultés Rencontrées et Solutions Apportées	63
4.10.1	Difficulté 1 : Intégration et Compatibilité d'Apache AGE avec PostgreSQL 16 .	64
4.10.2	Difficulté 2 : Optimisation de la Latence du LLM Local et Élimination des Hallucinations	64
4.10.3	Difficulté 3 : Isolation Hermétique du Bac à Sable (Docker Socket Proxy) . . .	64

4.10.4	Difficulté 4 : Normalisation Hétérogène des Sorties Scanners	64
4.10.5	Difficulté 5 : Gestion de la Concurrence et Résilience des Workers Redis Streams	65
4.11	Matrice de Traçabilité des Exigences (RTM)	65
4.12	Protocole Expérimental et Résultats des Tests	65
4.12.1	Protocole Expérimental	65
4.12.2	Synthèse des Tests Automatisés (140/140 Tests)	66
4.12.3	Détail des Cas de Tests Représentatifs	66
4.13	Évaluation des Performances sous Charge (Locust)	67
4.13.1	Conditions du Benchmark et Métriques Recueillies	67
4.13.2	Analyse Approfondie des Écarts de Latence	67
4.14	Conformité au Cahier des Charges (CDC)	67
4.15	Discussion Approfondie des Résultats	68
4.16	Limites du Système	69
	Conclusion Générale et Perspectives	70
	Annexe A : Catalogue de l'API RESTful	72
	Annexe B : Artéfacts DevSecOps	73
	Annexe C : Nginx et Requêtes openCypher	76
	Bibliographie et Webographie	78

Table des figures

1	Comparaison Conceptuelle : Démarche d'Audit Traditionnelle vs Plateforme Semi-Agentique Proposée	2
1.1	Logo Officiel de l'Organisme d'Accueil – Designet Web Agency	4
1.2	Organigramme Fonctionnel de Designet Web Agency	4
1.3	Processus Séquentiel d'Audit Traditionnel et Frictions Opérationnelles Associées . . .	6
1.4	Vue d'Ensemble du Processus Agile Scrum Appliqué au Projet	10
2.1	Matrice des Risques OWASP Top 10 (2021) – Exploitabilité vs Détectabilité	13
2.2	Cycle de Vie du Processus d'Attack Surface Management (ASM)	14
2.3	Cycle de Livraison DevSecOps : Intégration Continue des Portes de Sécurité (Shift-Left)	16
2.4	Pipeline de l'IA Locale Contrainte pour la Production de Remediation-as-Code	18
2.5	Patron d'Architecture Microservices et Découplage Asynchrone via Redis Streams . .	18
3.1	Diagramme Global des Cas d'Utilisation – Acteurs et Fonctionnalités Clés	25
3.2	Diagramme des Cas d'Utilisation – Analyste de Sécurité	26
3.3	Diagramme des Cas d'Utilisation – Administrateur Système	27
3.4	Architecture Globale en Microservices – Vue des Conteneurs Docker et Réseaux Segmentés	34
3.5	Flux de Traitement Asynchrone des Scans via Redis Streams et PostgreSQL AGE . .	35
3.6	Modèle Entité-Association (ERD) – Les 14 Tables de la Plateforme	35
3.7	Diagramme de Classes – Entités Métier et Relations du Modèle	36
3.8	Diagramme de Séquence – Lancement, Traitement Asynchrone et Remédiation IA . .	37
3.9	Diagramme de Séquence – Génération de Remediation-as-Code Assistée par IA	37
3.10	Diagramme de Séquence – Exploration du Graphe et Calcul de Blast Radius	38
3.11	Machine à États – Cycle de Vie d'une Tâche de Scan	38
3.12	Diagramme de Flux de Données (DFD) – Processus d'Audit, Graphe de Connaissances et Reporting	39
3.13	Cartographie de la Surface d'Attaque et Frontières de Confiance (Trust Boundaries STRIDE)	40
3.14	Pipeline DevSecOps Automatisé – Chaîne de Sécurité Intégrée sous GitHub Actions .	40
4.1	Architecture Modulaire du Framework d'Exécution et de Parsing des Scanners (BaseScannerService)	45

4.2	Interface d'Authentification Sécurisée de la Plateforme	48
4.3	Interface de Création de Compte et Enregistrement Utilisateur	48
4.4	Tableau de Bord Principal – Indicateurs Clés (KPIs) et Cartographie des Risques . . .	49
4.5	Liste des Projets d'Audit et Synthèse du Statut des Cibles	49
4.6	Formulaire de Création de Projet et Téléversement de la Preuve d'Autorisation (PoA)	50
4.7	Fiche Détaillée d'un Projet – Cibles Associées et Historique des Scans	50
4.8	Interface de Configuration et Lancement d'un Scan d'Audit	51
4.9	Liste et État d'Avancement des Scans d'Audit Orchestrés	52
4.10	Restitution des Résultats de Scan et Visualisation des Preuves Techniques	53
4.11	Fiche Détaillée d'un Constat et Scripts de Remediation-as-Code Générés par l'IA . . .	53
4.12	Visualisation Interactive du Graphe de Connaissances et du Rayon d'Impact	54
4.13	Module de Renseignement Passif OSINT – Analyse DNS, WHOIS et Certificats TLS .	55
4.14	Gestionnaire du Bac à Sable (Sandbox) pour la Validation d'Exploits	55
4.15	Centre de Gestion et d'Acquittement des Alertes de Sécurité	56
4.16	Module de Surveillance Continue et Télémétrie des Actifs Exposés	56
4.17	Assistant Conversationnel d'Analyse de Sécurité Assisté par IA Locale	57
4.18	Module d'Administration des Utilisateurs et Gestion des Rôles RBAC	57
4.19	Module de Journalisation d'Audit Immuable pour la Traçabilité des Événements . . .	58
4.20	Supervision de la Santé des Microservices et État des Conteneurs Docker	59
4.21	Module de Génération et Export Multi-Formats des Rapports d'Audit	59
4.22	Visualisation et Consultation d'un Rapport d'Expertise de Sécurité	60

Liste des tableaux

1.1	Analyse Comparative des Solutions d'Audit et de Gestion de Surface d'Attaque	6
1.2	Répartition et Responsabilités des Rôles Scrum sur le Projet	9
1.3	Planification Détaillée des Sprints Scrum et Calendrier des Réalisations	10
2.1	Analyse Comparative des Frameworks Web Backend	19
2.2	Analyse Comparative des Frameworks de Microservices	19
2.3	Analyse Comparative des Moteurs d'IA Locale	20
2.4	Analyse Comparative des Solutions de Base de Données et Graphes	20
2.5	Analyse Comparative des Systèmes de Message Broker	21
2.6	Tableau Récapitulatif de la Pile Technologique Retenue	21
3.1	Description Textuelle du Cas d'Utilisation CU01 – Lancement d'un Scan	28
3.2	Description Textuelle du Cas d'Utilisation CU02 – Génération de Remediation	29
3.3	Description Textuelle du Cas d'Utilisation CU03 – Graphe de Connaissances	30
3.4	Description Textuelle du Cas d'Utilisation CU04 – Gestion des Projets et PoA	31
3.5	Description Textuelle du Cas d'Utilisation CU05 – Génération de Rapports	32
3.6	Mesures de Sécurité en Profondeur par Couche Architecturale	39
3.7	Matrice d'Analyse des Menaces STRIDE et Contre-Mesures Architecturales	40
4.1	Spécification de la Configuration Matérielle Utilisée	42
4.2	Environnement Logiciel, Outils et Versions des Frameworks	43
4.3	Spécification des Principaux Endpoints de l'API RESTful de la Plateforme	46
4.4	Matrice de Traçabilité des Exigences (Requirements Traceability Matrix)	65
4.5	Résultats de la Suite de Tests Automatisés	66
4.6	Détail des Scénarios de Tests Représentatifs Validés	66
4.7	Métriques de Performance Mesurées sous Charge (50 Utilisateurs Simultanés)	67
4.8	Conformité aux Exigences Non Fonctionnelles (CDC vs Mesures)	68
4.9	Catalogue Exhaustif des Endpoints de l'API RESTful	72

Liste des Abréviations

—	2FA Two-Factor Authentication (Authentification à Deux Facteurs)
—	ABAC Attribute-Based Access Control
—	ACL Access Control List (Liste de Contrôle d'Accès)
—	AGE Apache AGE (A Graph Engine)
—	AI Artificial Intelligence / Intelligence Artificielle
—	API Application Programming Interface (Interface de Programmation Applicative)
—	ASM Attack Surface Management (Gestion de la Surface d'Attaque)
—	AWS Amazon Web Services
—	BFS Breadth-First Search (Parcours en Largeur)
—	CD Continuous Deployment (Déploiement Continu)
—	CI Continuous Integration (Intégration Continue)
—	CLI Command-Line Interface (Interface en Ligne de Commande)
—	CMS Content Management System (Système de Gestion de Contenu)
—	CSP Content Security Policy
—	CVE Common Vulnerabilities and Exposures
—	CVSS Common Vulnerability Scoring System
—	CWE Common Weakness Enumeration
—	DAST Dynamic Application Security Testing
—	DNS Domain Name System
—	DVWA Damn Vulnerable Web Application
—	DevOps Development and Operations
—	DevSecOps Development, Security, and Operations
—	DoS Denial of Service (Déni de Service)
—	GDPR/RGPD Règlement Général sur la Protection des Données
—	GHCR GitHub Container Registry
—	GUI Graphical User Interface (Interface Graphique Utilisateur)
—	HSTS HTTP Strict Transport Security

—	HTTP Hypertext Transfer Protocol
—	HTTPS Hypertext Transfer Protocol Secure
—	IDS Intrusion Detection System (Système de Détection d’Intrusion)
—	IP Internet Protocol
—	IaC Infrastructure as Code
—	JSON JavaScript Object Notation
—	JSONB JSON Binary (Type de colonne binaire PostgreSQL)
—	JSONL JavaScript Object Notation Lines
—	LLM Large Language Model (Grand Modèle de Langage)
—	MFA Multi-Factor Authentication (Authentification Multifacteur)
—	MVC Modèle-Vue-Contrôleur
—	NAT Network Address Translation
—	NSE Nmap Scripting Engine
—	NVD National Vulnerability Database
—	OIDC OpenID Connect
—	OPA Open Policy Agent
—	ORM Object-Relational Mapping
—	OS Operating System (Système d’Exploitation)
—	OSINT Open Source Intelligence (Renseignement d’Origine Source Ouverte)
—	OWASP Open Worldwide Application Security Project
—	PII Personally Identifiable Information (Données Personnelles Identifiables)
—	PoA Proof of Authorization (Mandat d’Autorisation d’Audit)
—	RBAC Role-Based Access Control (Contrôle d’Accès Basé sur les Rôles)
—	REST Representational State Transfer
—	SAN Subject Alternative Name (Noms Alternatifs du Sujet X.509)
—	SAST Static Application Security Testing
—	SBOM Software Bill of Materials
—	SCA Software Composition Analysis
—	SQL Structured Query Language

—	SQLi SQL Injection (Injection SQL)
—	SSH Secure Shell
—	SSIR Sécurité des Systèmes d'Information et Réseaux
—	SSRF Server-Side Request Forgery
—	TLS Transport Layer Security
—	URL Uniform Resource Locator
—	VPC Virtual Private Cloud
—	VPN Virtual Private Network (Réseau Privé Virtuel)
—	WAF Web Application Firewall (Pare-feu Applicatif Web)
—	WPScan WordPress Security Scanner
—	XSS Cross-Site Scripting
—	YAML YAML Ain't Markup Language

Introduction Générale

1. Contexte et Motivations

Dans un écosystème numérique marqué par l'adoption massive des infrastructures Cloud, la conteneurisation et les architectures en microservices, les systèmes d'information connaissent une extension sans précédent de leur surface d'exposition. Chaque nouveau service déployé, sous-domaine créé et port réseau ouvert constituent autant de points d'entrée potentiels pour des cyberattaquants de plus en plus outillés.

Face à cette réalité, les audits de sécurité périodiques traditionnels (annuels ou semestriels) révèlent d'importantes limites opérationnelles : entre deux campagnes manuelles, l'apparition d'une nouvelle vulnérabilité critique peut exposer durablement l'organisation. La gestion continue de la surface d'attaque externe (*External Attack Surface Management* – EASM) s'impose désormais comme un impératif stratégique pour garantir une visibilité permanente sur l'ensemble des actifs exposés.

2. Problématique et Approche Proposée

L'évaluation de la surface d'attaque implique l'orchestration conjointe d'un ensemble d'outils spécialisés (Nmap, Nuclei, Gobuster, Subfinder, WPScan). Néanmoins, dans la pratique industrielle au sein des centres de surveillance (SOC), cette démarche se heurte à quatre verrous majeurs :

- **Fragmentation et exécution manuelle en silos** : Absence d'orchestration centralisée et hétérogénéité des formats de sortie (XML, JSON, texte brut).
- **Déficit de corrélation sémantique** : Difficulté à modéliser les relations de dépendance entre actifs, ports, services et failles pour évaluer le risque réel.
- **Temps de latence de la remédiation (*MTTR*)** : Rédaction manuelle et chronophage de procédures de durcissement adaptées.
- **Exigences légales et éthiques** : Risque d'engager des actions de balayage offensif sans mandat formel d'autorisation (*Proof of Authorization* – PoA).

Pour lever ces verrous, ce projet propose une **plateforme web semi-agentique** combinant l'automatisation lourde des audits (orchestration multi-scanners asynchrone via Redis Streams), la corrélation relationnelle par graphe de connaissances (PostgreSQL 16 / Apache AGE / NetworkX) et l'assistance à la décision par une IA locale contrainte (Ollama / **qwen2.5-coder**). L'approche semi-agentique conserve l'humain dans la boucle (*Human-in-the-Loop*), garantissant que chaque décision sensible (scans intensifs, validation en bac à sable et déploiement de correctifs) demeure sous le contrôle

d'un analyste accrédité.

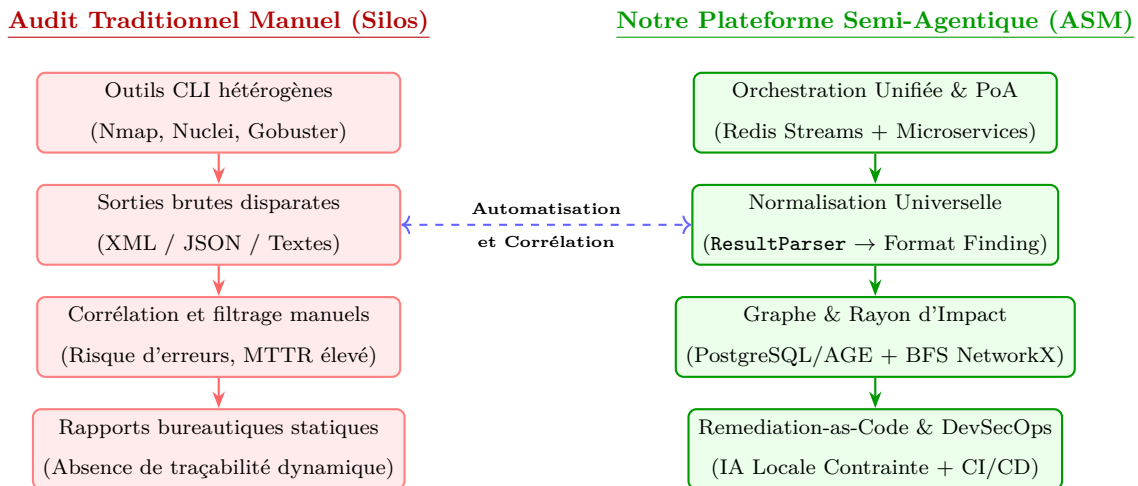


Figure 1 : Comparaison Conceptuelle : Démarche d'Audit Traditionnelle vs Plateforme Semi-Agentique Proposée

3. Cadre du Projet et Organisation du Rapport

Ce projet a été réalisé au sein de l'agence numérique **Designet Web Agency** (Menzel Temime, Tunisie), sous la direction conjointe de **M. Ali DORBOZ** (Directeur Technique) et de **Mme Sonia BEN AISSA** (Encadrante Universitaire, École TEK-UP). La conduite des travaux a suivi le cadre méthodologique Agile Scrum.

Le présent rapport d'ingénierie est structuré en quatre chapitres complémentaires :

- **Chapitre 1 – Cadre Général du Projet :** Présente l'organisme d'accueil, formalise le contexte métier, analyse l'état des solutions existantes du marché, définit la problématique et détaille la méthodologie Scrum adoptée.
- **Chapitre 2 – État de l'Art et Étude Comparative :** Expose les fondements théoriques (OWASP, ASM, DevSecOps, IA locale contrainte, microservices) et justifie le choix des technologies retenues.
- **Chapitre 3 – Analyse des Besoins et Conception du Système :** Définit les exigences fonctionnelles et non fonctionnelles, la modélisation UML (cas d'utilisation, séquences), l'architecture en microservices (7 composants applicatifs, 12 conteneurs Docker), la modélisation des menaces STRIDE et la sécurité de la chaîne DevSecOps.
- **Chapitre 4 – Réalisation et Validation Expérimentale :** Détaille l'implémentation des modules (Laravel 11, Flask, Apache AGE, Ollama), illustre les interfaces réalisées, analyse les difficultés surmontées, et valide la solution à travers 140 tests automatisés et des benchmarks sous charge Locust.

Cadre Général du Projet

Introduction

Ce premier chapitre pose le cadre méthodologique, industriel et opérationnel de notre projet de fin d'études. Nous commençons par présenter l'organisme d'accueil, **Designet Web Agency**, ainsi que son positionnement sur le marché des technologies de l'information. Nous décrivons ensuite le contexte industriel marqué par l'expansion exponentielle des cybermenaces ciblant les surfaces d'attaque externes. Nous dressons un état des lieux critique des procédures d'audit traditionnelles et une analyse comparative approfondie des solutions du marché, avant de formaliser la problématique, la solution proposée, les objectifs généraux et spécifiques ainsi que le périmètre d'application du projet. Enfin, nous détaillons la méthodologie Agile Scrum retenue, ses artéfacts, et le calendrier complet des réalisations.

1.1 Présentation de l'Organisme d'Accueil

1.1.1 Identité et Domaines d'Activité

Designet Web Agency est une agence d'ingénierie logicielle, d'infrastructures Cloud et de cybersécurité basée à Menzel Temime (Gouvernorat de Nabeul, Tunisie). Fondée par une équipe d'ingénieurs passionnés, la société s'est imposée comme un partenaire technologique privilégié auprès d'entreprises locales et internationales (PME, grands comptes, institutions financières et collectivités). Elle emploie une vingtaine de collaborateurs répartis entre développeurs full-stack, ingénieurs systèmes et réseaux, consultants en cybersécurité et experts DevOps.



Figure 1.1 : Logo Officiel de l'Organisme d'Accueil – Designet Web Agency

Les activités principales de la société s'articulent autour de quatre pôles d'excellence :

- **Ingénierie Logicielle & Web Avancé :** Conception et réalisation d'architectures applicatives complexes sous des frameworks modernes (Laravel, Node.js, Python), intégrant des exigences élevées de montée en charge et d'ergonomie utilisateur.
- **Administration Système & Cloud DevOps :** Déploiement, orchestration de conteneurs (Docker, Kubernetes) et automatisation de pipelines d'intégration et de livraison continues.
- **Audit & Conseil en Cybersécurité :** Réalisation de tests d'intrusion (*pentesting*), revues de code source, évaluations de conformité et réponse à incidents.
- **Solutions Métier et R&D :** Développement d'outils internes visant à automatiser les processus opérationnels et à réduire les délais de livraison des prestations d'audit.

1.1.2 Organigramme de l'Entreprise

La figure 1.2 illustre la structure organisationnelle de Designet Web Agency. L'agence est dirigée par une Direction Générale chapeautant la Direction Technique (CTO) et la Direction Opérationnelle.

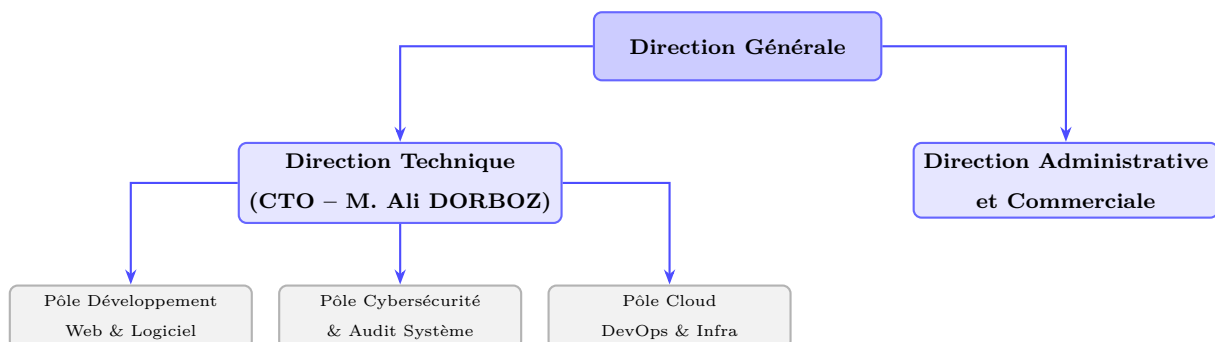


Figure 1.2 : Organigramme Fonctionnel de Designet Web Agency

1.1.3 Cadre Institutionnel du Stage

Ce Projet de Fin d'Études (PFE) s'inscrit dans le cadre de l'obtention du Diplôme National d'Ingénieur en Sciences Appliquées et Technologiques, spécialité Sécurité des Systèmes Informatiques et Réseaux (SSIR), au sein de l'École Supérieure Privée de Technologies et d'Ingénierie (**TEK-UP**).

Le stage s'est déroulé sur une période de cinq mois, du **2 février 2026 au 2 juillet 2026**. L'encadrement industriel a été assuré par **M. Ali DORBOZ**, Directeur Technique (CTO) de Designet Web Agency, tandis que l'encadrement pédagogique a été confié à **Mme Sonia BEN AISSA**, Enseignante-chercheuse à l'École TEK-UP.

1.2 Contexte Industriel et Motivations

La généralisation des modèles d'infrastructure hybrides et multi-cloud a fragmenté les périmètres réseaux des organisations. Contrairement aux modèles informatiques classiques où la sécurité reposait sur une défense périmétrique cloisonnée (*Castle-and-Moat*), les entreprises modernes exploitent des centaines d'actifs exposés : applications web, interfaces de programmation (API), passerelles distantes, sous-domaines temporaires de test et services SaaS.

Dans ce contexte, la gestion continue de la surface d'attaque externe (*External Attack Surface Management* – EASM) répond à un impératif stratégique : découvrir et auditer en permanence l'ensemble des actifs accessibles depuis l'Internet public avant qu'un attaquant ne puisse identifier et exploiter une faille. Selon les observations récentes du secteur, une vulnérabilité critique est souvent exploitée par des acteurs malveillants dans les heures suivant sa divulgation, ce qui rend obsolètes les audits manuels espacés dans le temps.

1.3 Étude de l'Existant et Analyse Critique

1.3.1 Procédure d'Audit Traditionnelle

L'analyse des pratiques d'audit observées chez Designet Web Agency et dans l'industrie révèle une forte dépendance à des processus manuels séquentiels :

1. L'auditeur exécute manuellement un ensemble d'outils CLI sur son terminal local (Nmap pour la cartographie des ports, Subfinder pour l'OSINT, Gobuster pour l'énumération web, Nuclei pour les signatures de vulnérabilités).
2. Chaque outil génère des fichiers de sortie bruts hétérogènes (XML, JSON, CSV ou texte brut non structuré).
3. L'auditeur lit, filtre et interprète manuellement ces données pour éliminer les faux positifs.

4. Les résultats sont ensuite transcrits manuellement dans un document traitement de texte ou un tableur afin de rédiger le rapport d’expertise remis au client.

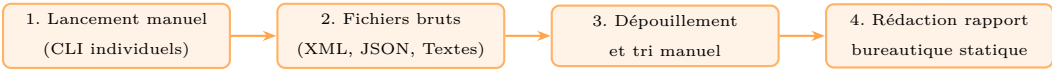


Figure 1.3 : Processus Séquentiel d’Audit Traditionnel et Frictions Opérationnelles Associées

1.3.2 Limites de la Démarche Traditionnelle

Ce mode opératoire présente cinq limites fondamentales :

- **Lenteur et coût opérationnel élevé :** Le temps consacré à la saisie et au filtrage manuel réduit le temps réellement dédié à l’analyse approfondie des risques.
- **Absence de corrélation transverse :** Les outils fonctionnant en silos, il est très difficile de visualiser les liens de dépendance entre un domaine, une adresse IP, un port, un service applicatif et une faille sous-jacente.
- **Latence de la remédiation (MTTR élevé) :** La rédaction manuelle des procédures de durcissement ralentit le traitement des incidents.
- **Risque de faux positifs et d’incohérences :** L’absence de normalisation automatique augmente le risque d’erreurs d’interprétation humaine.
- **Manque d’encadrement légal intégré :** Aucun mécanisme automatisé n’empêche l’exécution d’un scan sur une cible pour laquelle aucun mandat formel n’a été établi.

1.3.3 Comparatif des Solutions Existantes sur le Marché

Pour positionner notre projet, nous avons analysé les solutions logicielles d’audit et d’ASM existantes, qu’elles soient open-source ou commerciales. Le tableau 1.1 synthétise cette étude comparative.

Table 1.1 : Analyse Comparative des Solutions d’Audit et de Gestion de Surface d’Attaque

Critère d’Évaluation	Shodan	OWASP Amass	SpiderFoot	Intruder.io	Recon-ng	Notre Plateforme
Type d’Audit	Passif uniquement	Passif / Actif DNS	Passif / OSINT	Actif Cloud	Passif OSINT	Passif + Actif Unifié
Modélisation en Graphe	Non	Graphe local (d3)	Graphe basique	Non	Non	PostgreSQL + Apache AGE
Calcul de Rayon d’Impact	Non	Non	Non	Non	Non	Oui (BFS NetworkX)
Génération Remediation IA	Non	Non	Non	Recommandations fixes	Non	Oui (LLM Local Contraint)
Contrôle Légal (PoA requis)	Non	Non	Non	Manuel (déclaratif)	Non	Bloquant Logiciel Obligatoire
Bac à Sable (Sandbox Isolée)	Non	Non	Non	Non	Non	Oui (Docker Socket Proxy)
Chaîne DevSecOps Intégrée	Non	Non	Non	Non	Non	Oui (SAST/SCA/SBOM/Cosign)
Modèle de Déploiement	SaaS Externe	CLI Local	Web Auto-hébergé	SaaS Propriétaire	CLI Local	Web Auto-hébergé / VPS
Confidentialité des Données	Données publiques	Locale	Locale	Données Cloud tiers	Locale	Traitement 100 % Local

1.4 Problématique

À la lumière des limites identifiées et des lacunes des solutions existantes, la problématique centrale de ce travail d'ingénierie s'énonce ainsi :

« Comment concevoir et développer une plateforme web unifiée capable d'orchestrer automatiquement les outils de reconnaissance de sécurité, de corrélérer les constats hétérogènes via un graphe de connaissances, et d'assister la décision grâce à une IA locale contrainte dans un cadre légal et DevSecOps rigoureux ? »

1.5 Solution Proposée

Pour répondre à cette problématique, nous proposons la réalisation d'une **plateforme web semi-agentique de reconnaissance de surface d'attaque**. La solution s'appuie sur une architecture microservices découplée associant :

- Un portail web applicatif développé sous **Laravel 11** assurant l'authentification sécurisée, le contrôle d'accès RBAC granulaire, la gestion des projets et le rendu ergonomique.
- Cinq microservices spécialisés en **Python Flask** (reconnaissance multi-outils, sécurité offensive / sandbox, renseignement passif OSINT, moteur IA contraint, passerelle d'API).
- Un moteur d'exécution asynchrone piloté par **Redis Streams** garantissant la résilience, la tolérance aux pannes et le passage à l'échelle.
- Une base de données unifiée sous **PostgreSQL 16** avec l'extension **Apache AGE** permettant la persistance relationnelle et le requêtage openCypher du graphe de connaissances.
- Un moteur d'inférence locale (**Ollama**) hébergeant le modèle **qwen2.5-coder** pour la synthèse contextuelle et la production de code de remédiation (*Remediation-as-Code*).
- Une chaîne **DevSecOps** automatisée sous GitHub Actions assurant la sécurité intrinsèque du code source et des conteneurs de production.

1.6 Objectifs du Projet

1.6.1 Objectif Général

L'objectif général consiste à **automatiser et sécuriser l'intégralité du cycle de reconnaissance et d'audit de surface d'attaque**, de l'ingestion d'une cible légalement autorisée jusqu'à la génération d'un rapport certifié enrichi d'une cartographie relationnelle et de scripts de remédiation directement exploitables.

1.6.2 Objectifs Spécifiques

Cet objectif principal se structure en six objectifs spécifiques :

1. **Orchestration automatisée multi-scanners** : Encapsuler Nmap, Nuclei, Gobuster, Subfinder et WPScan dans des adaptateurs modulaires avec gestion de profils d'intensité (Silent, Balanced, Aggressive) et injection de délais aléatoires (*jitter*).
2. **Normalisation et dédoublonnage universel** : Développer un module (*ResultParser*) transformant les sorties disparates en entités normalisées *Finding* avec score CVSS et références CVE/CWE.
3. **Modélisation en graphe et calcul de blast radius** : Représenter la topologie sous forme de sommets et d'arêtes typées, et évaluer le rayon d'impact des failles via l'algorithme BFS sous NetworkX.
4. **Assistance IA locale contrainte** : Intégrer un LLM local opérant sans dépendance à un fournisseur externe pour contribuer à préserver la confidentialité des données, avec réduction du risque d'hallucination via des citations obligatoires vers les logs bruts.
5. **Encadrement légal et bac à sable isolé** : Imposer la validation d'une preuve d'autorisation (PoA) pour chaque projet et piloter les bacs à sable vulnérables (DVWA, SQLi-Labs) via un proxy sécurisé filtrant les commandes dangereuses.
6. **Sécurité intrinsèque DevSecOps** : Appliquer une chaîne de validation logicielle bloquante (SAST, SCA, SBOM CycloneDX, scans Trivy et signature Cosign).

1.7 Périmètre du Projet (Scope)

1.7.1 Périmètre Inclus

- Renseignement passif OSINT (DNS structuré, certificats SSL/TLS via `crt.sh`, requêtes WHOIS).
- Découverte active des ports, détection des services réseau et versions logicielles associées.
- Évaluation automatisée des vulnérabilités applicatives par modèles YAML et analyse de CMS (WordPress).
- Tests d'injections applicatives non destructives (SQL, commandes, XSS).
- Pilotage de conteneurs vulnérables isolés pour la validation des techniques d'audit.
- Visualisation dynamique du graphe et export des rapports d'expertise (PDF signé, HTML, JSON).

1.7.2 Périmètre Exclu (par conception)

- Exploitation offensive destructrice ou maintien d'accès persistant (*backdoors*) sur les cibles.
- Attaques par déni de service (DoS / DDoS) ou saturation intentionnelle des liens réseau.
- Exfiltration de bases de données sensibles ou altération de contenus sur des serveurs tiers.

1.8 Méthodologie de Travail et Conduite du Projet

1.8.1 Justification de la Démarche Agile

La complexité inhérente à l'intégration de multiples outils d'audit hétérogènes, de technologies de bases de données hybrides et de modèles d'IA justifie le recours à une démarche Agile. Contrairement aux cycles de développement traditionnels en cascade (*Waterfall*) où les spécifications sont figées dès le départ, la méthodologie Agile autorise une réévaluation continue des priorités techniques au fur et à mesure des résultats d'expérimentation.

1.8.2 Le Cadre Méthodologique Scrum

Nous avons adopté le cadre **Scrum**, formalisé par des rôles, des cérémonies et des artefacts clairement définis pour assurer une cadence de livraison régulière :

1.8.2.1 Description des Rôles Scrum

Le tableau 1.2 détaille la répartition des responsabilités au sein de l'équipe projet :

Table 1.2 : Répartition et Responsabilités des Rôles Scrum sur le Projet

Rôle Scrum	Titulaire	Responsabilités Opérationnelles
Product Owner (PO)	M. Ali DORBOZ (CTO)	Définition de la vision produit, priorisation du Product Backlog, validation des User Stories et recette des livrables de fin de sprint.
Scrum Master (SM)	Aymen AZIZI (Étudiant)	Animation des rituels Scrum, élimination des obstacles techniques, suivi de la vélocité et maintien des normes de qualité.
Development Team	Aymen AZIZI (Étudiant)	Conception technique, développement back-end (Laravel/Flask), intégration de l'IA (Ollama), tests et déploiement DevSecOps.
Encadrement Académique	Mme Sonia BEN AISSA (TEK-UP)	Suivi pédagogique, validation méthodologique et contrôle de conformité académique.

1.8.2.2 Artefacts et Cérémonies Scrum

- **Artefacts Scrum :** Gestion du *Product Backlog* dans GitHub Projects, décomposition en *Sprint Backlogs*, et formalisation de critères d'acceptation rigoureux (*Definition of Done* – DoD : code testé unitairement, documenté, validé par les gates DevSecOps et validé en environnement de développement).
- **Cérémonies Scrum :** Séances de *Sprint Planning* au début de chaque itération, réunions d'alignement hebdomadaires (*Weekly Sync*), et revues / rétrospectives de fin de sprint.

1.8.3 Planification des Sprints et Calendrier Global

Le projet s’est articulé autour de 6 Sprints successifs de deux à trois semaines, suivis d’une phase finale de recette et de validation expérimentale sur la période du 2 février au 2 juillet 2026.

Table 1.3 : Planification Détaillée des Sprints Scrum et Calendrier des Réalisations

Sprint / Phase	Période	Livrables Majeurs	Revue et Validation
Sprint 1	2 fév. – 22 fév.	Cahier des charges, modélisation UML/ERD, socle Laravel 11 et Flask	Validation de l’architecture avec le CTO
Sprint 2	23 fév. – 15 mar.	Microservice reconnaissance (Nmap, Nuclei, Gobuster, Subfinder, WPScan)	Démonstration d’exécution des scanners
Sprint 3	16 mar. – 5 avr.	Microservice sécurité, tests d’injections, proxy sandbox Docker	Validation des tests sur DVWA et SQLi-Labs
Sprint 4	6 avr. – 26 avr.	Microservice IA (Ollama), normalisation des failles, Remediation-as-Code	Évaluation de la pertinence des scripts IA
Sprint 5	27 avr. – 17 mai	IHM web (dashboard, scans, rapports, graphe Cytoscape.js, OSINT)	Validation de l’ergonomie utilisateur
Sprint 6	18 mai – 1 ^{er} juin	Passerelle API, worker Redis Streams, durcissement TLS et déploiement	Déploiement de l’instance de production
Recette & Clôture	2 juin – 2 juil.	Tests de charge Locust, audits Trivy/Cosign, documentation et rapport de PFE	Validation finale du projet

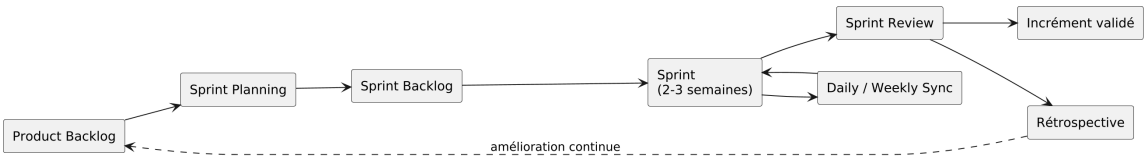


Figure 1.4 : Vue d’Ensemble du Processus Agile Scrum Appliqué au Projet

Conclusion

Ce premier chapitre a permis d’exposer le cadre général du projet, en démontrant par une analyse comparative approfondie les lacunes des outils existants et l’intérêt d’une approche unifiée semi-agentique. Nous avons défini la problématique, les objectifs, les périmètres et la méthodologie de conduite Agile Scrum. Le chapitre suivant développe l’état de l’art scientifique et l’étude comparative des technologies de référence.

État de l'Art et Étude Comparative

Introduction

Ce deuxième chapitre établit les fondements théoriques, méthodologiques et technologiques nécessaires à la conception de notre plateforme. Nous commençons par exposer les principes fondamentaux de la cybersécurité et analysons les dix risques majeurs du référentiel OWASP Top 10. Nous développons ensuite les concepts clés de la gestion continue de la surface d'attaque (*Attack Surface Management* – ASM) et présentons en détail les mécanismes internes des outils de reconnaissance de référence. Par la suite, nous approfondissons le paradigme DevSecOps, la sécurité de la chaîne logicielle, l'apport de l'intelligence artificielle locale contrainte et les architectures microservices événementielles. Enfin, nous menons une étude comparative systématique qui justifie rationnellement chaque choix technologique au regard des exigences du projet.

2.1 Fondements de la Cybersécurité

2.1.1 Définition et Enjeux Contemporains

La cybersécurité regroupe l'ensemble des technologies, processus et mesures de contrôle conçus pour protéger les systèmes informatiques, les réseaux, les programmes et les données contre les cyberattaques. Selon l'Union Internationale des Télécommunications (UIT) [4], l'expansion du numérique s'accompagne d'une professionnalisation accrue des menaces. Les organisations modernes, dont les activités reposent sur des infrastructures interconnectées et exposées sur Internet, sont régulièrement la cible d'attaques visant le vol de propriété intellectuelle, l'interruption d'activité ou l'extorsion de fonds. Cybersecurity Ventures [5] estime le coût annuel mondial de la cybercriminalité à plusieurs milliers de milliards de dollars, soulignant l'obligation pour les entreprises de déployer des approches d'audit continues et proactives.

2.1.2 Piliers Fondamentaux de la Sécurité

La posture de sécurité d'un système d'information repose traditionnellement sur la triade **CIA** (Confidentialité, Intégrité, Disponibilité) et ses principes directeurs associés :

- **Confidentialité** : Garantie que les informations ne sont divulguées qu'aux personnes, processus ou systèmes dûment autorisés. Sa mise en œuvre repose sur des mécanismes de chiffrement fort en transit (TLS 1.3) et au repos (AES-256), associés à un contrôle d'accès strict.
- **Intégrité** : Préservation de l'exactitude, de la complétude et de la non-altération des données tout au long de leur cycle de vie, garantie par des signatures numériques et des fonctions de hachage cryptographique (SHA-256).
- **Disponibilité** : Maintien de l'accessibilité opérationnelle et de la continuité de service pour les utilisateurs légitimes face aux pannes matérielles, aux congestions réseau ou aux attaques volumétriques.
- **Authentification et Autorisation** : Processus à deux facteurs consistant d'abord à prouver formellement l'identité revendiquée (*Authentification*), puis à valider les privilèges attribués selon le principe du moindre privilège (*Autorisation*).
- **Traçabilité et Non-répudiation** : Capacité d'attribuer sans équivoque toute action ou transaction à une entité donnée, empêchant la contestation ultérieure grâce à des journaux d'audit immuables et horodatés.

2.2 Le Référentiel OWASP Top 10

L'*Open Worldwide Application Security Project* (OWASP) [6] est une autorité de référence internationale en sécurité applicative. Son classement décennal **OWASP Top 10** synthétise les risques de sécurité les plus répandus et les plus critiques affectant les architectures web :

- **A01 :2021 – Rupture du Contrôle d'Accès (Broken Access Control)** : Défaillances dans l'application des restrictions d'accès permettant à des utilisateurs non privilégiés d'accéder à des ressources protégées ou d'altérer des enregistrements appartenant à des tiers (failles IDOR, contournement de contrôle RBAC).
- **A02 :2021 – Défaillances Cryptographiques (Cryptographic Failures)** : Utilisation d'algorithmes obsolètes (MD5, SHA1), absence de chiffrement des données sensibles ou transmission non sécurisée en clair sur le réseau.
- **A03 :2021 – Injections (Injection)** : Défaut de neutralisation des données fournies par l'utilisateur, permettant d'exécuter des instructions arbitraires dans un interpréteur (injections SQL, injection de commandes système, Cross-Site Scripting).
- **A04 :2021 – Conception Non Sécurisée (Insecure Design)** : Faiblesses architecturales résultant de l'absence de modélisation des menaces et d'application des principes de conception sécurisée dès l'amont du projet.

- **A05 :2021 – Mauvaise Configuration de Sécurité (Security Misconfiguration) :** Services ou ports superflus activés, messages d'erreurs verbeux révélant des traces de pile, identifiants par défaut non modifiés ou absence d'en-têtes de sécurité (HSTS, CSP).
- **A06 :2021 – Composants Vulnérables et Obsolètes :** Utilisation de bibliothèques, dépendances logicielles ou modules tiers contenant des failles documentées dans les bases de vulnérabilités publiques (CVEs).
- **A07 :2021 – Défaillances d'Identification et d'Authentification :** Vulnérabilités facilitant le bourrage d'identifiants (*credential stuffing*), l'énumération de comptes ou le détournement de session.
- **A08 :2021 – Manque d'Intégrité des Données et du Logiciel :** Risques liés à la désérialisation d'objets non sécurisée ou à l'inclusion de composants logiciels non vérifiés dans les chaînes CI/CD.
- **A09 :2021 – Défaillances de Journalisation et de Surveillance :** Absence ou insuffisance de journaux d'audit, retardant la détection des intrusions et rendant impossibles les investigations médico-légales.
- **A10 :2021 – Falsification de Requête Côté Serveur (SSRF) :** Vulnérabilité permettant à un attaquant d'inciter l'application côté serveur à émettre une requête HTTP/TCP vers une ressource interne non exposée.

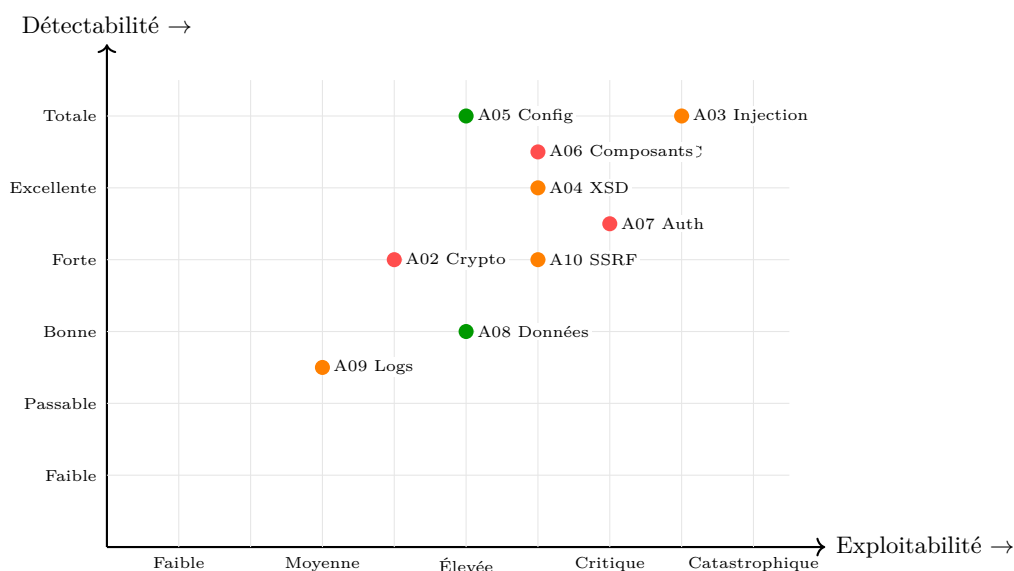


Figure 2.1 : Matrice des Risques OWASP Top 10 (2021) – Exploitabilité vs Détectabilité

2.3 Gestion Continue de la Surface d'Attaque (ASM)

2.3.1 Définition et Principes

La surface d'attaque d'une organisation représente l'ensemble cumulé des points d'entrée et vecteurs d'exposition par lesquels un attaquant potentiel peut tenter d'infiltrer un réseau, de corrompre un système ou d'exfiltrer des données. L'*Attack Surface Management* (ASM) est la discipline d'ingénierie consistant à identifier, cartographier, analyser et réduire en continu cette surface d'exposition numérique externe.

2.3.2 Cycle de Vie du Processus ASM

Une démarche ASM mature s'organise selon un cycle itératif structuré en six phases interdépendantes :

1. **Découverte (Discovery)** : Recherche et détection exhaustive de tous les actifs exposés sur Internet associés à l'organisation (domaines, sous-domaines, plages IP, services Cloud, certificats).
2. **Inventaire (Inventory)** : Structuration et centralisation des données d'actifs au sein d'un référentiel d'inventaire unifié.
3. **Classification (Classification)** : Évaluation de la nature des actifs, de leur niveau d'exposition et de leur criticité métier.
4. **Audit et Analyse (Assessment)** : Exécution de scans techniques ciblés pour détecter les vulnérabilités, faiblesses applicatives et défauts de configuration.
5. **Priorisation (Prioritization)** : Évaluation du rayon d'impact (*blast radius*) et corrélation relationnelle pour hiérarchiser les alertes en fonction de la menace réelle.
6. **Remédiation (Remediation)** : Déploiement de correctifs logiciels, durcissement des règles de filtrage et validation post-remédiation.

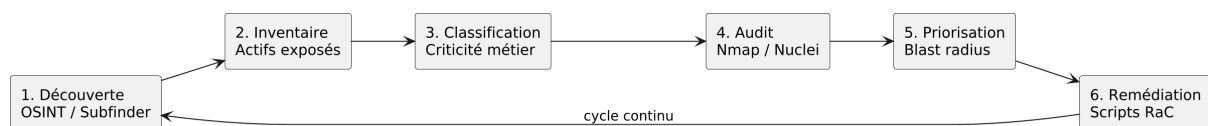


Figure 2.2 : Cycle de Vie du Processus d'Attack Surface Management (ASM)

2.4 Reconnaissance et Outils d’Audit de Sécurité

2.4.1 Rôle et Typologie de la Reconnaissance

La reconnaissance constitue la première phase incontournable de tout audit technique de sécurité. Elle se décompose en deux grandes modalités :

- **Reconnaissance Passive (OSINT)** : Collecte d’informations sans interaction réseau directe avec l’infrastructure de la cible (interrogation des registres WHOIS, des serveurs DNS publics, des registres de transparence de certificats `crt.sh`). Elle garantit une discrétion opérationnelle totale.
- **Reconnaissance Active** : Émission de paquets réseau et de requêtes applicatives directement vers les hôtes cibles (balayage de ports SYN, requêtes HTTP/HTTPS, énumération de répertoires). Elle fournit une cartographie technique exacte des services et versions exposés.

2.4.2 Outils d’Audit Intégrés dans la Plateforme

Notre plateforme intègre cinq outils de reconnaissance et d’audit de sécurité de référence, encapsulés au sein de conteneurs isolés :

- **[Nmap] – Network Mapper [8]** : Outil d’exploration réseau mondialement reconnu. Il utilise l’émission de paquets TCP SYN bruts (*SYN stealth scan*) pour déterminer l’état des ports, identifie les bannières applicatives et détecte les systèmes d’exploitation via l’analyse de signatures TCP/IP. Son moteur NSE (*Nmap Scripting Engine*) permet d’exécuter des scripts Lua pour évaluer des vulnérabilités connues.
- **[Nuclei] – Vulnerability Scanner [9]** : Moteur d’évaluation de vulnérabilités piloté par des modèles déclaratifs YAML communautaires. Conçu en langage Go, il excelle dans la détection rapide de failles web complexes (CVEs récentes, faiblesses d’authentification, fuites de données d’infrastructure) en envoyant des requêtes ciblées avec un taux de faux positifs inférieur aux scanners génériques.
- **[Gobuster] – Content & DNS Enumeration [10]** : Outil d’énumération brute rapide développé en Go exploitant des routines concurrentes. Il permet d’identifier les chemins, répertoires cachés, fichiers sensibles d’environnement (`.env`, `.git`, sauvegardes `.bak`) et sous-domaines par confrontation à des dictionnaires volumineux (*SecLists*).
- **[Subfinder] – Passive OSINT Subdomains [11]** : Outil d’énumération passive de sous-domaines. Il agrège les données issues de dizaines de sources passives ouvertes sans jamais contacter directement la cible, assurant la découverte exhaustive des périmètres oubliés.

- **[WPScan] – WordPress Vulnerability Scanner [12]** : Scanner spécialisé dans l'audit des systèmes WordPress. Il analyse le code source HTML, les en-têtes HTTP et les répertoires standards pour recenser les thèmes et extensions obsolètes répertoriés dans la base de données WPScan Vulnerability Database.

2.5 DevSecOps et Sécurité de la Chaîne Logicielle

2.5.1 Origine et Philosophie Shift-Left

Le modèle traditionnel de développement logiciel réservait les contrôles de sécurité à la fin du cycle de livraison, juste avant le passage en production. Cette approche tardive engendrait des goulots d'étranglement opérationnels majeurs : la correction d'une faille architecturale ou d'une dépendance vulnérable découverte tardivement entraînant des coûts prohibitifs et des retards de déploiement importants.

Le paradigme **DevSecOps** [7] propose de fusionner le développement (*Dev*), la sécurité (*Sec*) et les opérations (*Ops*) en intégrant des contrôles automatisés à chaque étape du cycle de vie logiciel. Cette philosophie, qualifiée de **Shift-Left Security**, consiste à déplacer les vérifications de sécurité le plus tôt possible dans la chaîne de fabrication logicielle, permettant aux développeurs d'identifier et de corriger les vulnérabilités dès la phase d'écriture du code source.

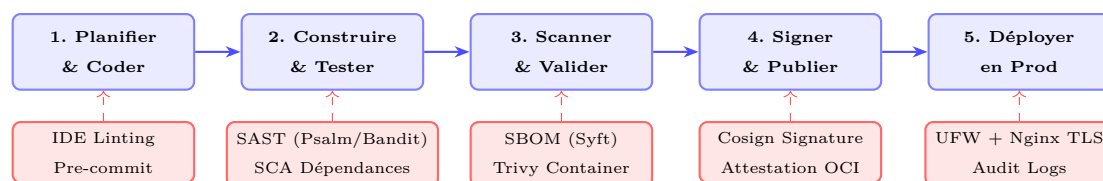


Figure 2.3 : Cycle de Livraison DevSecOps : Intégration Continue des Portes de Sécurité (Shift-Left)

2.5.2 Sécurité de la Chaîne d'Approvisionnement Logicielle (Software Supply Chain)

Dans les applications modernes, plus de 80 % du code déployé provient de dépendances open-source et de conteneurs tiers. Les attaquants ciblent désormais fréquemment la chaîne d'approvisionnement logicielle (*Supply Chain Attacks*) en injectant du code malveillant dans des bibliothèques populaires. Pour répondre à ce risque, le cadre DevSecOps déploie cinq mécanismes de défense complémentaires :

- **SAST (Static Application Security Testing)** : Analyse statique automatisée du code source au repos sans l'exécuter. Elle révèle les failles d'implémentation (injections, déréférencements de pointeurs nuls, gestion de secrets en clair). Dans notre environnement, **Psaln** [24] analyse le

code PHP/Laravel et **Bandit** [25] audite le code Python/Flask.

- **SCA (Software Composition Analysis)** : Analyse de la composition logicielle identifiant les dépendances tierces obsolètes ou affectées par des CVEs connues répertoriées dans la base NVD.
- **SBOM (Software Bill of Materials)** : Inventaire exhaustif, standardisé et lisible par machine de l’ensemble des composants logiciels, versions et licences intégrés à l’artefact de livraison. Nous exploitons l’outil **Syft** [26] pour générer des fichiers SBOM au standard ouvert **CycloneDX**.
- **Scan d’Images de Conteneurs** : Analyse multicouche des images Docker permettant d’identifier les vulnérabilités présentes dans le système d’exploitation sous-jacent et les packages système via l’analyseur de référence **Trivy** [27].
- **Signature Cryptographique de Conteneurs** : Application d’une signature numérique immuable sur les images conteneurisées validées grâce à **Cosign (Sigstore)** [28]. Seules les images dûment signées et vérifiées sont autorisées à être déployées sur le serveur de production.

2.6 Intelligence Artificielle Locale et Grands Modèles de Langage (LLM)

2.6.1 Apport des LLM dans l’Audit de Sécurité

L’intégration des grands modèles de langage (*Large Language Models* – LLM) dans les plateformes de sécurité apporte une valeur ajoutée décisive : capacité de synthétiser des volumes massifs de journaux techniques pour les décideurs managériaux, explication vulgarisée du mécanisme des failles détectées, et génération automatisée de scripts durcis de remédiation (*Remediation-as-Code*).

2.6.2 Impératif de Confidentialité et Modèles Contraints

Le recours à des API d’IA hébergées dans le Cloud public (telles qu’OpenAI ou Anthropic) soulève des problèmes majeurs de confidentialité industrielle : les vulnérabilités critiques d’infrastructures sensibles et les adresses IP d’actifs audités seraient transmises à des serveurs tiers.

Pour résoudre ce défi, nous avons fait le choix stratégique d’un moteur d’inférence local (**Ollama**) hébergeant le modèle **qwen2.5-coder**. Cette approche contribue à préserver la confidentialité des données en évitant leur transmission à un fournisseur externe de LLM. De plus, le microservice contribue à réduire le risque d’hallucination inhérent aux modèles génératifs en imposant un gabarit de prompt système (*system prompt*) contraignant la sortie à un format JSON strict et exigeant des références explicites aux numéros de lignes des journaux bruts d’audit.

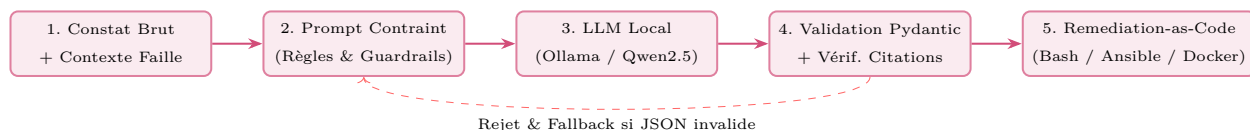


Figure 2.4 : Pipeline de l'IA Locale Contrainte pour la Production de Remediation-as-Code

2.7 Architectures Microservices et Systèmes Événementiels

L'architecture microservices décompose une application complexe en un ensemble de services autonomes, indépendamment déployables et hautement spécialisés, communiquant via des protocoles réseau légers.

Pour une plateforme d'audit où les tâches de balayage (scans Nmap, Gobuster) peuvent durer de plusieurs secondes à plusieurs dizaines de minutes, une architecture purement synchrone HTTP conduirait inévitablement à des blocages de requêtes (*timeouts*). L'adoption d'un modèle asynchrone piloté par événements via **Redis Streams** permet de découpler le portail web de l'exécution des scanners : les requêtes d'audit sont insérées dans un flux persistant, puis consommées par des processus dédiés (*workers*) capables de gérer les réessais automatiques avec repli exponentiel.

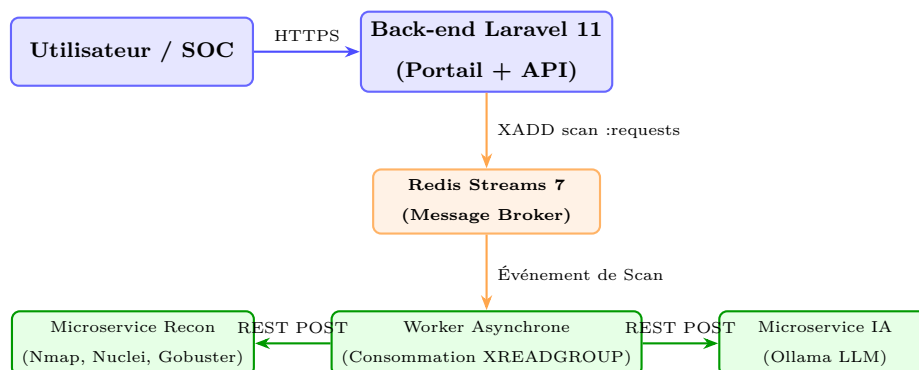


Figure 2.5 : Patron d'Architecture Microservices et Découplage Asynchrone via Redis Streams

2.8 Étude Comparative et Justification des Choix Technologiques

Afin de garantir la robustesse et la pérennité du système, nous avons mené des études comparatives systématiques sur chaque brique technologique.

2.8.1 Comparatif des Frameworks Backend Web

Table 2.1 : Analyse Comparative des Frameworks Web Backend

Critère	Laravel 11 (PHP)	Django 5 (Python)	Spring Boot (Java)	Express.js (Node)
ORM & Modélisation	Eloquent (très intuitif)	Django ORM (puissant)	Hibernate / JPA (complexe)	Prisma / TypeORM
Système de Files / Queues	Natif (Redis, DB)	Celery (externe)	Spring JMS / RabbitMQ	BullMQ (externe)
Authentification & RBAC	Sanctum + Spatie Permission	Intégré	Spring Security (lourd)	Passport.js (manuel)
Moteur de Templates	Blade (performant)	Django Templates	Thymeleaf	EJS / Pug
Courbe d'apprentissage	Modérée	Modérée	Élevée	Faible
Expertise Organisme	Élevée (Designet Web)	Modérée	Faible	Modérée
Retenu	Oui	Non	Non	Non

Justification du Choix de Laravel 11 : Laravel 11 a été retenu car il offre une gestion native et éprouvée de l'authentification et du contrôle d'accès RBAC via Spatie Permission, un ORM Eloquent permettant de manipuler de façon expressive les 14 tables du modèle, et une intégration directe des files de messages Redis. Enfin, ce choix capitalise sur l'expertise forte de Designet Web Agency, garantissant la maintenabilité à long terme.

2.8.2 Comparatif des Frameworks de Microservices Python

Table 2.2 : Analyse Comparative des Frameworks de Microservices

Critère	Flask	FastAPI	Django REST	Nameko
Empreinte mémoire	Très légère (~100 Mo)	Légère (~120 Mo)	Lourde (~300 Mo)	Modérée (~200 Mo)
Gestion des subprocess	Contrôle total synchrone	Async / Subprocess	Bloquant	Basée sur workers
Intégration outils CLI	Directe et robuste	Très bonne	Modérée	Modérée
Courbe d'apprentissage	Très faible	Modérée	Élevée	Élevée
Retenu	Oui	Non	Non	Non

Justification du Choix de Flask : Flask a été sélectionné pour sa légèreté, sa stabilité et son contrôle absolu sur les processus système (`subprocess.Popen`) encapsulant Nmap, Nuclei et Gobuster. L'exécution d'outils CLI lourds s'accommode idéalement du modèle synchrone maîtrisé de Flask au sein de conteneurs dédiés.

2.8.3 Comparatif des Moteurs d'IA Locale (LLM)

Table 2.3 : Analyse Comparative des Moteurs d'IA Locale

Critère	Ollama	LM Studio	LocalAI	vLLM
Licence Open Source	Oui (MIT)	Propriétaire	Oui (MIT)	Oui (Apache 2.0)
Intégration Docker	Image officielle durcie	Application bureau	Image officielle	Complexe (CUDA)
Mode Serveur Headless	Natif (API REST :11434)	Limité	Natif	Natif
Gestion des modèles	Intégrée (pull, run)	Manuelle	Manuelle	Manuelle
Retenu	Oui	Non	Non	Non

Justification du Choix d'Ollama et de Qwen2.5-Coder : Ollama s'intègre parfaitement dans un environnement Docker Compose en mode conteneurisé sans interface graphique. Le modèle `qwen2.5-coder` offre un ratio performances/taille optimal pour la production de code système (Bash, Ansible, Dockerfile) et respecte fidèlement les structures JSON imposées.

2.8.4 Comparatif de la Base de Données et du Moteur de Graphe

Table 2.4 : Analyse Comparative des Solutions de Base de Données et Graphes

Critère	PostgreSQL 16 + Apache AGE	MySQL + Neo4j	MongoDB
Architecture	Instance unifiée (Relationnel + Graphe)	2 serveurs distincts	Base documentaire seule
Langage Graphe	openCypher natif	Cypher (Neo4j)	Agrégations complexes
Support JSONB	Excellent (Index GIN)	Modéré	Natif
Empreinte mémoire	Réduite (une seule instance DB)	Élevée (JVM Neo4j + MySQL)	Modérée
Retenu	Oui	Non	Non

Justification du Choix de PostgreSQL 16 avec Apache AGE : L'extension Apache AGE permet de manipuler des requêtes relationnelles SQL et des graphes openCypher au sein d'une seule et même instance PostgreSQL. Cette architecture évite la redondance et la surcharge mémoire liées au déploiement d'un moteur de graphe dédié sous JVM (comme Neo4j).

2.8.5 Comparatif du Message Broker pour Traitement Asynchrone

Table 2.5 : Analyse Comparative des Systèmes de Message Broker

Critère	Redis 7 (Streams)	RabbitMQ	Apache Kafka
Persistance	Oui (AOF / RDB)	Oui	Oui (Partitionnée)
Groupes de Consommateurs	Natif (XREADGROUP)	Natif	Natif
Empreinte mémoire	Très faible (<50 Mo)	Modérée (~150 Mo)	Très lourde (JVM + Zookeeper/Kraft)
Usage multifonction	Broker + Cache + Sessions	Broker uniquement	Streaming uniquement
Retenu	Oui	Non	Non

Justification du Choix de Redis 7 Streams : Redis 7 offre à la fois un rôle de Message Broker avec persistance et acquittement (XACK), de cache de requêtes IHM et de gestionnaire de sessions, réduisant ainsi le nombre de composants d'infrastructure requis.

2.8.6 Synthèse Globale des Choix Technologiques

Le tableau 2.6 récapitule l'ensemble des choix technologiques adoptés pour la réalisation de la plateforme avec leur justification stratégique.

Table 2.6 : Tableau Récapitulatif de la Pile Technologique Retenue

Composant / Couche	Technologie Retenue	Justification Stratégique
Portail Web Backend	Laravel 11 (PHP 8.3)	ORM Eloquent expressif, RBAC Spatie, queues Redis natives.
Microservices d'Audit	Flask 3.1 (Python 3.12)	Légèrement conteneurisé, contrôle absolu sur les sous-processus CLI.
Base de Données	PostgreSQL 16 + Apache AGE	Instance unifiée supportant à la fois SQL relationnel et graphe openCypher.
Message Broker & Cache	Redis 7 (Streams)	Files persistantes asynchrones, reprise sur panne et cache performant.
Calcul de Rayon d'Impact	NetworkX 3.2	Algorithmes de parcours en largeur (BFS) ultra-rapides en mémoire (<75 ms).
IA Locale & Remediation	Ollama / qwen2.5-coder	Confidentialité totale 100 % locale, modèle spécialiste code, schémas JSON stricts.
Front-end & Rendu IHM	Blade + Tailwind CSS + Cytoscape.js	Rendu serveur réactif, visualisation dynamique et interactive du graphe.
Conteneurisation & Proxy	Docker + Docker Socket Proxy	Isolation hermétique des conteneurs non-root et filtrage de l'API Docker.
Pipeline DevSecOps	GitHub Actions, Psalm, Bandit, Trivy, Cosign	Automatisation des portes bloquantes (SAST, SCA, SBOM CycloneDX, signature).

Conclusion

Ce chapitre a établi l'état de l'art scientifique et conceptuel de la cybersécurité, de l'ASM, des scanners d'audit, du DevSecOps, de l'IA locale et des microservices. Les études comparatives systématiques ont justifié l'ensemble des choix technologiques retenus. Le chapitre suivant détaillera l'analyse approfondie des besoins, la modélisation formelle des menaces STRIDE et la conception architecturale de la plateforme.

Analyse des Besoins et Conception du Système

Introduction

Ce troisième chapitre présente la démarche rigoureuse d'analyse des exigences et de conception architecturale de notre plateforme. Nous débutons par l'analyse exhaustive des besoins fonctionnels, non fonctionnels, des contraintes de sécurité et des exigences légales (PoA). Nous formalisons ensuite la modélisation fonctionnelle à travers les diagrammes de cas d'utilisation UML. Par la suite, nous détaillons l'architecture globale en distinguant explicitement les **7 composants applicatifs métier** et les **12 conteneurs Docker d'infrastructure**. Nous exposons la conception des données (modèle relationnel sur 14 tables, diagramme de classes, graphe openCypher, séquences d'exécution, machine à états et DFD). Nous développons l'architecture de sécurité multicouche et menons une modélisation formelle des menaces selon la méthodologie STRIDE. Enfin, nous détaillons la chaîne DevSecOps intégrée au pipeline de livraison logicielle.

3.1 Analyse des Besoins

3.1.1 Identification des Acteurs

Le système interagit avec quatre profils d'acteurs opérationnels distincts :

- **L'Analyste de Sécurité (Security Analyst)** : Utilisateur opérationnel principal qui configure les cibles d'audit, téléverse les preuves d'autorisation (PoA), lance les scans, analyse les résultats techniques, explore le graphe de connaissances, valide les scripts de remédiation IA et exporte les rapports d'expertise.
- **Le Client / Lecteur (Client / Viewer)** : Bénéficiaire des audits, consultant en lecture seule les tableaux de bord synthétiques, les rapports validés et acquittant les alertes de sécurité liées à son périmètre d'actifs.
- **L'Administrateur Système (System Administrator)** : Supervise la gouvernance globale de

la plateforme, la gestion des comptes utilisateurs, l'attribution des rôles RBAC, la configuration des quotas de scan et la surveillance de l'état de santé des microservices conteneurisés.

- **L'Auditeur / Responsable Conformité (Auditor / Compliance)** : Profil dédié au contrôle réglementaire, consultant en lecture seule les journaux d'audit immuables (*Audit Logs*), contrôlant la validité juridique des mandats PoA et vérifiant la traçabilité exhaustive de l'ensemble des actions menées.

Remarque : Le Responsable Technique (CTO) de l'entreprise d'accueil assure l'encadrement industriel, la revue d'architecture et la gouvernance globale du projet, sans constituer un acteur opérationnel direct au sens de la modélisation UML du système.

3.1.2 Besoins Fonctionnels

Conformément au cahier des charges validé avec Designet Web Agency, les besoins fonctionnels prioritaires couvrent :

- **Gestion des Cibles et Mandats** : Enregistrement sécurisé des cibles d'audit (domaines, sous-domaines, plages IP) avec téléversement et validation obligatoire d'une preuve d'autorisation (PoA).
- **Orchestration des Scans** : Configuration et déclenchement asynchrone des scanners (Nmap, Nuclei, Gobuster, Subfinder, WPScan) selon 3 profils d'intensité (Silent, Balanced, Aggressive) avec gestion du jitter anti-blocage.
- **Normalisation des Constats** : Transformation automatique des sorties brutes hétérogènes en entités normalisées *Finding* avec score de sévérité, mapping CVE/CWE et citations des numéros de lignes de logs bruts.
- **Modélisation en Graphe de Connaissances** : Projection de la topologie de la cible sous forme de graphe relationnel avec calcul du rayon d'impact (*blast radius*) par l'algorithme BFS sous NetworkX.
- **Assistance IA et Remediation-as-Code** : Génération automatisée de synthèses managériales et de scripts durcis de correction (Bash, Ansible, Dockerfile, Terraform) via un modèle de langage local contraint.
- **Modules de Sécurité Offensive et Sandbox** : Tests d'injection (SQL, commandes, XSS), analyse de WAF et déploiement à la demande d'environnements vulnérables isolés (DVWA, SQLi-Labs, WebGoat, bWAPP).
- **Génération et Export de Rapports** : Production de rapports d'audit complets aux formats PDF signé, HTML et JSON, avec mécanisme de partage par liens temporaires sécurisés.

3.1.3 Besoins Non Fonctionnels

- **Sécurité & Confidentialité** : Application du principe de moindre privilège, conteneurs non-root (UID 1001), système de fichiers en lecture seule, chiffrement des secrets et terminaison TLS 1.3.
- **Scalabilité & Résilience** : Découplage des tâches lourdes par files Redis Streams avec réessais automatiques et repli exponentiel.
- **Performance** : Temps de réponse de l'interface inférieur à 200 ms (P95) pour les pages du tableau de bord.
- **Maintenabilité** : Standardisation des microservices via la classe abstraite `BaseScannerService` et documentation complète des endpoints REST.

3.1.4 Contraintes et Exigences de Sécurité

La plateforme manipule des outils et des informations hautement sensibles. Elle impose par conséquent une isolation hermétique des processus d'audit, l'interdiction stricte de toute action de défacement ou de déni de service, et une journalisation infalsifiable de toutes les requêtes administratives.

3.1.5 Contraintes Légales et Preuve d'Autorisation (Proof of Authorization – PoA)

L'exécution d'activités de reconnaissance offensive sur des systèmes d'information est strictement encadrée par la législation (Code Pénal tunisien, Convention de Budapest, réglementations internationales). Afin de renforcer la conformité légale et éthique, notre plateforme implémente une barrière bloquante logicielle : aucun scan de reconnaissance actif ne peut être lancé sans qu'un document de preuve d'autorisation (PoA) valide, dûment signé par le propriétaire de la cible, ne soit attaché et validé dans le projet.

3.2 Modélisation Fonctionnelle UML

3.2.1 Diagramme Global des Cas d'Utilisation

Le diagramme global modélise l'ensemble des cas d'utilisation structurés selon les quatre rôles du système.

3.2.2 Cas d'Utilisation de l'Analyste de Sécurité

L'analyste configure les audits, explore les résultats, manipule le graphe et valide les remédiations.

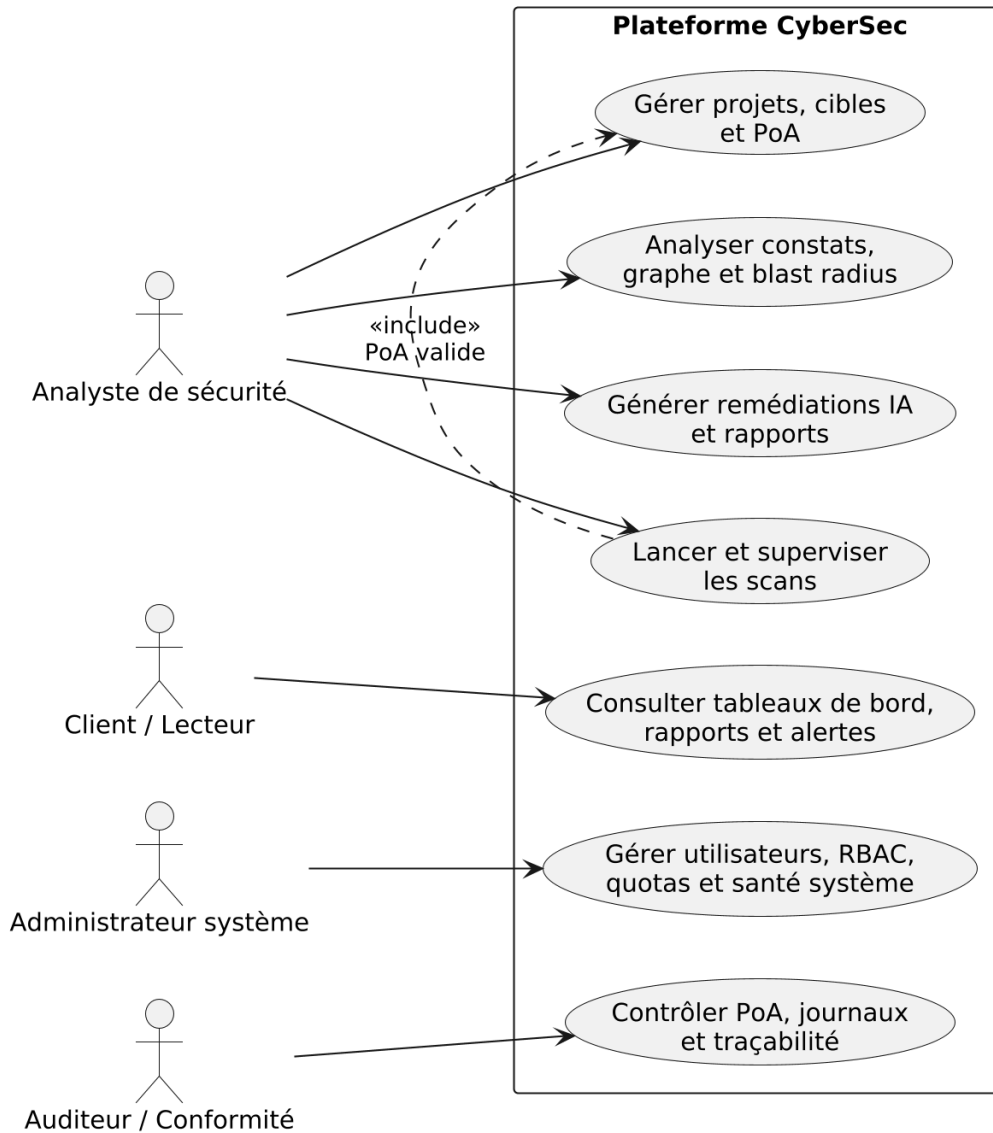


Figure 3.1 : Diagramme Global des Cas d'Utilisation – Acteurs et Fonctionnalités Clés

3.2.3 Cas d'Utilisation de l'Administrateur Système

L'administrateur gère les privilèges, supervise la santé des microservices et contrôle les pistes d'audit.

3.2.4 Description Textuelle Détaillée des Cas d'Utilisation Clés

Afin de formaliser avec précision la dynamique des interactions homme-système, nous détaillons ci-après les cinq cas d'utilisation majeurs de la plateforme sous forme de fiches descriptives standardisées :

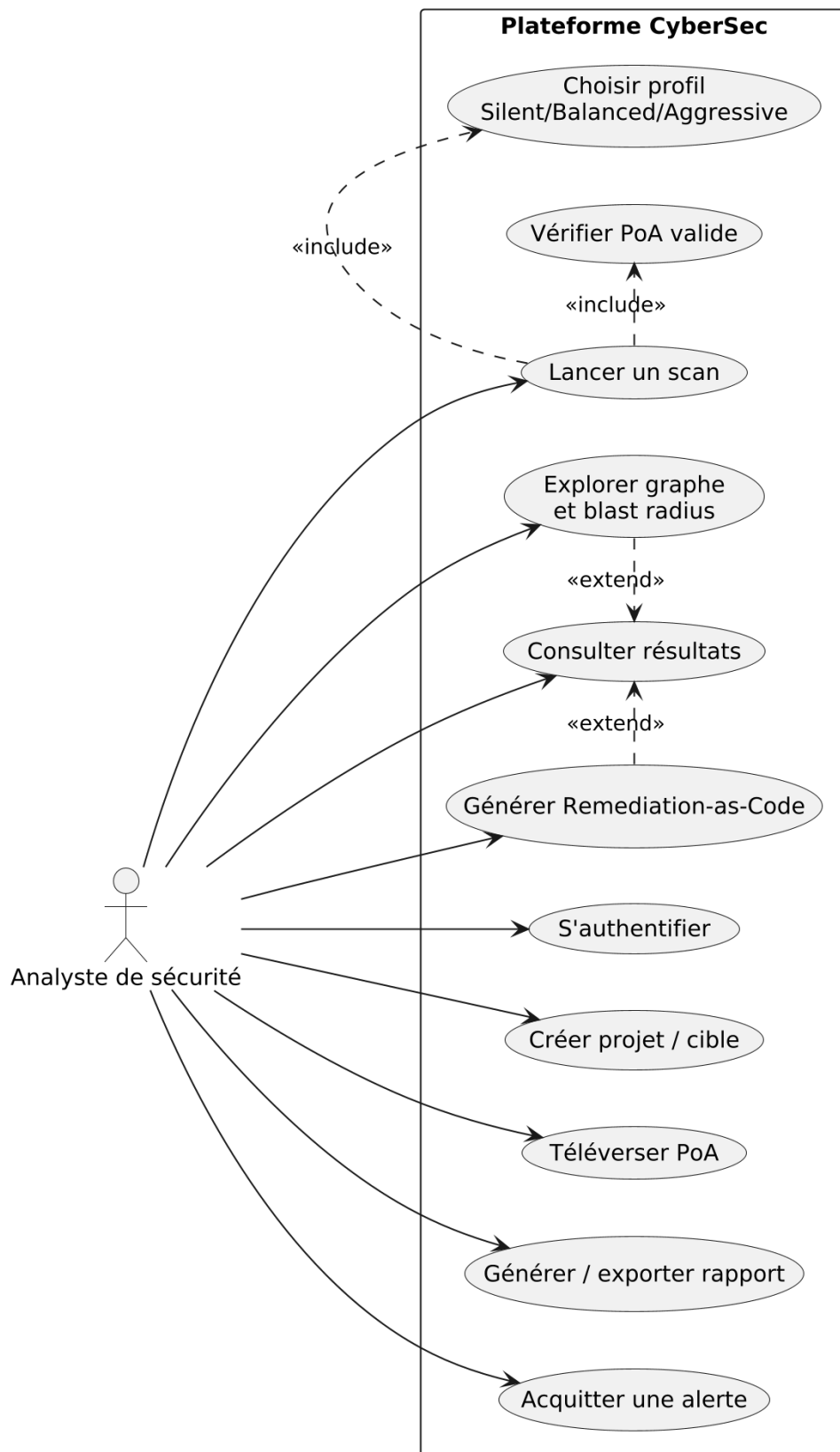


Figure 3.2 : Diagramme des Cas d'Utilisation – Analyste de Sécurité

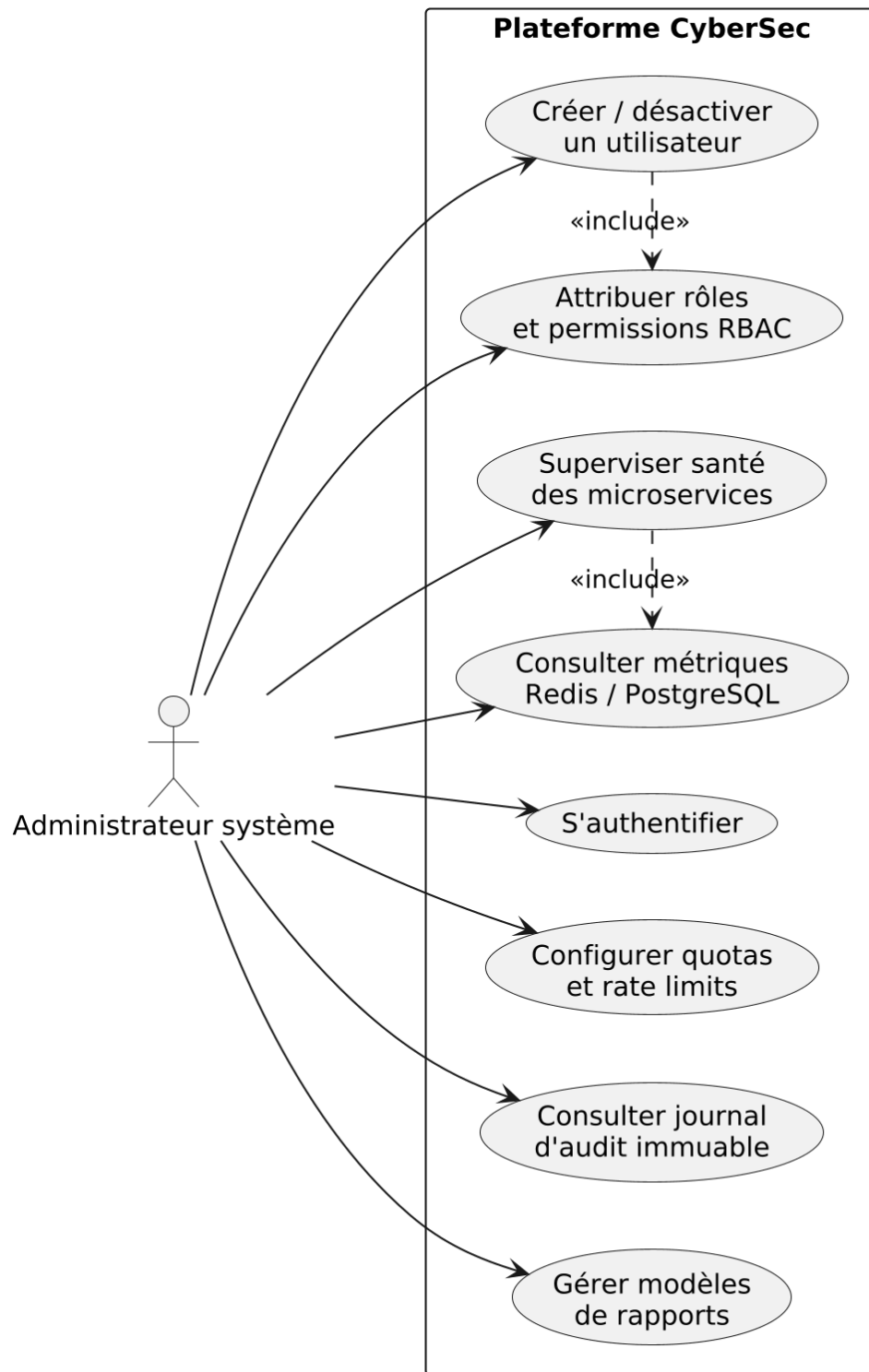


Figure 3.3 : Diagramme des Cas d’Utilisation – Administrateur Système

CU01 : Lancement et Orchestration d'un Scan d'Audit**Table 3.1** : Description Textuelle du Cas d'Utilisation CU01 – Lancement d'un Scan

Cas d'Utilisation	CU01 : Lancement et Orchestration d'un Scan de Reconnaissance
Acteur Principal	Analyste de Sécurité (<i>Security Analyst</i>) / Administrateur
Pré-conditions	Authentification réussie, projet existant avec mandat PoA valide et cible déclarée.
Post-conditions	Tâche de scan insérée dans Redis Streams, statut Queued , suivi en temps réel activé.
Scénario Nominal	1. L'analyste accède au formulaire de lancement de scan (<code>/scans/create</code>). 2. Il sélectionne le projet, la cible d'audit et l'outil de balayage souhaité. 3. Il choisit le profil d'intensité (Silent, Balanced, Aggressive) avec délais de jitter. 4. Le système vérifie automatiquement la validité de la preuve d'autorisation (PoA). 5. L'analyste soumet la demande de scan. 6. Le back-end Laravel publie la commande dans Redis Streams (<code>XADD scan:requests</code>) et renvoie l'ID du scan. 7. L'interface redirige l'analyste vers la page de progression du scan.
Scénarios Alternatifs	4.a. <i>Mandat PoA manquant ou expiré</i> : Le système bloque immédiatement la soumission avec une alerte 403 et journalise la tentative dans les pistes d'audit. 6.a. <i>Service Redis injoignable</i> : Le système bascule sur une mise en file locale d'urgence et notifie l'administrateur.

CU02 : Génération de Remediation-as-Code Assistée par IA**Table 3.2** : Description Textuelle du Cas d'Utilisation CU02 – Génération de Remediation

Cas d'Utilisation	CU02 : Génération de Remediation-as-Code par IA Locale Contrainte
Acteur Principal	Analyste de Sécurité (<i>Security Analyst</i>)
Pré-conditions	Scan terminé avec succès, constats de vulnérabilités normalisés présents en base.
Post-conditions	Scripts de durcissement multi-cibles générés, validés, signés et attachés au constat.
Scénario Nominal	1. L'analyste consulte la fiche d'une vulnérabilité détectée. 2. Il clique sur le bouton « Générer Remédiation ». 3. Le contrôleur extrait le contexte technique (service, version, CVE, lignes de logs bruts). 4. Le microservice IA formate le prompt contraint et interroge le modèle local Ollama (<code>qwen2.5-coder</code>). 5. Le modèle produit le correctif au format JSON strict avec citations de preuves. 6. Le validateur Pydantic contrôle la structure et la véracité des citations. 7. Le script est persisté dans la table <code>remediation_scripts</code> et affiché à l'écran avec coloration syntaxique.
Scénarios Alternatifs	6.a. <i>Réponse JSON invalide ou hallucination de citation détectée</i> : Le système rejette la sortie brute et génère un script de durcissement déterministe durci de repli (fallback).

CU03 : Consultation et Exploration du Graphe de Connaissances**Table 3.3 :** Description Textuelle du Cas d'Utilisation CU03 – Graphe de Connaissances

Cas d'Utilisation	CU03 : Consultation du Graphe et Évaluation du Rayon d'Impact
Acteur Principal	Analyste de Sécurité, Client / Auditeur
Pré-conditions	Projet sélectionné disposant d'actifs et de services cartographiés.
Post-conditions	Topologie relationnelle affichée sous Cytoscape.js avec calcul du blast radius.
Scénario Nominal	1. L'utilisateur accède à l'onglet « Graphe de Connaissances » du projet. 2. Le back-end exécute une requête openCypher dans Apache AGE pour récupérer les nœuds et arêtes. 3. L'interface Cytoscape.js restitue la topologie interactive avec filtres par criticité. 4. L'analyste clique sur un sommet de vulnérabilité pour déclencher le calcul du rayon d'impact. 5. L'algorithme BFS sous NetworkX calcule les dépendances jusqu'à une profondeur de 3 sauts. 6. Les actifs et services potentiellement impactés sont mis en surbrillance rouge à l'écran.
Scénarios Alternatifs	2.a. <i>Projet sans scan antérieur</i> : Le graphe affiche l'actif racine seul avec une invite à lancer un premier balayage.

CU04 : Gestion des Projets et Validation du Mandat PoA**Table 3.4 :** Description Textuelle du Cas d'Utilisation CU04 – Gestion des Projets et PoA

Cas d'Utilisation	CU04 : Création de Projet et Validation du Mandat d'Autorisation
Acteur Principal	Administrateur Système / Analyste de Sécurité
Pré-conditions	Compte utilisateur authentifié avec privilège de gestion de projet.
Post-conditions	Projet créé, cibles validées, document PoA chiffré et stocké sur disque isolé.
Scénario Nominal	1. L'utilisateur accède à la vue de création de projet (/projects/create). 2. Il saisit le nom du projet, la description, et déclare la liste des domaines/IPs cibles. 3. Il téléverse le document PDF de mandat d'autorisation dûment signé par le client. 4. Le back-end vérifie le type MIME du fichier, sa signature numérique et calcule son empreinte SHA-256. 5. Le projet est enregistré et activé, autorisant désormais les scans sur le périmètre validé.
Scénarios Alternatifs	4.a. <i>Fichier non conforme ou corrompu</i> : Le formulaire rejette le document et bloque la création du projet.

CU05 : Génération et Export Multi-Formats des Rapports d'Expertise**Table 3.5** : Description Textuelle du Cas d'Utilisation CU05 – Génération de Rapports

Cas d'Utilisation	CU05 : Production et Téléchargement de Rapport d'Audit Certifié
Acteur Principal	Analyste de Sécurité / Client / Auditeur
Pré-conditions	Scans achevés, constats évalués et remédiations associées.
Post-conditions	Rapport PDF/JSON généré, horodaté, signé et mis à disposition au téléchargement.
Scénario Nominal	1. L'utilisateur sélectionne un scan ou une vue projet et clique sur « Générer Rapport ». 2. Il sélectionne le format d'export souhaité (PDF exécutif, JSON technique, archive ZIP). 3. Le service de reporting agrège les constats, les matrices de risques et les scripts de durcissement. 4. Le document est compilé et horodaté avec un jeton cryptographique de vérification d'intégrité. 5. L'utilisateur télécharge le document ou génère un lien d'accès temporaire sécurisé.
Scénarios Alternatifs	5.a. <i>Accès par lien externe expiré</i> : Le serveur refuse la requête (HTTP 410 Gone) et invite à réémettre un lien.

3.3 Architecture Globale du Système

3.3.1 Vue d'Ensemble du Système

La plateforme adopte une architecture microservices distribuée et segmentée, assurant une séparation nette des responsabilités, une meilleure résilience et une sécurité renforcée par conception.

3.3.2 Composants Applicatifs Métier (7 Composants Logiques)

Sur le plan fonctionnel, l'application est répartie en sept composants logiciels autonomes :

1. **Back-end Web Applicatif (Laravel 11, Port 8000)** : Portail principal, gestion de l'authentification Sanctum, contrôle RBAC Spatie, modèle ORM Eloquent, rendu des vues Blade et distribution des tâches vers les files de messages.

2. **Microservice de Reconnaissance (Flask, Port 5000)** : Moteur d'exécution des scanners Nmap, Nuclei, Gobuster, Subfinder et WPScan, normalisation des résultats (**ResultParser**) et initialisation de la topologie de graphe.
3. **Microservice de Sécurité Offensive (Flask, Port 5001)** : Modules d'évaluation des injections SQL/commandes/XSS, analyse de WAF et pilotage sécurisé des bacs à sable conteneurisés.
4. **Microservice OSINT (Flask, Port 5002)** : Renseignement passif (enregistrements DNS, requêtes WHOIS, certificats SSL/TLS via `crt.sh`, détection d'empreintes technologiques).
5. **Microservice d'IA Contrainte (Flask, Port 5003)** : Couche d'assistance intelligente connectée au modèle local `qwen2.5-coder` via Ollama pour la production de Remediation-as-Code.
6. **Passerelle d'API (API Gateway, Flask, Port 8080)** : Point d'entrée et routeur interne vers les microservices, assurant le contrôle de débit (*rate limiting*) et la traçabilité par `X-Correlation-ID`.
7. **Worker Asynchrone (Python)** : Processus d'arrière-plan consommant les événements de scan via `XREADGROUP` sur Redis Streams, coordonnant les tâches longues et persistant les constats dans PostgreSQL.

3.3.3 Conteneurs d'Infrastructure Docker (12 Services Conteneurisés)

Sur le plan infrastructurel, le déploiement orchestre douze conteneurs Docker complémentaires :

- Les 7 conteneurs hébergeant les 7 composants applicatifs décrits ci-dessus.
- **8. Nginx 1.27** : Serveur mandataire inverse (*reverse proxy*) assurant la terminaison TLS 1.3 et le durcissement des en-têtes HTTP.
- **9. PostgreSQL 16 + Apache AGE** : Système de gestion de base de données unifié relationnel (14 tables) et graphe openCypher.
- **10. Redis 7** : Magasin de données en mémoire servant de Message Broker Streams, cache applicatif et gestionnaire de sessions.
- **11. Ollama** : Serveur d'inférence local exécutant le modèle dédié au code `qwen2.5-coder`.
- **12. Docker-Socket-Proxy** : Proxy de sécurité interdisant les accès directs au socket Unix de Docker pour prévenir toute évasion de conteneur.

3.3.4 Communication Inter-Services et Flux Asynchrone

La figure 3.5 présente la cartographie des flux d'échanges internes : requêtes REST synchrones, streaming d'événements asynchrones et persistance normalisée.

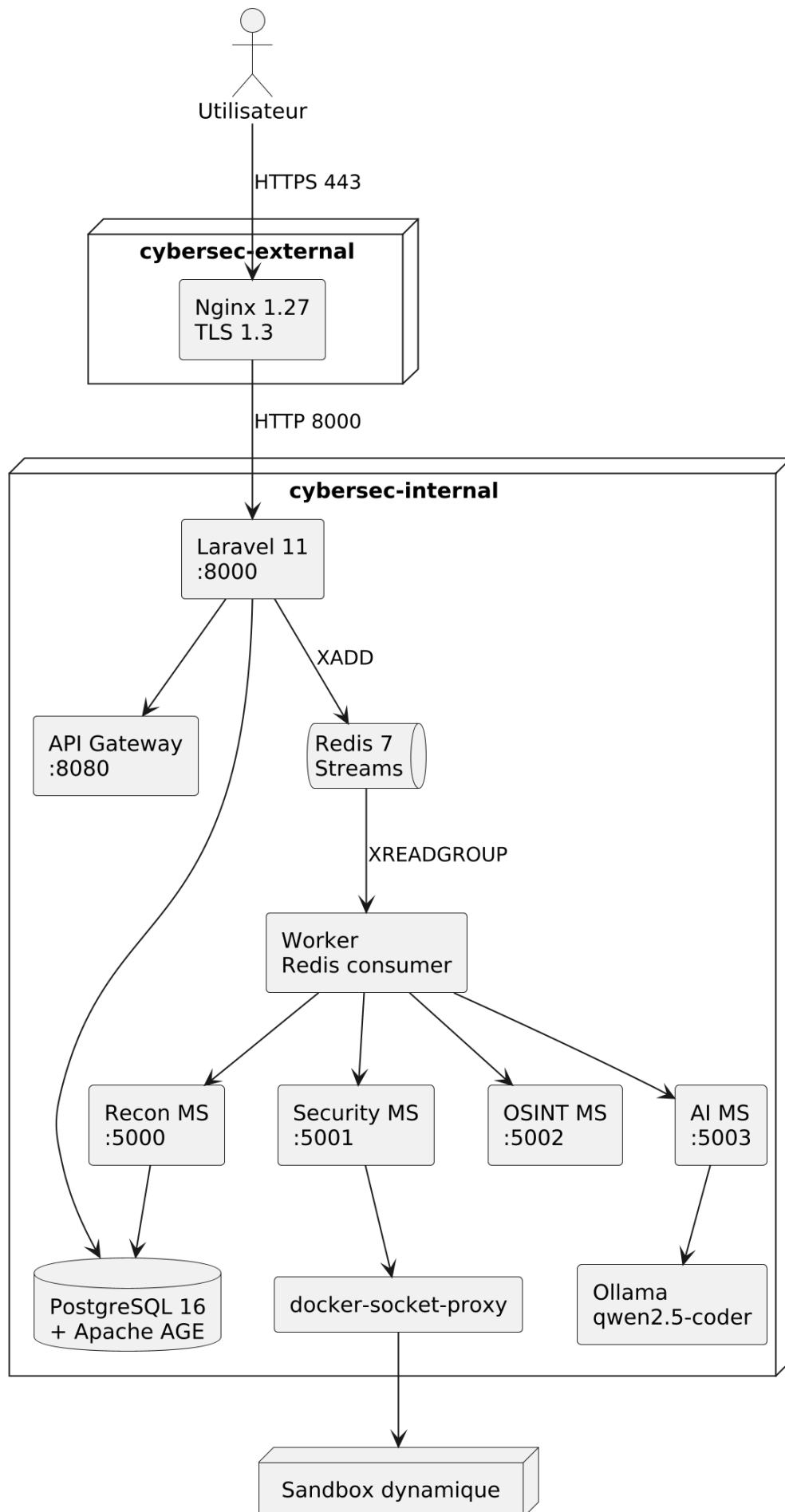


Figure 3.4 : Architecture Globale en Microservices — Vue des Conteneurs Docker et Réseaux Segmentés

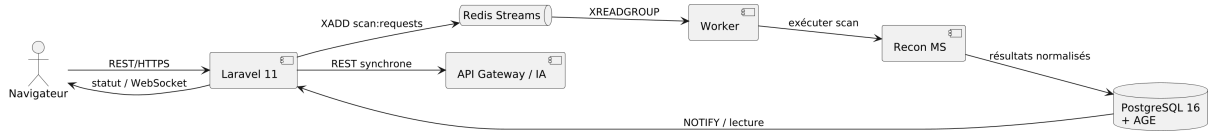


Figure 3.5 : Flux de Traitement Asynchrone des Scans via Redis Streams et PostgreSQL AGE

3.3.5 Communication Synchrone REST vs Asynchrone par Événements

La plateforme combine deux paradigmes d'échange complémentaires :

- **Communication synchrone REST (HTTP/JSON) :** Réservée aux requêtes immédiates (interrogation du tableau de bord, requêtes OSINT rapides, inférence de dialogue IA).
- **Communication asynchrone (Redis Streams) :** Utilisée pour l'exécution des scanners lourds. Le back-end publie un message (XADD), immédiatement acquitté pour l'utilisateur, tandis que le worker dépile le travail en arrière-plan sans bloquer l'IHM.

3.4 Conception des Données et Dynamique du Système

3.4.1 Modèle Entité-Association (ERD)

La base de données structure 14 tables relationnelles liées par des contraintes d'intégrité référentielle strictes et enrichies de colonnes JSONB indexées par index GIN.

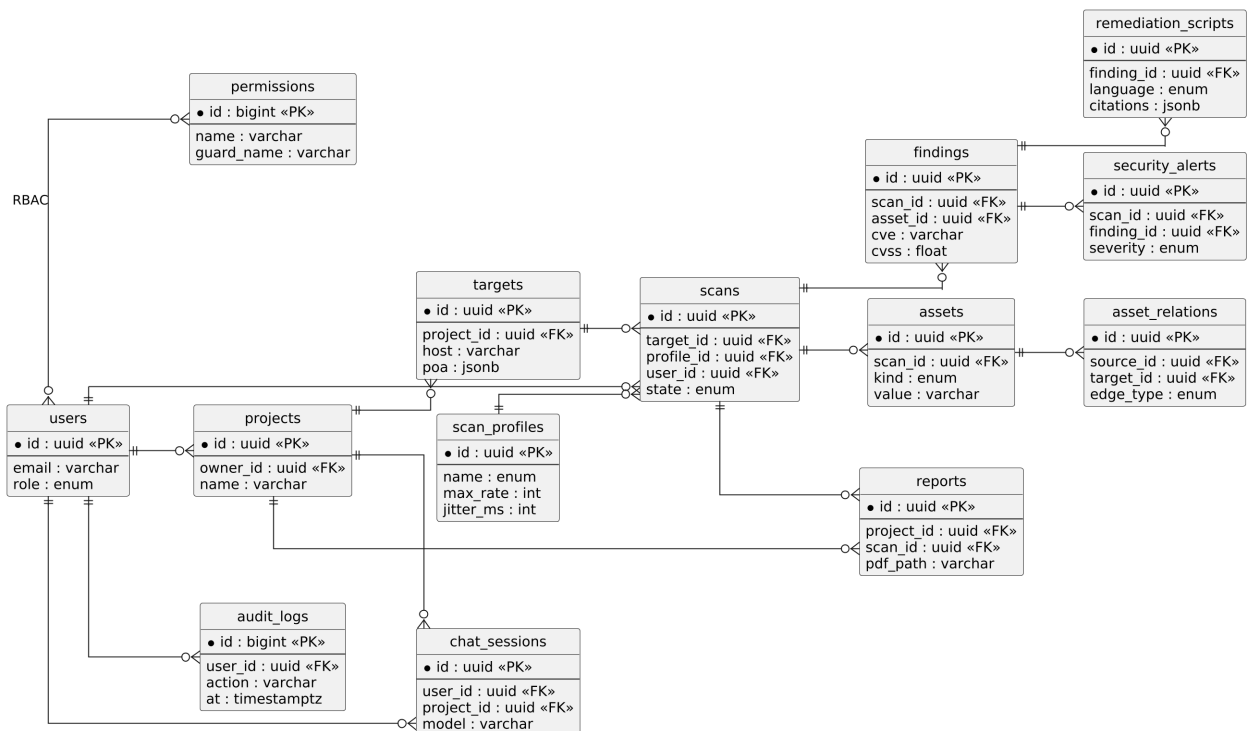


Figure 3.6 : Modèle Entité-Association (ERD) – Les 14 Tables de la Plateforme

3.4.2 Diagramme de Classes du Modèle de Données

Tandis que le Modèle Entité-Association (ERD) formalise la structure relationnelle et physique de la base de données, le diagramme de classes représente la structure logique des principales entités métier et leurs relations au sein du framework applicatif. La structure objet et les relations cardinales entre les entités sont illustrées dans la figure 3.7.

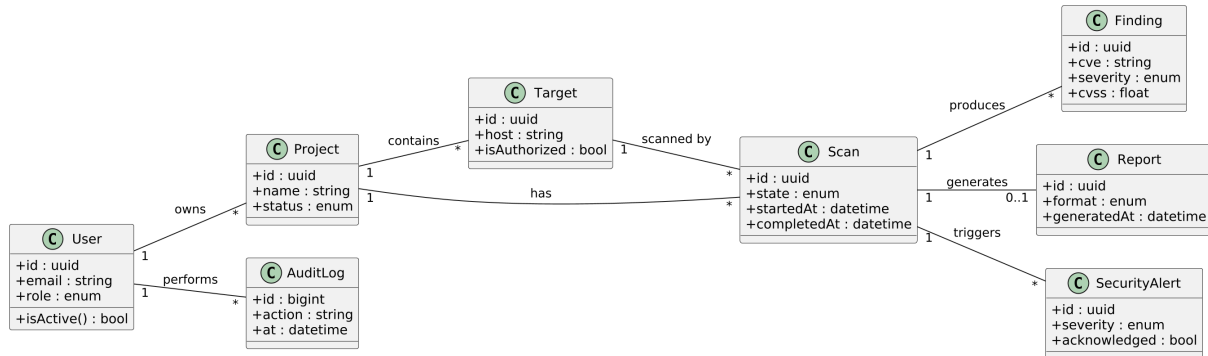


Figure 3.7 : Diagramme de Classes – Entités Métier et Relations du Modèle

3.4.3 Modèle du Graphe de Connaissances et Projection Apache AGE

Les relations de sécurité sont modélisées sous forme de nœuds (*Target*, *Asset*, *Port*, *Service*, *Vulnerability*) et d'arêtes typées (*RESOLVES_TO*, *HOSTS*, *RUNS*, *HAS_VULN*, *IMPACTS*). Cette modélisation permet de calculer le rayon d'impact (*blast radius*) par parcours de graphe en largeur (BFS) implémenté sous NetworkX.

3.4.4 Diagrammes de Séquence du Système

Afin de couvrir l'ensemble des scénarios critiques de la plateforme, nous modélisons trois flux d'interaction séquentiels majeurs :

3.4.4.1 Orchestration Asynchrone d'un Scan

La figure 3.8 modélise la séquence d'exécution complète d'un audit de reconnaissance, de la soumission initiale jusqu'à la persistance des constats.

3.4.4.2 Génération de Remediation-as-Code Assistée par IA

La figure 3.9 illustre le flux d'interaction entre l'analyste, le back-end Laravel, le microservice d'IA et le modèle local Ollama lors de la création d'un correctif de sécurité.

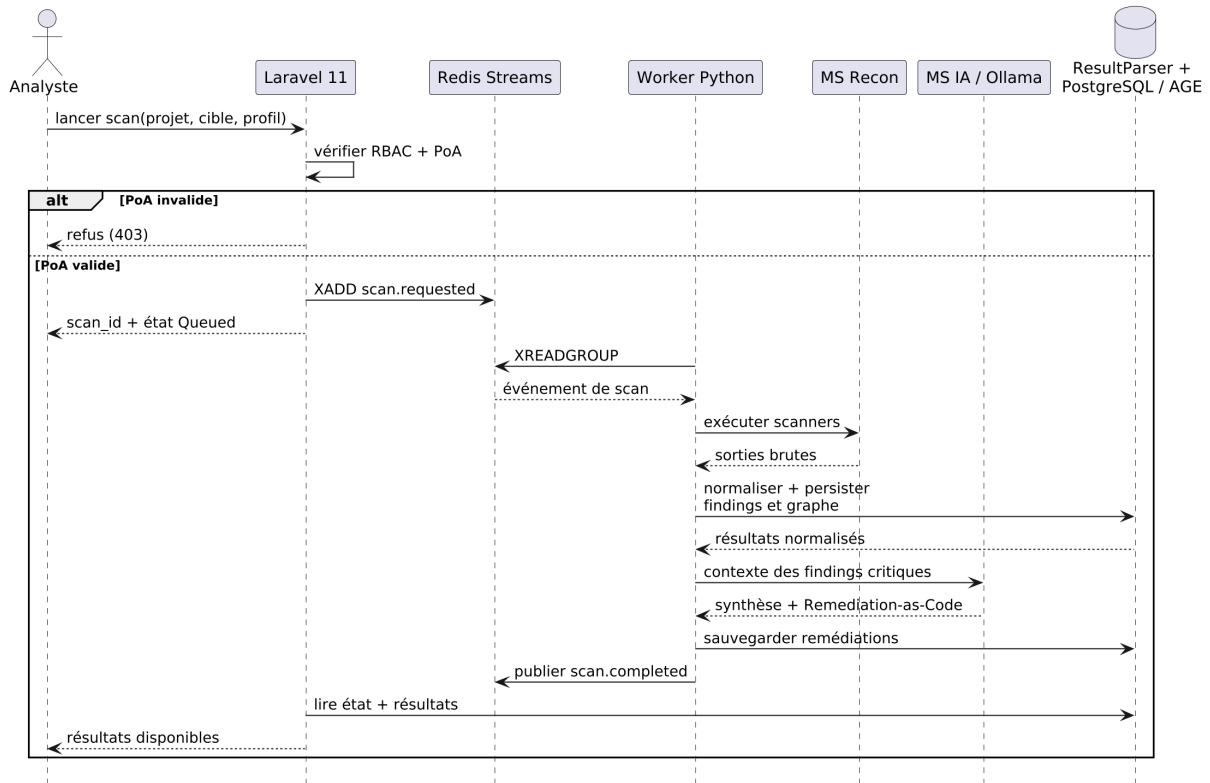


Figure 3.8 : Diagramme de Séquence – Lancement, Traitement Asynchrone et Remédiation IA

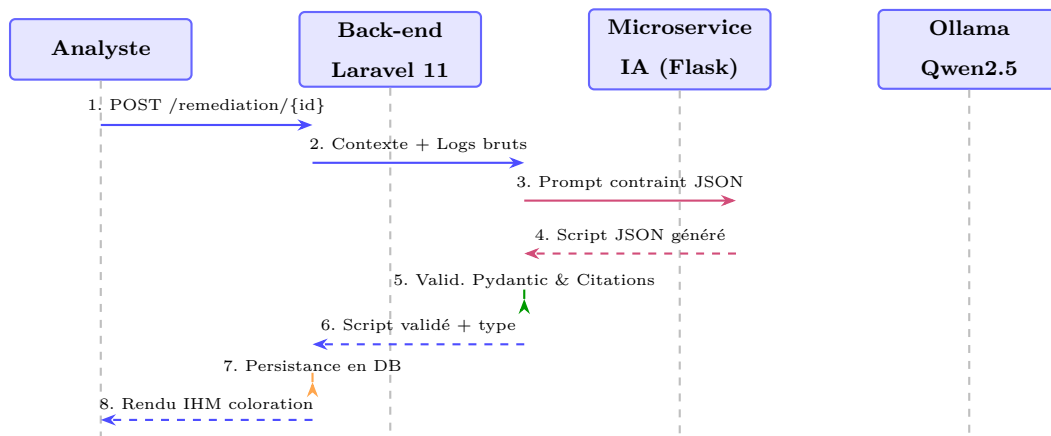


Figure 3.9 : Diagramme de Séquence – Génération de Remediation-as-Code Assistée par IA

3.4.4.3 Consultation du Graphe et Propagation du Rayon d'Impact

La figure 3.10 détaille les échanges lors du requêtage openCypher du graphe de connaissances et du calcul algorithmique du rayon d'impact via l'algorithme BFS sous NetworkX.

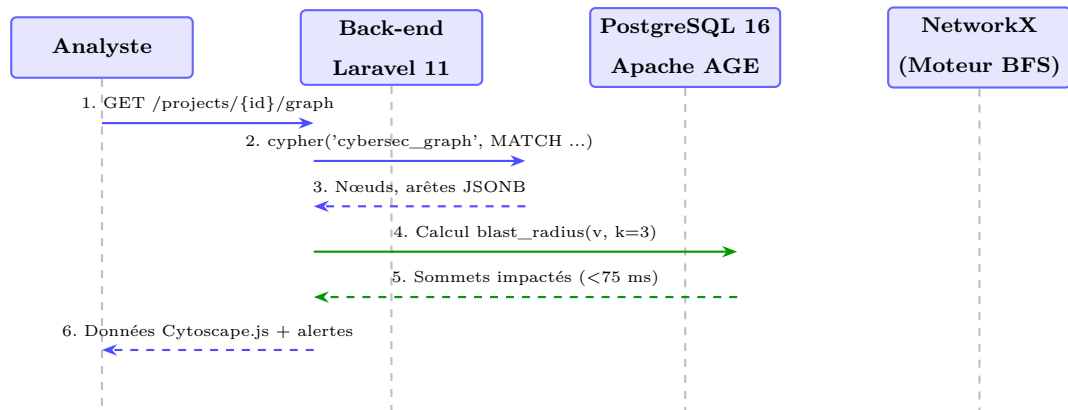


Figure 3.10 : Diagramme de Séquence – Exploration du Graphe et Calcul de Blast Radius

3.4.5 Machine à États d'un Scan

Le cycle de vie d'une tâche de scan est régi par une machine à états finis déterministe comprenant les états **Pending**, **Queued**, **Running**, **Completed**, **Failed** et **Cancelled**.

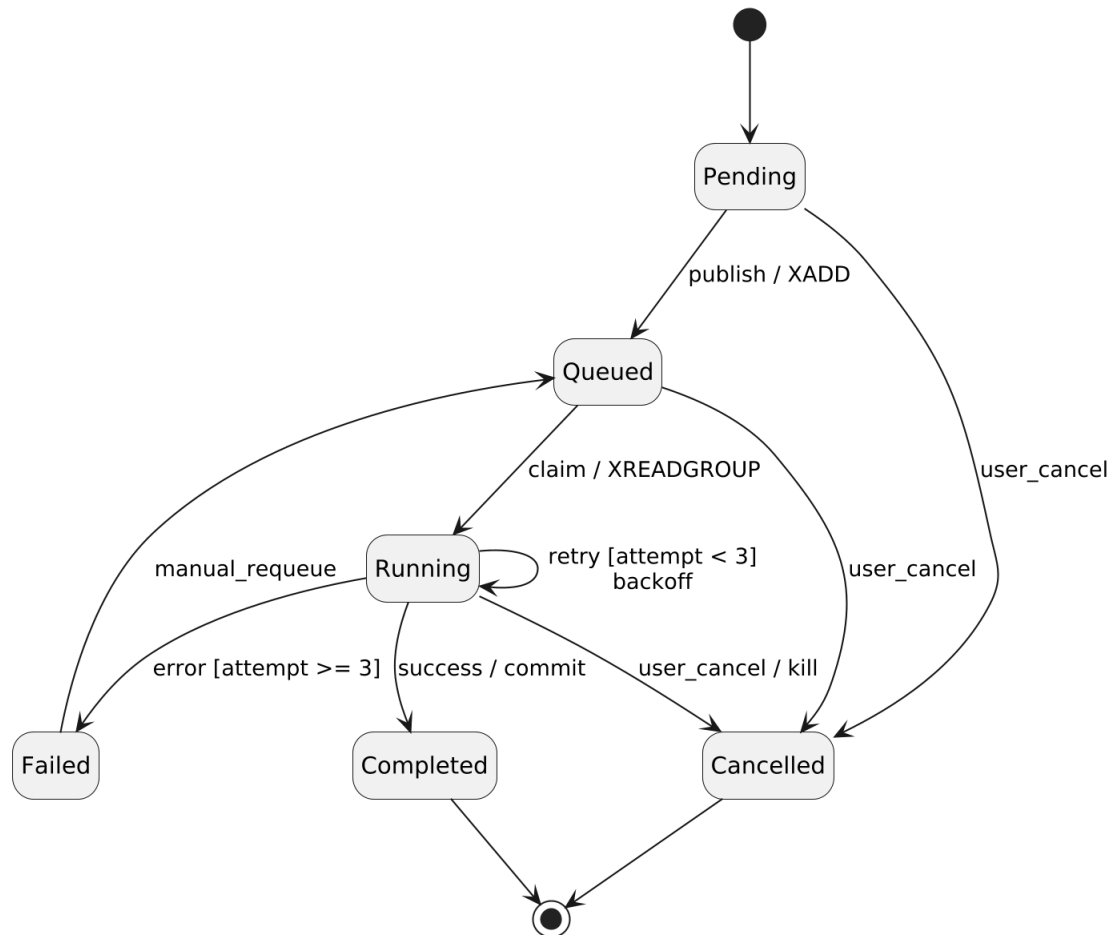


Figure 3.11 : Machine à États – Cycle de Vie d'une Tâche de Scan

3.4.6 Diagramme de Flux de Données (DFD)

Le cheminement et les transformations successives des données sont synthétisés dans le diagramme de flux de la figure 3.12.

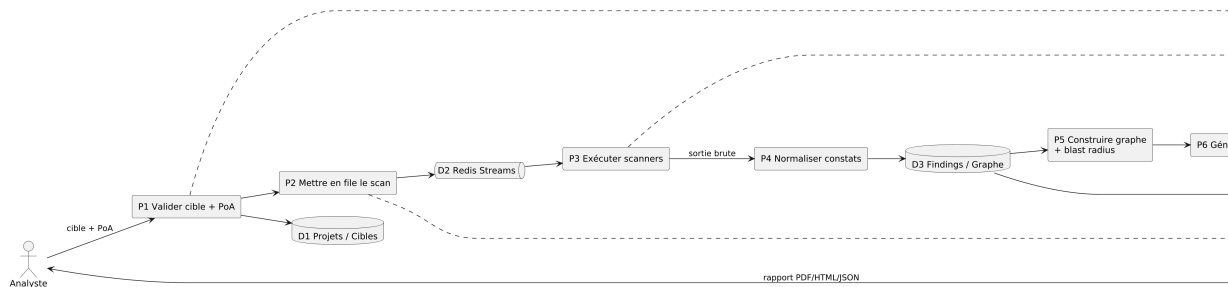


Figure 3.12 : Diagramme de Flux de Données (DFD) – Processus d’Audit, Graphe de Connaissances et Reporting

3.5 Architecture de Sécurité Multicouche

Afin de protéger la plateforme elle-même contre toute compromission, nous appliquons une défense en profondeur (*Defense-in-Depth*) sur six niveaux détaillés dans le tableau 3.6.

Table 3.6 : Mesures de Sécurité en Profondeur par Couche Architecturale

Couche de Sécurité	Mécanismes Implémentés	Objectif de Protection
1. Réseau & DMZ	Nginx TLS 1.3, réseaux bridges isolés (internal vs external)	Isolation des flux inter-services et terminaison sécurisée.
2. Conteneurs Docker	Utilisateur non-root (UID 1001), <code>read_only: true</code> , <code>cap_drop: ALL</code>	Prévention de l'évasion de conteneurs et de l'élévation de privilèges.
3. Applicative Web	Tokens CSRF, assainissement XSS, requêtes préparées Eloquent	Neutralisation complète des risques OWASP Top 10 web.
4. Données & Secrets	Chiffrement <code>bcrypt</code> , clés d'API chiffrées, pistes d'audit immuables	Intégrité, confidentialité et traçabilité sans faille.
5. Contrôle d'Accès	Modèle RBAC strict (4 rôles), validation systématique de PoA	Respect du principe de moindre privilège et conformité légale.
6. Bac à Sable (Sandbox)	Docker Socket Proxy filtrant les appels d'API du daemon	Confinement strict des tests sur environnements vulnérables.

3.6 Modélisation Formelle des Menaces (Threat Modeling STRIDE)

3.6.1 Méthodologie et Frontières de Confiance

Une plateforme de reconnaissance offensive concentre par nature des outils puissants et des informations critiques de vulnérabilité. Elle constitue par conséquent une cible de choix pour des attaquants. Afin d'évaluer méthodiquement les risques pesant sur le système, nous avons appliqué la méthodologie de modélisation des menaces **STRIDE** développée par Microsoft.

La figure 3.13 illustre la cartographie des frontières de confiance (*Trust Boundaries*) et l'exposition de la surface d'attaque interne de la plateforme.

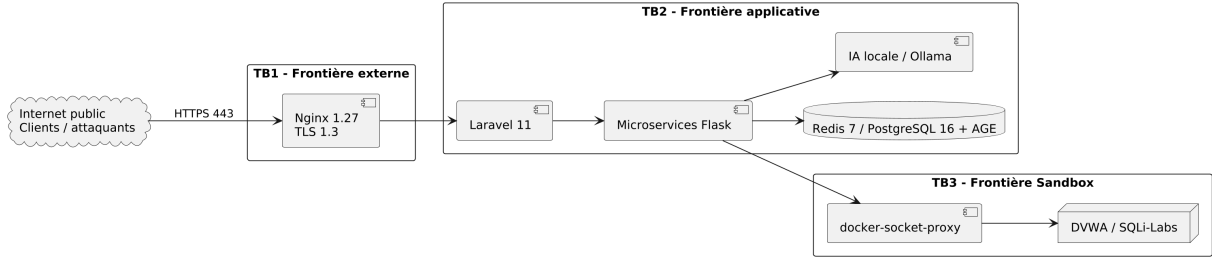


Figure 3.13 : Cartographie de la Surface d’Attaque et Frontières de Confiance (Trust Boundaries STRIDE)

3.6.2 Matrice des Menaces STRIDE et Contre-Mesures Appliquées

Le tableau 3.7 détaille l’analyse systématique des six catégories de menaces STRIDE sur chaque composant clé de la plateforme, le niveau de risque associé et les contre-mesures architecturales implémentées.

Table 3.7 : Matrice d’Analyse des Menaces STRIDE et Contre-Mesures Architecturales

Composant Cibl�	Menace STRIDE Identifi�e	Niveau de Risque	Contre-Mesures et Att�nuations Impl�ment�es
Passerelle Nginx	Spoofing / Usurpation d’identit�	�lev�	Terminaison TLS 1.3 avec HSTS, validation stricte des certificats Let’s Encrypt
Back-end Laravel	Tampering / Alt�ration des donn�es	�lev�	Requ�tes pr�par�es Eloquent (anti-SQLi), protection CSRF native, assainissement HTML
Syst�me de Journalisation	Repudiation / R�pudiation d’actions	Moyen	Piste d’audit immuable (<i>Audit Log</i>) horodat�e avec tra�abilit� X-Correlation-ID
Base PostgreSQL / AGE	Information Disclosure / Fuite d’infos	Critique	R�seau priv� <i>cybersec-internal</i> non expos�, chiffrement des cl�s d’API au repos
Files Redis Streams	Denial of Service / D�ni de service	�lev�	Quotas stricts de scans simultan�s par utilisateur, limitation de d�bit (<i>rate limiting</i>)
Microservice Sandbox	Elevation of Privilege / �vasion Docker	Critique	Isolation via <i>docker-socket-proxy</i> , conteneurs non-root (UID 1001), <i>cap_drop: ALL</i>
Microservice IA (Ollama)	Tampering / Injection de Prompts	�lev�	<i>System prompt</i> fig�, validation de sch�ma JSON strict et v�rification des citations de logs
Moteur de Scanners	Ex�cution sur cible non autoris�e	Critique	Barri�re logicielle bloquante : v�rification obligatoire du mandat d’autorisation (PoA)

3.7 Architecture DevSecOps D taill e

3.7.1 Pipeline CI/CD sous GitHub Actions

La s curit  de la cha ne logicielle est enti rement automatis e au sein du d p t de code via le workflow GitHub Actions d clar  dans `.github/workflows/devsecops.yml`. Chaque commit et requ te de tirage (*pull request*) d clenche l’ex cution de huit  tapes successives bloquantes.



Figure 3.14 : Pipeline DevSecOps Automatis  – Cha ne de S curit  Int gr e sous GitHub Actions

3.7.2 Description des Portes de S curit  (Security Gates)

Le pipeline GitHub Actions applique des barri res de contr le strictes interdisant tout d ploiement si une anomalie critique est d tect e :

- ** tape 1 – D clenchement (Trigger) :** Analyse automatique sur push/PR ciblant la branche `main`.

- **Étape 2 – Analyse SAST** : Analyse statique PHP (Psalm niveau 3) et Python (Bandit). Si une vulnérabilité haute sévérité est détectée, le build échoue immédiatement.
- **Étape 3 – Analyse SCA (Software Composition Analysis)** : Trivy scanne l'arbre des dépendances (`composer.lock` et `requirements.txt`). Toute CVE de niveau *HIGH* ou *CRITICAL* non corrigée bloque la chaîne.
- **Étape 4 – Génération du SBOM** : L'outil Syft inventorie l'ensemble des paquets et génère un fichier conforme à la norme CycloneDX JSON.
- **Étape 5 – Construction de l'Image** : Docker BuildKit compile l'image conteneurisée de production avec mise en cache optimisée.
- **Étape 6 – Scan de l'Image Conteneur** : Trivy analyse l'image Docker finale (système de fichiers racine et paquets OS).
- **Étape 7 – Signature Cryptographique** : Cosign (Sigstore) signe l'image conteneur et pousse la signature sur le registre GitHub Container Registry (GHCR).
- **Étape 8 – Déploiement en Production** : Déploiement sur le serveur VPS uniquement si toutes les étapes précédentes sont au statut *PASS*.

Conclusion

Ce chapitre a présenté la formalisation complète des besoins, la modélisation fonctionnelle UML, l'architecture découplée en microservices (7 composants applicatifs et 12 conteneurs Docker), la conception relationnelle et en graphe de connaissances, la modélisation formelle des menaces STRIDE, ainsi que la politique de défense en profondeur et le pipeline DevSecOps. Le chapitre suivant expose la réalisation logicielle concrète et la validation expérimentale du système.

Réalisation et Validation Expérimentale

Introduction

Ce quatrième chapitre détaille la réalisation technique de notre plateforme et présente les résultats rigoureux de sa validation expérimentale. Nous commençons par décrire l’environnement de développement, puis nous détaillons l’architecture interne du back-end Laravel 11, le framework d’encapsulation des scanners Python, le pipeline de normalisation des résultats et la spécification de l’API RESTful. Nous présentons ensuite les interfaces utilisateurs clés, avant d’approfondir l’implémentation algorithmique du graphe de connaissances (Apache AGE / NetworkX) et du moteur d’IA locale contrainte (Ollama / `qwen2.5-coder`). Nous exposons les modalités de déploiement durci en production et l’exécution de la chaîne DevSecOps. Enfin, nous dressons la matrice de traçabilité des exigences, analysons les résultats des 140 tests unitaires et d’intégration, détaillons les benchmarks de charge menés avec Locust, et conduisons une discussion approfondie sur les performances et les limites du système.

4.1 Environnement de Développement et Outillage

Le développement, l’intégration et la validation de la plateforme ont été conduits dans un environnement standardisé et reproductible. Les tableaux 4.1 et 4.2 détaillent respectivement la configuration matérielle et logicielle exploitée.

Table 4.1 : Spécification de la Configuration Matérielle Utilisée

Composant Matériel	Poste de Développement Local	Serveur VPS de Production
Processeur (CPU)	AMD Ryzen 7 5800H @ 3.20 GHz (8 Cœurs / 16 Threads)	Intel Xeon @ 2.60 GHz (4 vCPUs)
Mémoire Vive (RAM)	16 Go DDR4 3200 MHz	8 Go DDR4 ECC
Stockage	512 Go SSD NVMe M.2	100 Go SSD NVMe haute performance
Système d’Exploitation	Windows 11 Professionnel / WSL2 Ubuntu 24.04	Ubuntu 24.04 LTS (Noyau Linux 6.8 x86_64)
Connectivité Réseau	Fibre optique 100 Mbit/s symétrique	Connexion dédiée 1 Gbit/s non plafonnée

Table 4.2 : Environnement Logiciel, Outils et Versions des Frameworks

Catégorie	Outil / Technologie	Version & Rôle
Environnement d'Exécution	Docker Engine / Compose	v27.0.3 / v2.28 (Conteneurisation et isolation)
Serveur Web & Reverse Proxy	Nginx	v1.27.0 (Terminaison TLS 1.3, reverse proxy)
Back-end Web	PHP / Laravel	PHP 8.3 / Laravel 11.x (Portail métier, ORM, Auth)
Microservices	Python / Flask	Python 3.12 / Flask 3.1.0 (Scanners, IA, OSINT)
Base de Données	PostgreSQL + Apache AGE	PostgreSQL 16.3 + Apache AGE 1.5.0 (Relationnel/Graphe)
Message Broker & Cache	Redis	Redis 7.2 (Streams, files de tâches, sessions)
Moteur d'IA Locale	Ollama / qwen2.5-coder	Ollama v0.3.4 / Modèle 7B Q4_K_M (Inference RaC)
Front-end	Tailwind CSS / Vite / Cytoscape.js	Tailwind v3.4 / Vite v5.2 / Cytoscape.js v3.28
Outils de Sécurité Intégrés	Nmap, Nuclei, Gobuster, Subfinder	Nmap 7.94, Nuclei v3.2.9, Gobuster v3.6, Subfinder v2.6.6
Chaîne DevSecOps	Psalm, Bandit, Trivy, Syft, Cosign	Psalm 5.24, Bandit 1.7.9, Trivy v0.52, Cosign v2.4

4.2 Implémentation du Back-end Applicatif (Laravel 11)

4.2.1 Architecture Interne en Couches du Back-end

Le back-end Laravel 11 adopte une architecture modulaire en couches garantissant une stricte séparation des responsabilités :

- Couche de Routage (`routes/web.php` et `routes/api.php`) :** Déclaration des points d'entrée HTTP avec application des middlewares de sécurité.
- Couche des Requêtes de Formulaire (*Form Requests*) :** Validation rigoureuse des paramètres d'entrée (formats d'URL, présence PoA) avant transmission aux contrôleurs.
- Couche des Contrôleurs (*Controllers*) :** Réception des requêtes validées, coordination métier et renvoi des réponses JSON ou vues Blade.
- Couche de Services (*Services*) :** Logique métier pure (*ScanOrchestration*, *ReportGeneration*, *GraphDataService*).
- Couche des Tâches Asynchrones (*Jobs*) :** Traitements d'arrière-plan (*ExecuteScanJob*, *ExportReportJob*) délégués à Redis.
- Couche d'Accès aux Données (*Eloquent Models*) :** Représentation objet des 14 tables relationnelles avec contraintes et colonnes JSONB.

4.2.2 Authentification, RBAC et Middlewares de Sécurité

La gestion des identités et des privilèges est assurée par l'intégration de **Laravel Sanctum** et du module **Spatie Permission** :

- **Modèle RBAC Granulaire :** Quatre rôles stricts sont définis : *Admin* (administration totale), *Security Analyst* (gestion des cibles, déclenchement de scans, remédiations), *Client* / *Viewer*

(consultation des synthèses et acquittement d’alertes) et *Auditor / Compliance* (consultation des journaux immuables et des mandats PoA).

- **Protection contre les Attaques Web** : Activation native des tokens anti-CSRF sur toutes les requêtes mutantes, assainissement automatique des sorties pour prévenir les failles XSS, et requêtes préparées paramétrées dans Eloquent empêchant toute injection SQL.
- **Limitation de Débit (*Rate Limiting*)** : Le middleware `throttle:60,1` protège les endpoints d’authentification et de soumission de scans contre les attaques par force brute et les dénis de service applicatifs.

4.2.3 Orchestration Asynchrone des Scans (Jobs & Queues)

Lors de la soumission d’une requête de scan, le contrôleur vérifie la validité du mandat PoA, instancie la tâche `App\Jobs\ExecuteScan` et injecte la commande dans Redis Streams via l’instruction `XADD scan:requests * target $target profile $profile`. Le serveur retourne immédiatement un code HTTP 202 (Accepted) avec l’identifiant du scan, garantissant une réactivité instantanée de l’IHM pour l’utilisateur sans aucun blocage réseau.

4.3 Implémentation des Microservices Python Flask

4.3.1 Microservice de Reconnaissance (Port 5000)

Encapsule l’exécution des scanners Nmap, Nuclei, Gobuster, Subfinder et WPScan à travers des classes dédiées héritant de l’interface abstraite `BaseScannerService`.

4.3.2 Microservice de Sécurité Offensive et Bac à Sable (Port 5001)

Intègre les algorithmes de test de vulnérabilités applicatives non destructives (injections SQL, XSS, détection de WAF) et pilote les bacs à sable conteneurisés (DVWA, SQLi-Labs, WebGoat, bWAPP) via l’API isolée de `docker-socket-proxy`.

4.3.3 Microservice OSINT (Port 5002)

Regroupe cinq modules d’énumération passive : résolution DNS structurée (`dnspython`), interrogation WHOIS (`python-whois`), analyse des chaînes de certificats TLS, requêtage des journaux de transparence de certificats (`crt.sh`) et détection d’empreintes technologiques web.

4.3.4 Microservice d'IA Contrainte (Port 5003)

Assure l'inférence locale auprès du démon Ollama (<http://ollama:11434/api/generate>). Il applique un *system prompt* rigide forçant le modèle `qwen2.5-coder` à structurer ses réponses selon un schéma JSON défini et à fournir des citations exactes pointant vers les numéros de lignes des journaux bruts.

4.3.5 Architecture du Framework de Scanners (BaseScannerService)

Afin d'éviter la duplication de code et d'assurer une extensibilité maximale pour l'ajout futur de nouveaux outils d'audit, nous avons conçu un patron de conception Adaptateur (*Adapter Pattern*) articulé autour de la classe abstraite `BaseScannerService`.

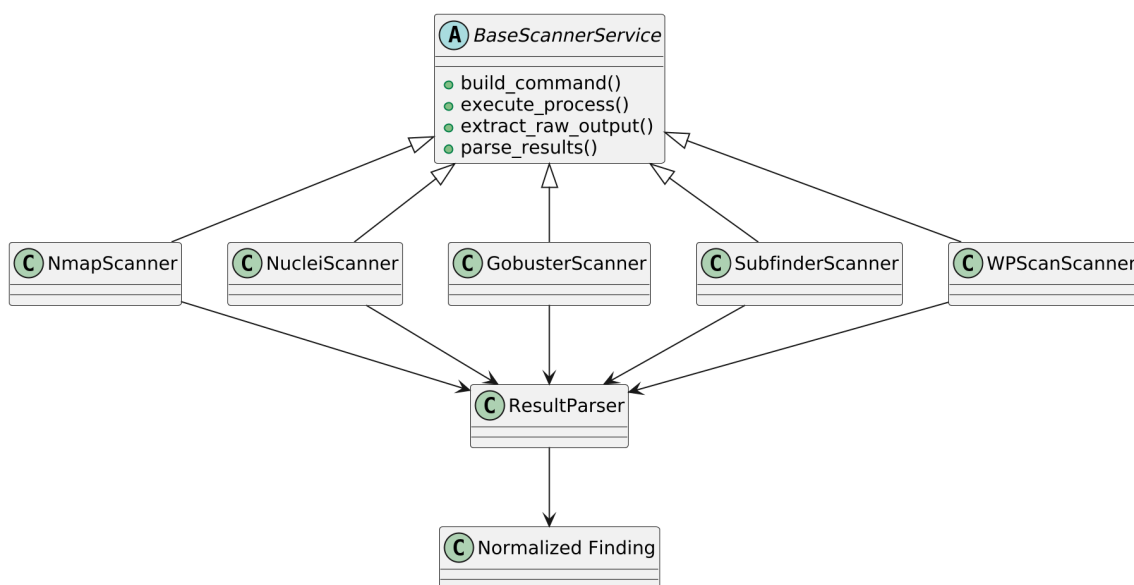


Figure 4.1 : Architecture Modulaire du Framework d'Exécution et de Parsing des Scanners (`BaseScannerService`)

Chaque adaptateur d'outil implémente quatre méthodes fondamentales :

- `build_command(target, profile, options) -> list[str]` : Construit la ligne de commande selon le profil d'intensité (Silent, Balanced, Aggressive).
- `execute_process(cmd, timeout) -> tuple[int, str, str]` : Exécute le binaire de façon isolée via `subprocess.Popen` avec contrôle de timeout et capture séparée de `stdout` et `stderr`.
- `extract_raw_output() -> dict` : Récupère les fichiers temporaires (XML, JSON ou texte brut).
- `parse_results(raw_data) -> list[dict]` : Transmet les flux à la classe `ResultParser` pour normalisation unifiée.

4.3.6 Pipeline de Normalisation des Données (**ResultParser**)

Les scanners produisent des formats hétérogènes (XML pour Nmap, JSON-Lines pour Nuclei, flux textuel pour Gobuster). Le module **ResultParser** réalise une transformation séquentielle en six étapes :

1. **Désérialisation Syntaxique** : Parsing du flux source en structures Python typées (`dict`, `list`).
2. **Extraction des Attributs Clés** : Identification de l'actif, du port, du protocole, du service et de la version logicielle.
3. **Calcul du Hash d'Empreinte (*Fingerprint*)** : Génération d'une clé SHA-256 unique `hash(target + port + vuln_id)` permettant d'éliminer instantanément les doublons lors de scans répétés.
4. **Normalisation de la Sévérité** : Conversion des libellés hétérogènes en échelle unifiée : *Info*, *Low*, *Medium*, *High*, *Critical*, conforme aux scores CVSS v3.1.
5. **Mapping CVE / CWE** : Rapprochement avec les référentiels de vulnérabilités du NIST et de l'OWASP.
6. **Génération de l'Entité Finding** : Création de l'objet normalisé stocké dans PostgreSQL et projeté dans le graphe Apache AGE.

4.4 Spécification et Conception de l'API RESTful

La plateforme expose une API RESTful normalisée permettant l'intégration avec des outils tiers (SIEM, SOAR) et assurant la communication entre le front-end Blade et les microservices d'arrière-plan. Le tableau 4.3 présente les principaux endpoints de l'API.

Table 4.3 : Spécification des Principaux Endpoints de l'API RESTful de la Plateforme

Point d'Entrée (Endpoint)	Méthode	Description Fonctionnelle	Rôles Autorisés	Code Succès
/api/v1/projects	GET	Liste des projets et cibles d'audit configurés	Admin, Analyste, Client	200 OK
/api/v1/projects	POST	Création d'un nouveau projet avec attachement de la PoA	Admin, Analyste	201 Created
/api/v1/scans	POST	Soumission d'une requête de scan (asynchrone Redis)	Admin, Analyste	202 Accepted
/api/v1/scans/{id}	GET	État d'avancement et métriques d'exécution d'un scan	Admin, Analyste, Client	200 OK
/api/v1/findings/{id}	GET	Fiche détaillée d'une vulnérabilité avec preuves brutes	Admin, Analyste, Client	200 OK
/api/v1/projects/{id}/graph	GET	Topologie complète du graphe de connaissances openCypher	Admin, Analyste, Client	200 OK
/api/v1/assets/{id}/impact	GET	Calcul algorithmique du rayon d'impact (BFS)	Admin, Analyste	200 OK
/api/v1/ai/remediate	POST	Génération de script Remediation-as-Code via LLM	Admin, Analyste	200 OK
/api/v1/reports/{id}/export	GET	Téléchargement du rapport d'audit (PDF signé / JSON)	Admin, Analyste, Client	200 OK
/api/v1/audit-logs	GET	Consultation des pistes d'audit immuables	Admin, Auditeur	200 OK

4.4.1 Exemple de Requête et Réponse JSON Normalisée

L'extrait suivant illustre le format JSON de réponse produit lors de la consultation d'un constat de vulnérabilité normalisé via l'endpoint GET `/api/v1/findings/442` :

```
{
  "status": "success",
  "data": {
    "id": 442,
    "scan_id": 87,
    "target": "ensit.tn",
    "port": 22,
    "service": "OpenSSH 8.9p1 Ubuntu",
    "title": "SSH Weak Password Authentication Enabled",
    "severity": "medium",
    "cvss_score": 5.3,
    "cve_id": "CVE-2023-48795",
    "cwe_id": "CWE-287",
    "evidence_file": "nmap_scan_87.xml",
    "remediation_status": "generated",
    "created_at": "2026-08-22T10:42:33Z"
  }
}
```

4.5 Interfaces Utilisateurs de la Plateforme

L'interface graphique a été développée sous le moteur de templates Laravel Blade, enrichie de Tailwind CSS et orchestrée via Vite. Elle offre un thème sombre (*Dark Theme*) optimisé pour les salles de contrôle SOC.

4.5.1 Authentification et Enregistrement Sécurisé

L'accès à la plateforme est protégé par une authentification forte intégrant la protection anti-CSRF, la limitation de débit et la journalisation des tentatives d'accès (figures 4.2 et 4.3).

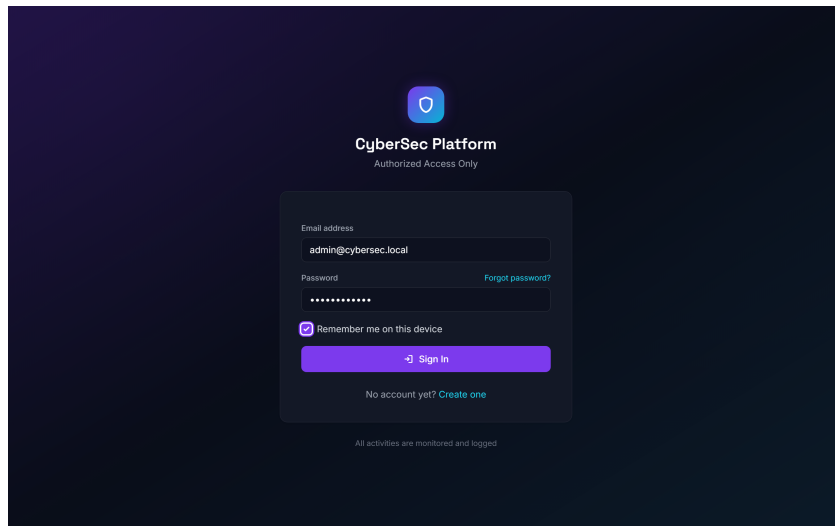


Figure 4.2 : Interface d'Authentification Sécurisée de la Plateforme

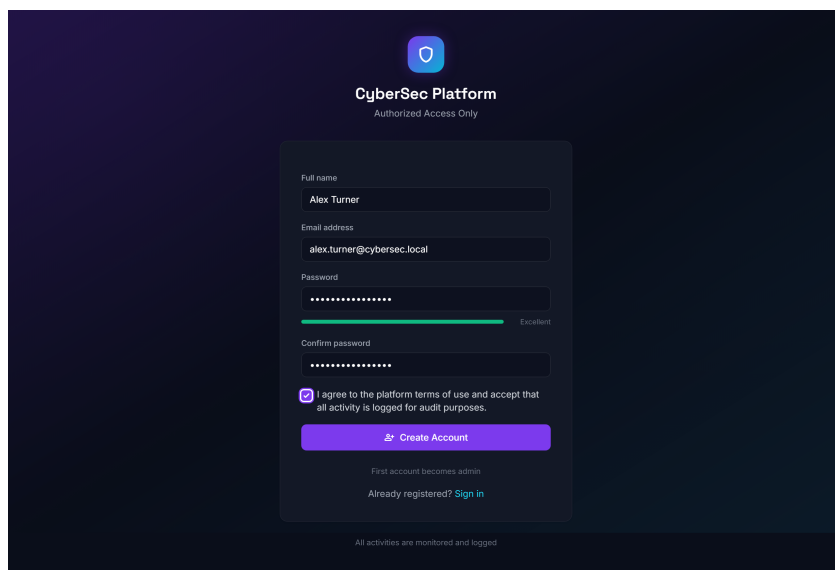


Figure 4.3 : Interface de Création de Compte et Enregistrement Utilisateur

4.5.2 Tableau de Bord Principal (Dashboard ASM)

Le tableau de bord principal (figure 4.4) synthétise l'état global de la surface d'attaque : indicateurs clés (KPIs), répartition des vulnérabilités par criticité (Critical, High, Medium, Low, Info), statut des scans actifs et alertes récentes.

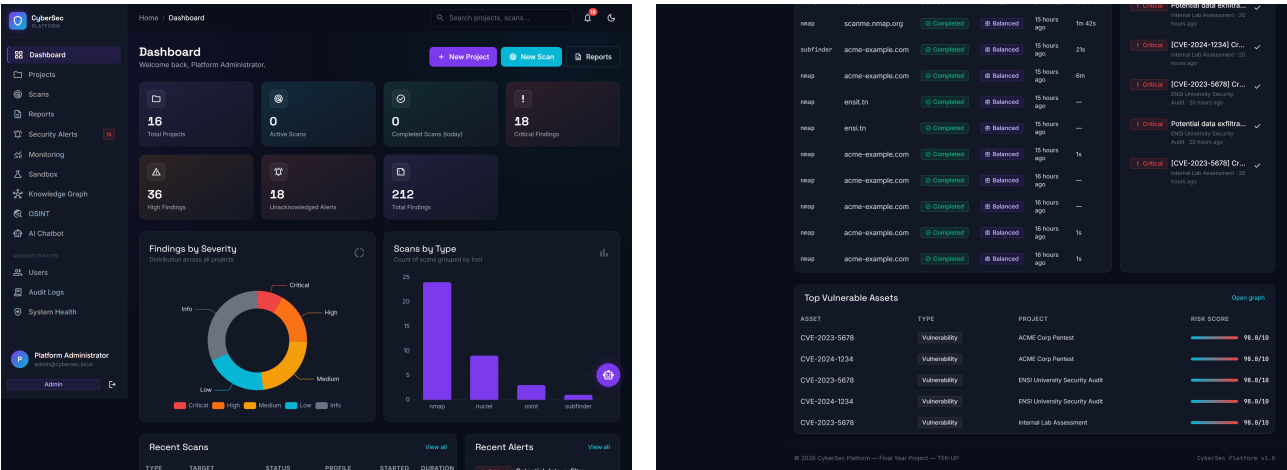


Figure 4.4 : Tableau de Bord Principal – Indicateurs Clés (KPIs) et Cartographie des Risques

4.5.3 Gestion des Projets et Cibles d’Audit

La gestion des périmètres s’effectue via les interfaces dédiées aux projets (figures 4.5, 4.6 et 4.7), permettant de déclarer les cibles et de rattacher la preuve d’autorisation (PoA).

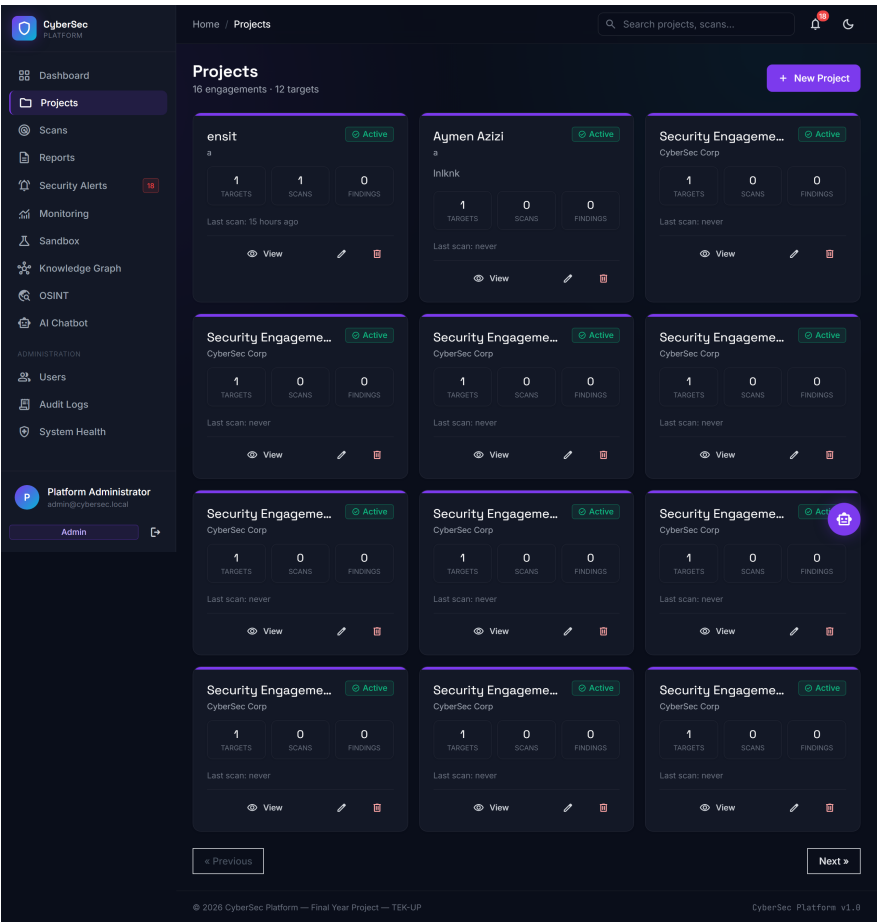


Figure 4.5 : Liste des Projets d’Audit et Synthèse du Statut des Cibles

Figure 4.6 : Formulaire de Création de Projet et Téléversement de la Preuve d’Autorisation (PoA)

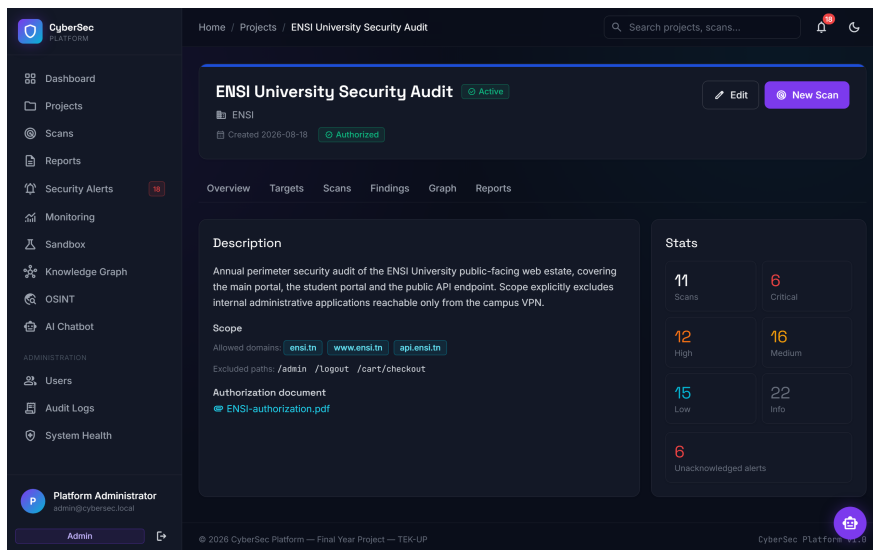


Figure 4.7 : Fiche Détaillée d’un Projet – Cibles Associées et Historique des Scans

4.5.4 Lancement et Suivi des Scans de Reconnaissance

Les figures 4.8 et 4.9 illustrent la configuration des profils de balayage (Silent, Balanced, Aggressive) et le suivi en temps réel de l’avancement des scans via Redis Streams.

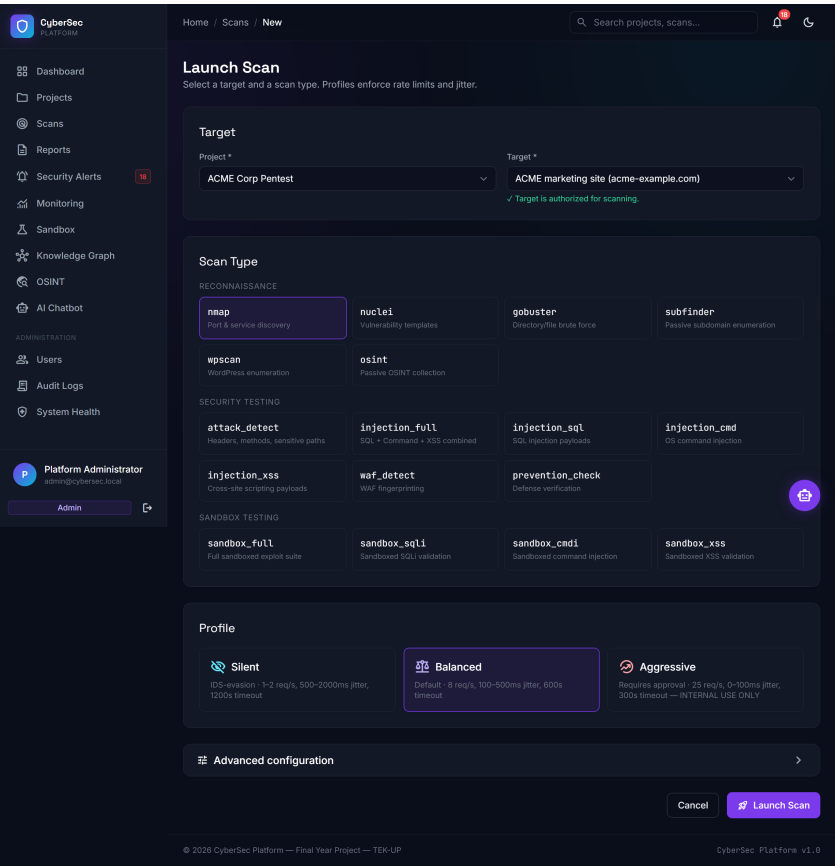


Figure 4.8 : Interface de Configuration et Lancement d'un Scan d'Audit

The screenshot shows the 'Scans' page in the CyberSec Platform. The left sidebar contains navigation links: Dashboard, Projects, Scans (active), Reports, Security Alerts (15), Monitoring, Sandbox, Knowledge Graph, OSINT, AI Chatbot, and an ADMINISTRATION section with Users, Audit Logs, and System Health. The main header shows 'Home / Scans' and a search bar. Below the header, there are filters for Project (All projects), Status (All), Type (All), and Profile (All), with 'Apply filters' and 'Reset' buttons. The main content is a table of scans with columns: TYPE, TARGET, PROJECT, PROFILE, STATUS, SEVERITY, STARTED, and DURATION. The table lists various scans, including nmap, subfinder, and osint, with their respective targets, projects, and statuses. A 'Platform Administrator' user is shown in the bottom right corner.

TYPE	TARGET	PROJECT	PROFILE	STATUS	SEVERITY	STARTED	DURATION
nmap	scanme.nmap.org	ENSI University Security Audit	Balanced	Completed	1 2	15 hours ago	1m 42s
subfinder	acme-example.com	ACME Corp Pentest	Balanced	Completed	3	15 hours ago	21s
nmap	acme-example.com	ACME Corp Pentest	Balanced	Completed	2	15 hours ago	6m
nmap	ensit.tn	ensit	Balanced	Completed	—	15 hours ago	—
nmap	ensit.tn	ENSI University Security Audit	Balanced	Completed	—	15 hours ago	—
nmap	acme-example.com	ACME Corp Pentest	Balanced	Completed	—	15 hours ago	1s
nmap	acme-example.com	ACME Corp Pentest	Balanced	Completed	—	16 hours ago	—
nmap	acme-example.com	ACME Corp Pentest	Balanced	Completed	—	16 hours ago	—
nmap	acme-example.com	ACME Corp Pentest	Balanced	Completed	—	16 hours ago	1s
nmap	acme-example.com	ACME Corp Pentest	Balanced	Completed	—	16 hours ago	1s
nmap	acme-example.com	ACME Corp Pentest	Balanced	Completed	—	16 hours ago	—
nmap	acme-example.com	ACME Corp Pentest	Balanced	Completed	—	16 hours ago	1s
nmap	acme-example.com	ACME Corp Pentest	Balanced	Completed	—	16 hours ago	1s
nmap	acme-example.com	ACME Corp Pentest	Balanced	Completed	—	16 hours ago	—
nmap	ensit.tn	ENSI University Security Audit	Balanced	Failed	—	16 hours ago	—
nmap	www.ensit.tn	ENSI University Security Audit	Balanced	Cancelled	—	—	—
osint	ensit.tn	ENSI University Security Audit	Balanced	Completed	1 2 5	2 days ago	1m 19s
nmap	api.ensit.tn	ENSI University Security Audit	Balanced	Completed	1 2 3 4	1 day ago	2m 6s
nmap	acme-example.com	ACME Corp Pentest	Balanced	Completed	1 2 3 4	1 day ago	2m 51s
nuclei	api.ensit.tn	ENSI University Security Audit	Balanced	Completed	2 3 5 2 1	1 day ago	1m 33s

Figure 4.9 : Liste et État d’Avancement des Scans d’Audit Orchestrés

4.5.5 Restitution des Résultats et Preuves Brutes

La vue détaillée d’un scan (figure 4.10) affiche l’ensemble des constats identifiés avec filtrage dynamique par niveau de gravité et accès aux extraits bruts des logs.

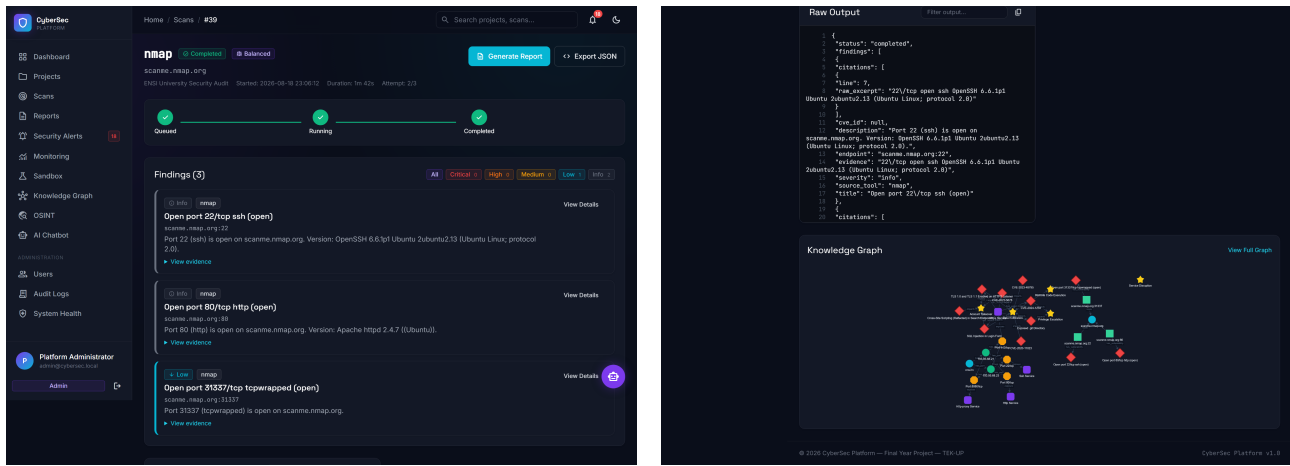


Figure 4.10 : Restitution des Résultats de Scan et Visualisation des Preuves Techniques

4.5.6 Fiche de Constat et Remediation-as-Code Assistée par IA

La figure 4.11 présente la fiche complète d'une faille, incluant le score CVSS, le descriptif technique et les scripts de remédiation automatisés (Bash, Ansible, Dockerfile) générés par l'IA locale contrainte.

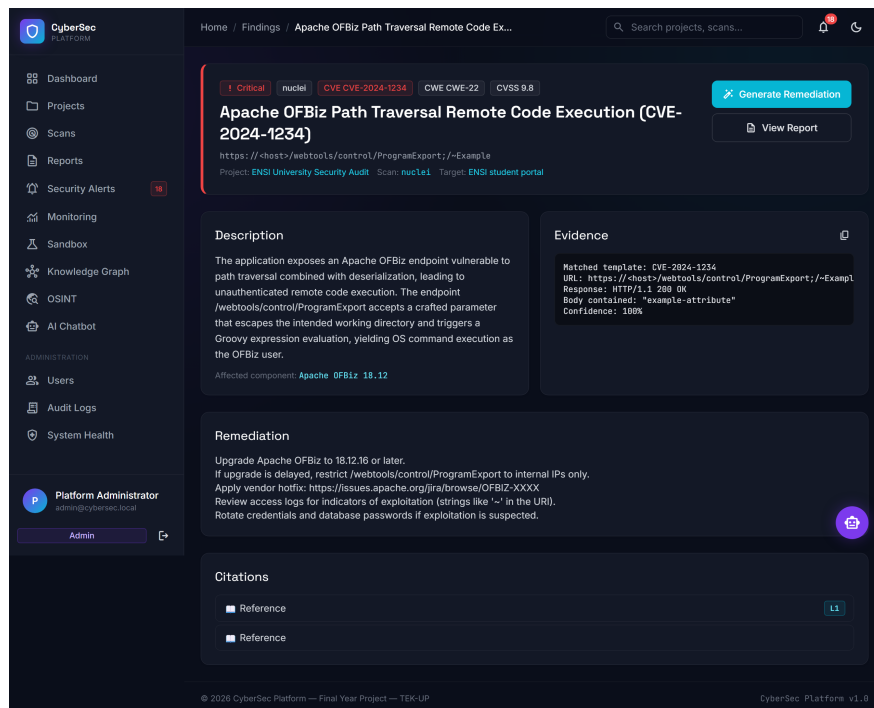


Figure 4.11 : Fiche Détaillée d'un Constat et Scripts de Remediation-as-Code Générés par l'IA

4.5.7 Visualisation Interactive du Graphe de Connaissances

L'interface Cytoscape.js (figure 4.12) projette la topologie relationnelle des actifs (*Target* → *Asset* → *Port* → *Service* → *Vulnerability*) et met en évidence le rayon d'impact calculé par parcours BFS.

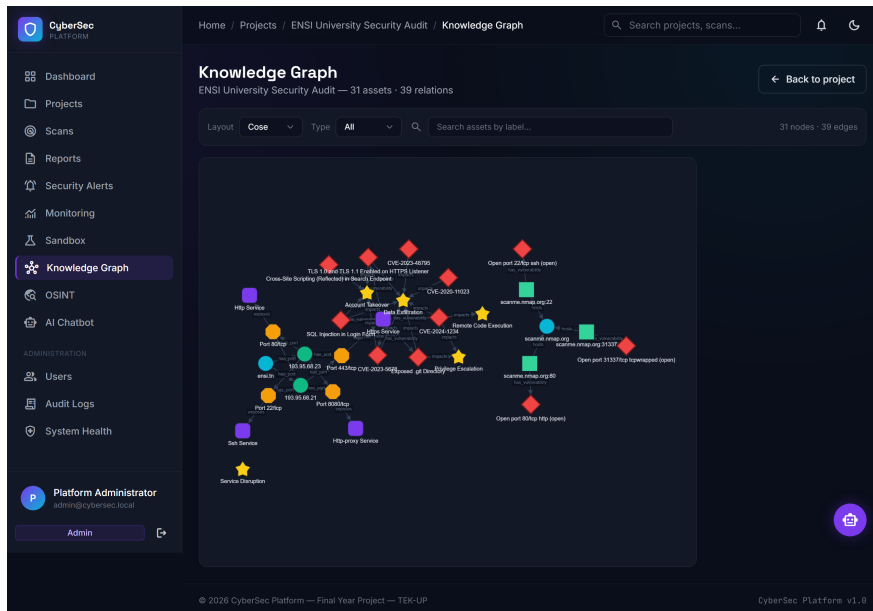


Figure 4.12 : Visualisation Interactive du Graphe de Connaissances et du Rayon d’Impact

4.5.8 Module de Renseignement Passif (OSINT)

Le module OSINT (figure 4.13) regroupe les données d’énumération passive : résolutions DNS complètes, enregistrements WHOIS, certificats SSL/TLS via `crt.sh` et technologies web détectées.

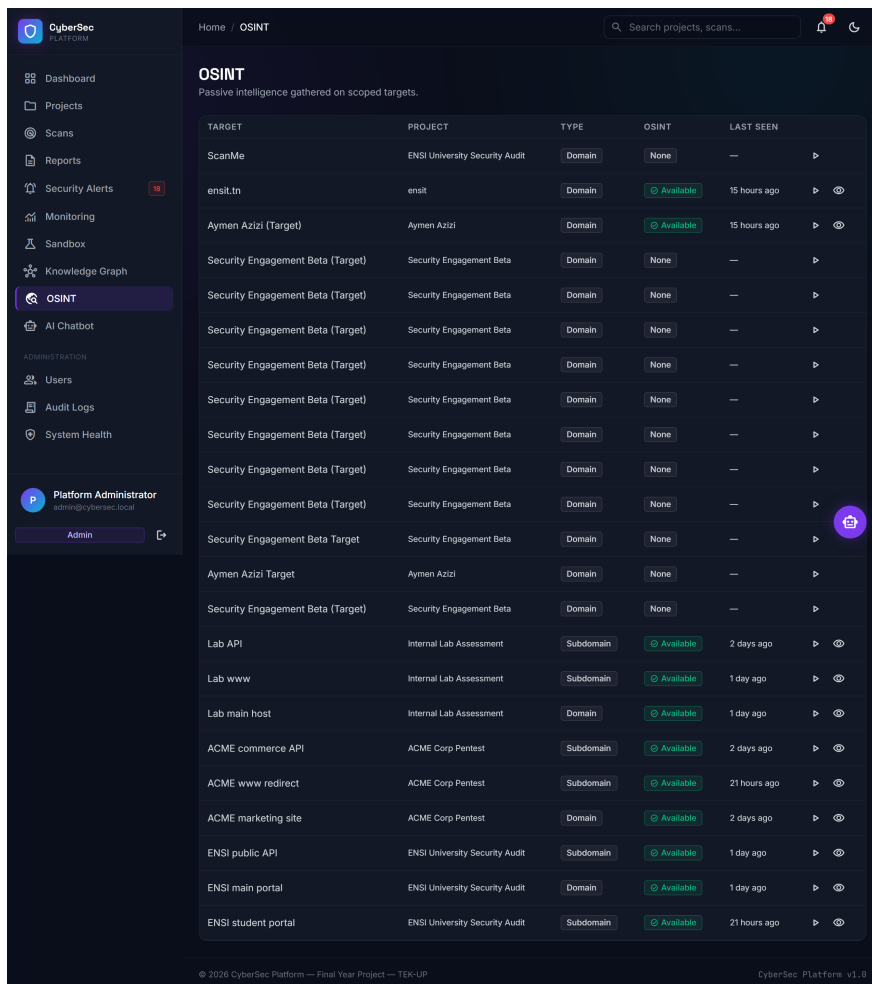


Figure 4.13 : Module de Renseignement Passif OSINT – Analyse DNS, WHOIS et Certificats TLS

4.5.9 Gestionnaire du Bac à Sable (Sandbox Isolée)

La figure 4.14 illustre le pilotage sécurisé des environnements vulnérables conteneurisés (DVWA, SQLi-Labs) via **docker-socket-proxy** pour la validation contrôlée des techniques de détection.

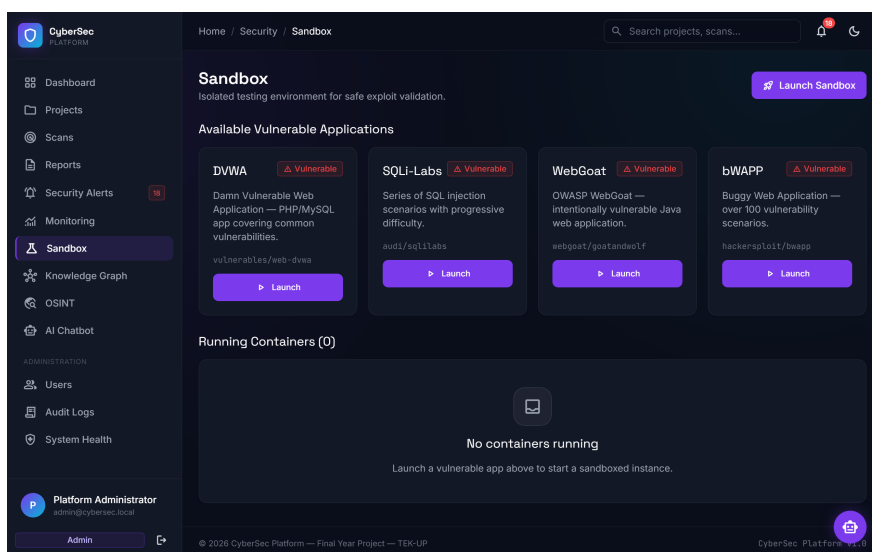


Figure 4.14 : Gestionnaire du Bac à Sable (Sandbox) pour la Validation d'Exploits

4.5.10 Alertes de Sécurité et Surveillance Continue

Les figures 4.15 et 4.16 présentent le centre de gestion des alertes critiques et le module de télémétrie pour la surveillance en temps réel de la surface d'attaque.

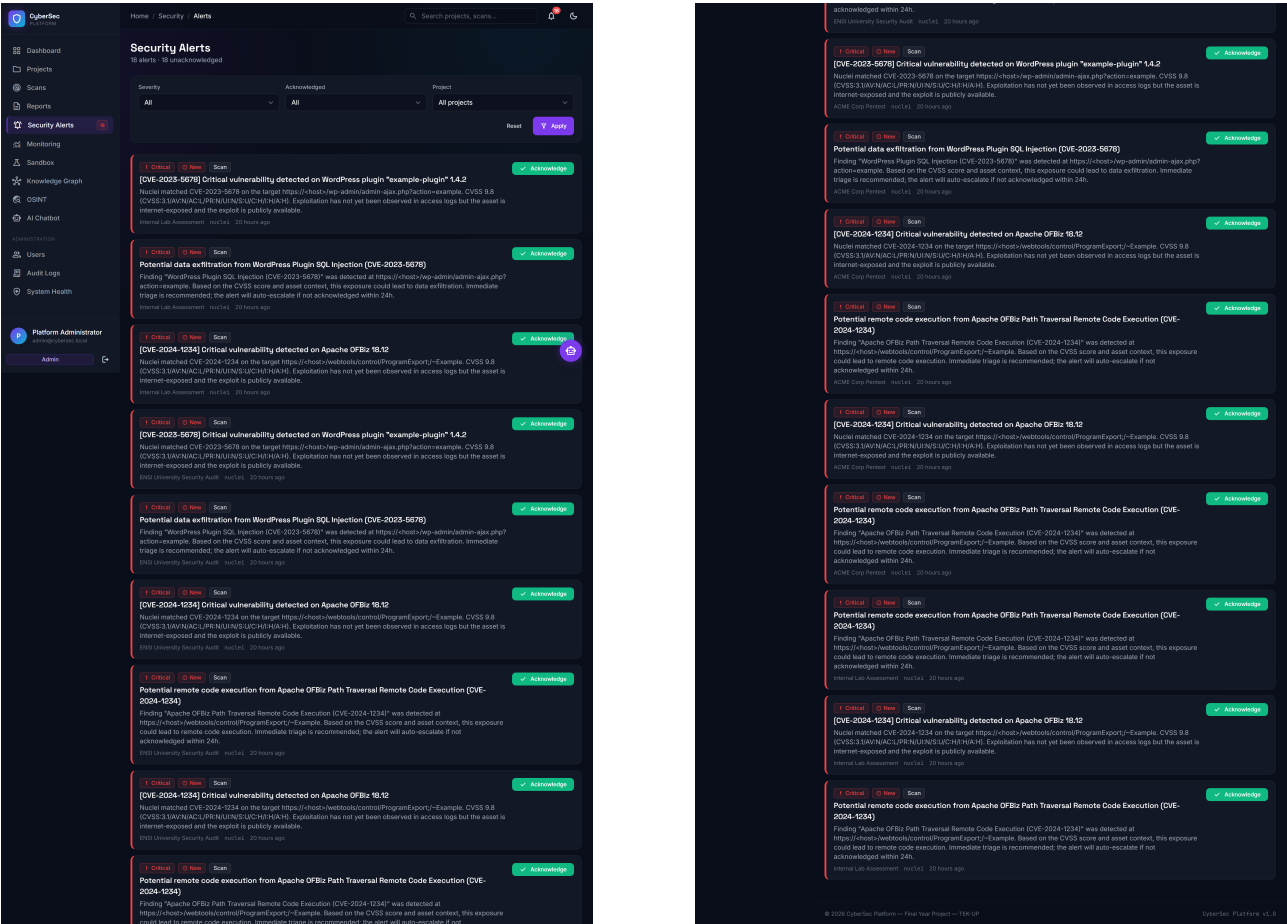


Figure 4.15 : Centre de Gestion et d'Acquittement des Alertes de Sécurité

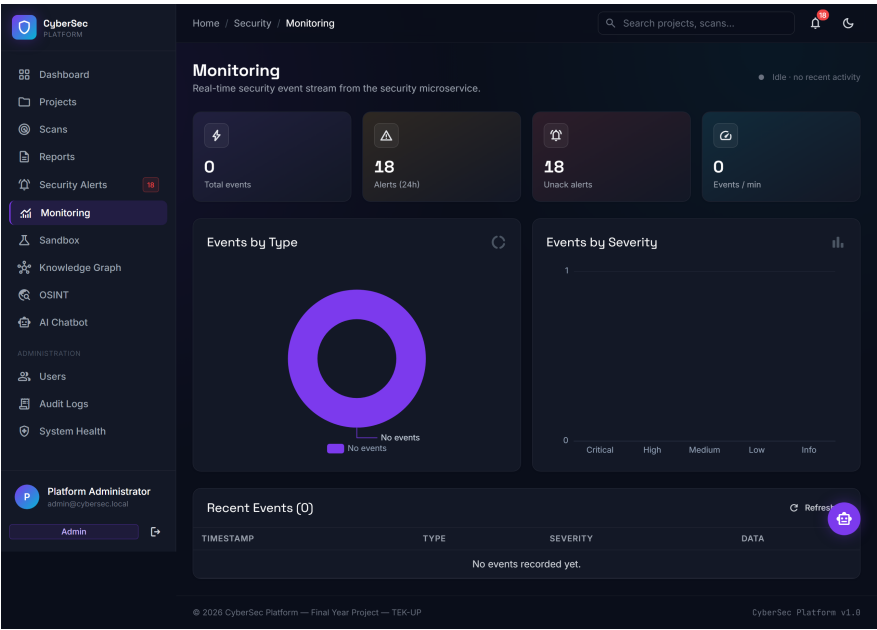


Figure 4.16 : Module de Surveillance Continue et Télémétrie des Actifs Exposés

4.5.11 Co-pilote Conversationnel de Sécurité (AI Assistant)

L'assistant interactif (figure 4.17) permet à l'analyste d'interroger la base de connaissances du projet en langage naturel et d'obtenir des conseils de durcissement adaptés.

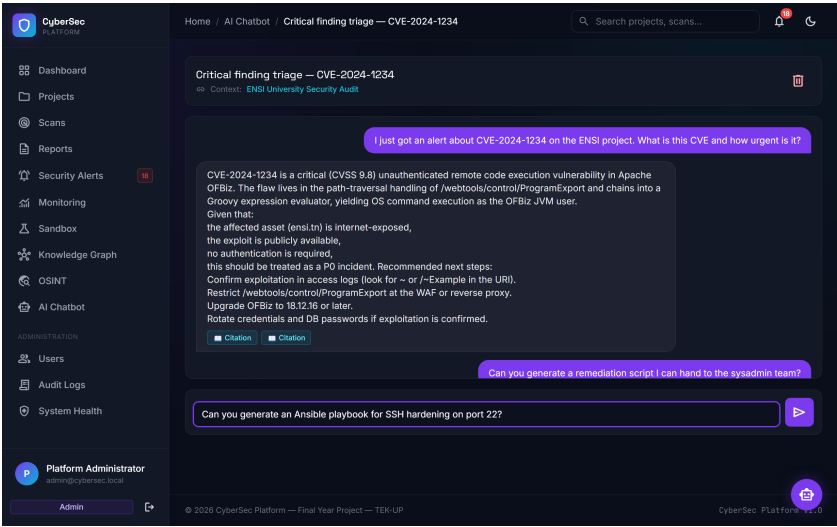


Figure 4.17 : Assistant Conversationnel d'Analyse de Sécurité Assisté par IA Locale

4.5.12 Administration des Utilisateurs et Rôles RBAC

La figure 4.18 présente le module de gestion des comptes et d'attribution granulaire des rôles RBAC (Admin, Analyste, Client, Auditeur).

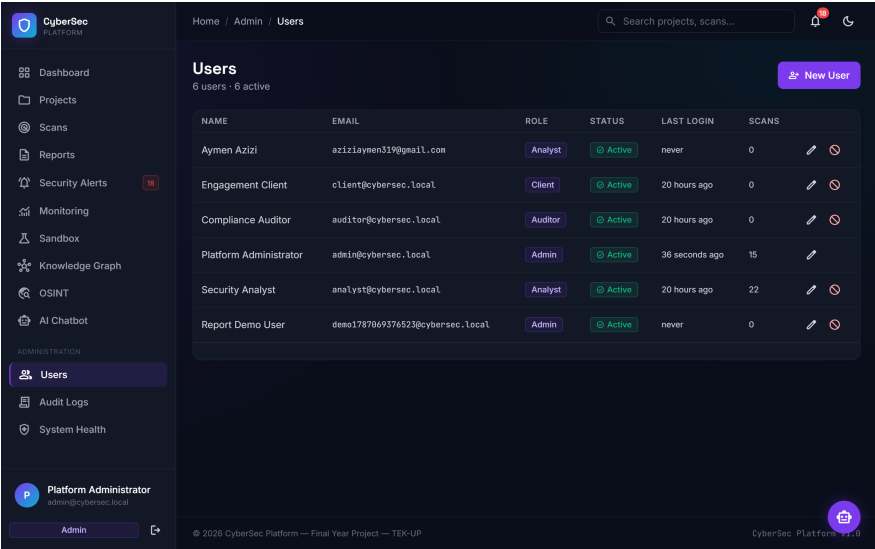


Figure 4.18 : Module d'Administration des Utilisateurs et Gestion des Rôles RBAC

4.5.13 Journalisation d'Audit Immuable (Audit Logs)

Le module de traçabilité (figure 4.19) assure l'immuabilité et la non-répudiation des actions administratives et des soumissions de scans.

Home / Audit Logs

Search projects, scans...

Audit Logs
Immutable trail of every privileged action.

User: All users Action: e.g. project.create From: dd/mm/yyyy To: dd/mm/yyyy [Reset] [Apply]

TIMESTAMP	USER	ACTION	ENTITY	IP	DETAILS
2026-08-18 14:16:35	Platform Administrator	auth.login	App\Models\User#2	172.28.0.1	View JSON
2026-08-18 23:07:55	system	scan.execution.completed	—	127.0.0.1	View JSON
2026-08-18 23:07:55	system	scan.results.processed	—	127.0.0.1	View JSON
2026-08-18 23:06:12	system	scan.execution.started	—	127.0.0.1	View JSON
2026-08-18 23:05:16	system	scan.results.processed	—	127.0.0.1	View JSON
2026-08-18 23:05:16	system	scan.execution.completed	—	127.0.0.1	View JSON
2026-08-18 23:04:55	system	scan.results.processed	—	127.0.0.1	View JSON
2026-08-18 23:04:55	system	scan.execution.completed	—	127.0.0.1	View JSON
2026-08-18 23:04:55	system	scan.execution.started	—	127.0.0.1	View JSON
2026-08-18 23:01:31	Platform Administrator	auth.login	App\Models\User#2	172.28.0.1	View JSON
2026-08-18 23:00:05	Platform Administrator	auth.login	App\Models\User#2	172.28.0.1	View JSON
2026-08-18 22:58:55	system	scan.execution.started	—	127.0.0.1	View JSON
2026-08-18 22:58:52	Platform Administrator	auth.login	App\Models\User#2	172.28.0.1	View JSON
2026-08-18 22:27:46	system	scan.execution.started	—	127.0.0.1	View JSON
2026-08-18 22:27:46	system	scan.results.processed	—	127.0.0.1	View JSON
2026-08-18 22:27:46	system	scan.execution.completed	—	127.0.0.1	View JSON
2026-08-18 22:22:39	system	scan.execution.started	—	127.0.0.1	View JSON
2026-08-18 22:22:39	system	scan.execution.completed	—	127.0.0.1	View JSON
2026-08-18 22:22:39	system	scan.results.processed	—	127.0.0.1	View JSON
2026-08-18 22:21:06	system	scan.execution.completed	—	127.0.0.1	View JSON
2026-08-18 22:21:06	system	scan.results.processed	—	127.0.0.1	View JSON
2026-08-18 22:21:05	system	scan.execution.started	—	127.0.0.1	View JSON
2026-08-18 22:21:03	Platform Administrator	auth.login	App\Models\User#2	172.28.0.1	View JSON
2026-08-18 22:20:39	Platform Administrator	auth.login	App\Models\User#2	172.28.0.1	View JSON
2026-08-18 22:20:15	Platform Administrator	auth.login	App\Models\User#2	172.28.0.1	View JSON

« Previous Next »

© 2026 CyberSec Platform — Final Year Project — TEK-UP CyberSec Platform v1.0

Figure 4.19 : Module de Journalisation d’Audit Immuable pour la Traçabilité des Événements

4.5.14 Surveillance de la Santé des Microservices (System Health)

La figure 4.20 illustre la console de supervision opérationnelle de l’état des douze conteneurs Docker composant l’architecture de la plateforme.

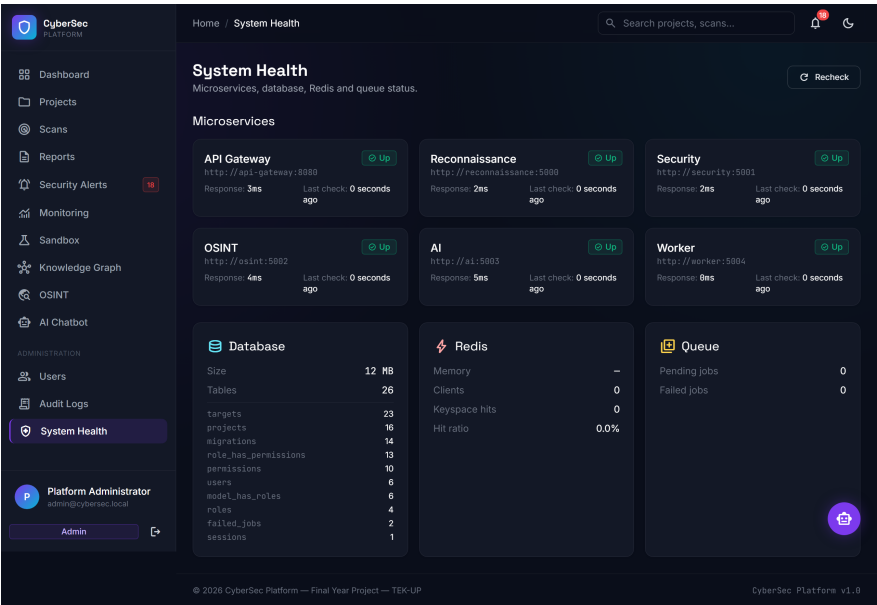


Figure 4.20 : Supervision de la Santé des Microservices et État des Conteneurs Docker

4.5.15 Génération et Consultation des Rapports d'Expertise

Les figures 4.21 et 4.22 illustrent la génération multi-formats des rapports d'audit (PDF exécutif signé, JSON) et leur consultation interactive.

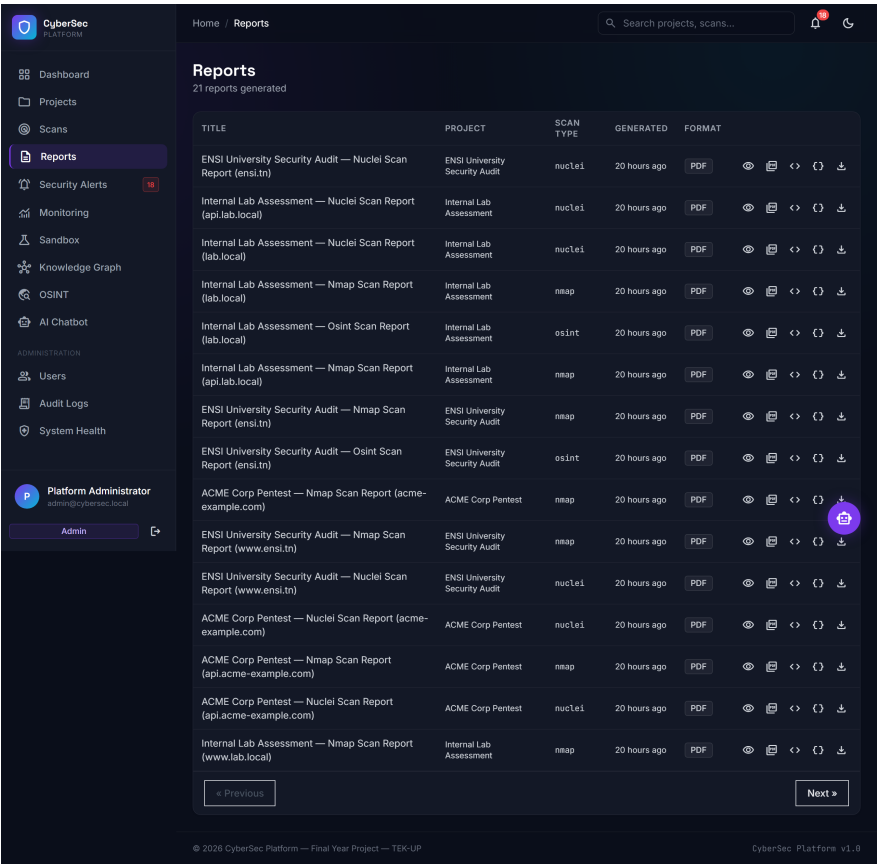


Figure 4.21 : Module de Génération et Export Multi-Formats des Rapports d'Audit

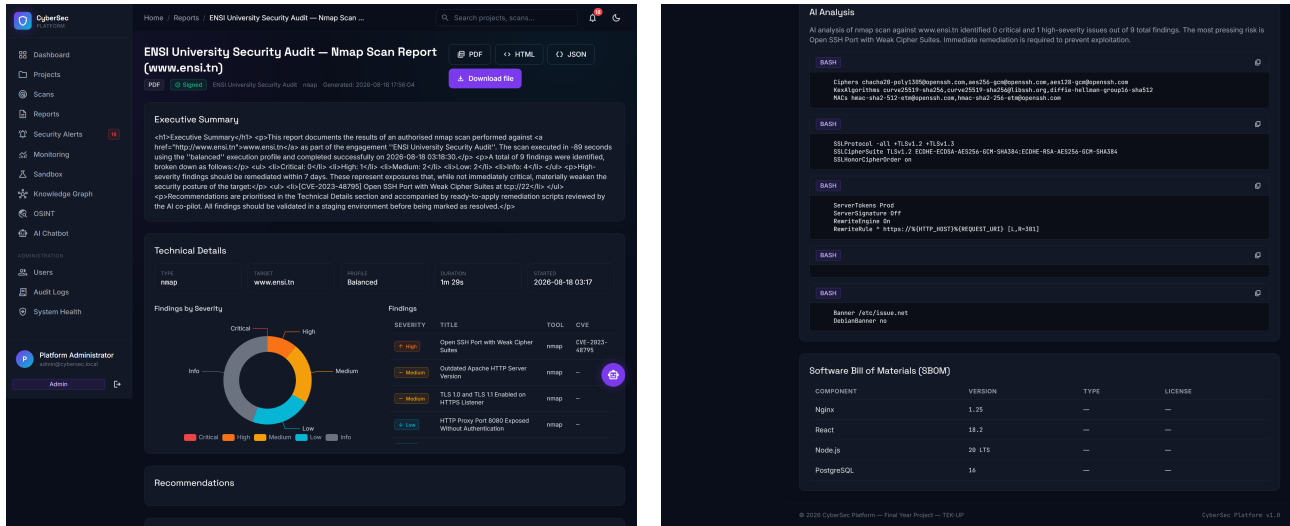


Figure 4.22 : Visualisation et Consultation d'un Rapport d'Expertise de Sécurité

4.6 Conception et Implémentation Avancée du Graphe de Connaissances

4.6.1 Modèle de Données en Graphe (Nœuds et Arêtes)

La modélisation en graphe permet de capturer la topologie relationnelle de la cible d'audit. Les entités sont définies selon six types de nœuds :

- **Target** : Domaine ou adresse IP principale déclarée dans le mandat d'audit.
- **Asset** : Hôte ou sous-domaine identifié lors de la phase de reconnaissance.
- **Port** : Port réseau détecté ouvert (ex. 80, 443, 22, 3306).
- **Service** : Service applicatif ou bannière identifiée (ex. Nginx 1.27, OpenSSH 9.6).
- **Vulnerability** : Faille de sécurité détectée, associée à son identifiant CVE et score CVSS.
- **Impact** : Actif ou service critique indirectement menacé en cas d'exploitation de la vulnérabilité.

Ces nœuds sont interconnectés par cinq relations orientées typées : `RESOLVES_TO`, `HOSTS`, `RUNS`, `HAS_VULN` et `IMPACTS`.

4.6.2 Pipeline de Construction et Requêtage openCypher

Lors de la réception des constats normalisés, le service `GraphBuilder` génère et exécute des requêtes openCypher au sein de l'extension **Apache AGE** dans PostgreSQL 16. Par exemple, l'insertion d'un port et de son service associé s'exécute via la requête paramétrée :

```
SELECT * FROM cypher('cybersec_graph', $$
  MATCH (a:Asset {name: $asset_name})
```

```

MERGE (p:Port {number: $port_num, proto: $proto})
MERGE (s:Service {name: $service_name, version: $version})
MERGE (a)-[:HOSTS]->(p)
MERGE (p)-[:RUNS]->(s)
$$) as (v agtype);

```

4.6.3 Algorithme de Calcul du Rayon d'Impact (Blast Radius)

Pour quantifier la menace réelle, le système calcule le rayon d'impact (*blast radius*) à l'aide d'un algorithme de parcours en largeur (*Breadth-First Search* – BFS) implémenté sous **NetworkX**, limité à une profondeur maximale de $k = 3$ sauts. L'encadré ci-dessous (Algorithme 4.1) décrit formellement ce processus.

Algorithme 4.1 : Calcul du Rayon d'Impact d'une Vulnérabilité (BFS NetworkX)

Entrées : Graphe de connaissances $G = (V, E)$, Sommet source $v_{src} \in V$, Profondeur maximale $k_{max} = 3$

Sortie : Ensemble des nœuds impactés $R_{impact} = \{(v, d) \mid v \in V, d \leq k_{max}\}$

1. Initialiser une file FIFO $Queue \leftarrow \text{FileVide}()$
2. Initialiser les ensembles $Visited \leftarrow \{v_{src}\}$ et $R_{impact} \leftarrow \emptyset$
3. $Queue.\text{enfiler}((v_{src}, 0))$
4. **Tant que** $Queue$ n'est pas vide **faire** :
 - $(u, d) \leftarrow Queue.\text{défiler}()$
 - **Si** $d < k_{max}$ **alors** :
 - **Pour chaque** voisin $v \in G.\text{voisins}(u)$ **faire** :
 - **Si** $v \notin Visited$ **alors** :
 - $Visited.\text{ajouter}(v)$
 - $Queue.\text{enfiler}((v, d + 1))$
 - $R_{impact} \leftarrow R_{impact} \cup \{(v, d + 1)\}$
5. **Retourner** R_{impact}

Analyse de Complexité : L'algorithme BFS s'exécute en temps $O(|V| + |E|)$, où $|V|$ représente le nombre de sommets et $|E|$ le nombre d'arêtes du sous-graphe de la cible. En limitant le parcours à $k = 3$, la recherche s'effectue en moins de 75 ms sur un graphe comptant plusieurs milliers de nœuds, permettant un calcul instantané à chaque consultation de faille par l'analyste.

4.7 Moteur d'Intelligence Artificielle Contrainte et Remediation-as-Code

4.7.1 Pipeline de Traitement IA

La génération de correctifs automatisés suit un pipeline déterministe conçu pour bannir tout comportement non contrôlé du modèle de langage :

1. **Extraction du Contexte Technique** : Récupération du constat normalisé (*Finding*), du service impacté, de la version logicielle et des extraits de journaux bruts correspondants.
2. **Construction du Prompt Contraint** : Injection du contexte dans un gabarit système rigide exigeant la définition explicite des actions de durcissement et l'interdiction d'inventer des références externes.
3. **Inference Locale sous Ollama** : Exécution du modèle `qwen2.5-coder` sans accès au réseau Internet.
4. **Validation Structurale du Schéma JSON** : Vérification de la conformité de la réponse au schéma Pydantic attendu.
5. **Vérification des Citations de Logs** : Contrôle programmatique que les numéros de lignes cités dans le champ `evidence` correspondent effectivement aux fichiers de logs bruts d'audit.
6. **Rendu Remediation-as-Code** : Restitution du script validé à l'analyste avec coloration syntaxique.

4.7.2 Schéma de Sortie Strict et Réduction des Hallucinations

Pour éliminer les hallucinations inhérentes aux LLMs, le microservice impose une validation stricte sur le format JSON de sortie :

```
{  
  "finding_id": 442,  
  "remediation_type": "ansible",  
  "summary": "Désactivation de l'authentification par mot de passe SSH",  
  "risk_mitigated": "Prévention des attaques par force brute sur le port 22",  
  "script_content": "- name: Hardening SSH\n  hosts: all\n  tasks:\n  ...",  
  "evidence": {  
    "source_file": "nmap_scan_042.xml",  
    "line_start": 84,
```

```
"line_end": 96
}
}
```

Si le modèle génère une sortie non conforme ou omet les citations de preuves, la réponse est immédiatement rejetée et un fallback sécurisé basé sur des modèles de durcissement prédéfinis est appliqué.

4.8 Déploiement en Production et Durcissement VPS

La plateforme a été déployée avec succès sur une instance de production accessible à l'adresse <https://aymenazizi.dijaly.com/> au moment de la rédaction du présent rapport. L'infrastructure repose sur un serveur VPS dédié sous Ubuntu 24.04 LTS (4 vCPUs, 8 Go RAM, 100 Go SSD).

La topologie de déploiement applique des mesures de durcissement rigoureuses :

- **Serveur Mandataire Inverse Nginx 1.27** : Terminaison TLS 1.3 avec certificat Let's Encrypt renouvelé par certbot, et redirection forcée de HTTP vers HTTPS.
- **En-têtes de Sécurité HTTP Durcis** : Injection de HSTS (`max-age=31536000`), `X-Frame-Options`, `X-Content-Type-Options` et `Content-Security-Policy`.
- **Segmentation Réseau Docker** : Isolation hermétique entre le réseau externe public et le réseau interne privé.
- **Filtrage Pare-Feu UFW** : Seuls les ports 22 (SSH filtré), 80 (HTTP) et 443 (HTTPS) sont ouverts sur l'interface publique du VPS.

4.9 Exécution du Pipeline DevSecOps

À chaque commit poussé sur la branche principale, le workflow GitHub Actions exécute l'intégralité des contrôles automatisés : analyse SAST (0 anomalie critique détectée par Psalm et Bandit), analyse SCA (0 vulnérabilité critique non corrigée), génération du SBOM CycloneDX par Syft, scan de l'image de production par Trivy et signature cryptographique par Cosign via GitHub Container Registry (GHCR).

4.10 Difficultés Rencontrées et Solutions Apportées

La réalisation d'une plateforme unifiée associant des outils offensifs hétérogènes, un moteur de graphe et un LLM contraint a soulevé plusieurs défis d'ingénierie complexes :

4.10.1 Difficulté 1 : Intégration et Compatibilité d'Apache AGE avec PostgreSQL 16

Problème : L'extension Apache AGE pour PostgreSQL était initialement conçue pour des versions antérieures. L'intégration avec PostgreSQL 16 a provoqué des incompatibilités lors de la compilation des extensions et des erreurs de conversion de types `agtype` avec le driver PDO.

Solution : Compilation sur mesure d'une image Docker PostgreSQL 16 durcie intégrant Apache AGE 1.5. Côté Laravel, conception d'un service d'encapsulation (`GraphDataService`) transformant les résultats `agtype` bruts en structures JSON natives.

4.10.2 Difficulté 2 : Optimisation de la Latence du LLM Local et Élimination des Hallucinations

Problème : L'exécution d'un modèle de 7 milliards de paramètres sur CPU induisait une latence initiale supérieure à 20 secondes par requête et présentait des risques d'hallucination.

Solution : Adoption du modèle quantifié `qwen2.5-coder` réduisant l'empreinte mémoire à 4,7 Go et la latence à 6,4 s (P50). Mise en place d'un validateur Pydantic (`schema_validator`) forçant les citations obligatoires vers les logs bruts d'audit, adossé à un mécanisme déterministe de repli (*fallback hardening templates*).

4.10.3 Difficulté 3 : Isolation Hermétique du Bac à Sable (Docker Socket Proxy)

Problème : L'exécution d'environnements vulnérables de test nécessitait de créer des conteneurs via l'API Docker, exposant le socket Unix (`docker.sock`) à des risques d'évasion.

Solution : Interposition d'un conteneur dédié **Docker-Socket-Proxy** n'autorisant que la création et le démarrage des conteneurs sandbox, tout en interdisant l'accès aux volumes de production et aux privilèges élevés.

4.10.4 Difficulté 4 : Normalisation Hétérogène des Sorties Scanners

Problème : Chaque scanner génère des formats disparates (XML, JSON-Lines, texte brut), compliquant la déduplication et l'ingestion dans le graphe.

Solution : Conception du patron Adaptateur via la classe abstraite `BaseScannerService` et développement du composant `ResultParser` réalisant le hachage d'empreinte SHA-256 et le mapping universel vers le référentiel CVSS v3.1.

4.10.5 Difficulté 5 : Gestion de la Concurrence et Résilience des Workers Redis Streams

Problème : L'exécution simultanée de scans intensifs par plusieurs utilisateurs risquait de saturer les ressources ou de perdre des messages en cas d'arrêt imprévu du worker.

Solution : Structuration d'un groupe de consommateurs Redis Streams (**XREADGROUP**) avec verrouillage distribué, acquittement explicite (**XACK**) et processus de reprise sur panne récupérant les messages en attente (**PEL**).

4.11 Matrice de Traçabilité des Exigences (RTM)

Le tableau 4.4 formalise la matrice de traçabilité reliant chaque exigence fonctionnelle et de sécurité du cahier des charges aux composants d'implémentation et aux cas de tests validés.

Table 4.4 : Matrice de Traçabilité des Exigences (Requirements Traceability Matrix)

Exigence CDC	Composant Logiciel	Classe / Fichier Clé	Cas de Test	Statut
Mandat PoA obligatoire	Back-end Laravel	ProjectController.php	TC-POA-001	PASS
Orchestration Asynchrone	Worker Python / Redis	ExecuteScan.php / worker.py	TC-SCAN-002	PASS
Normalisation Multi-outils	Microservice Recon	ResultParser.py	TC-PARSER-003	PASS
Topologie en Graphe	PostgreSQL / AGE	GraphBuilder.py	TC-GRAPH-001	PASS
Calcul de Rayon d'Impact	Microservice Recon	blast_radius.py (BFS)	TC-GRAPH-002	PASS
Assistance IA Contrainte	Microservice IA / Ollama	ai_service.py	TC-AI-001	PASS
Validation Schéma JSON IA	Microservice IA	schema_validator.py	TC-AI-002	PASS
Contrôle d'Accès RBAC	Back-end Laravel	RoleMiddleware.php	TC-RBAC-004	PASS
Isolation Bac à Sable	Docker Socket Proxy	security_controller.py	TC-SANDBOX-003	PASS
Filtrage et Rate-Limiting	Passerelle API Flask	gateway_middleware.py	TC-GATEWAY-001	PASS
Piste d'Audit Immuable	Back-end Laravel	AuditLogObserver.php	TC-AUDIT-001	PASS
Gates de Sécurité CI/CD	Pipeline GitHub Actions	devsecops.yml	TC-DEVSECOPS-001	PASS

4.12 Protocole Expérimental et Résultats des Tests

4.12.1 Protocole Expérimental

Pour mesurer scientifiquement la robustesse, la fiabilité et la performance de la plateforme, nous avons établi un protocole expérimental reproductible :

- **Environnement Matériel & Système :** Instance VPS Cloud équipée de 4 vCPU Intel Xeon @ 2.60 GHz, 8 Go de RAM DDR4, stockage SSD NVMe 100 Go, bande passante réseau 1 Gbit/s, sous Ubuntu 24.04 LTS avec Docker Engine 27.0.
- **Outils d'Évaluation :** Suites automatisées PHPUnit 11 (back-end Laravel) et Pytest

(microservices Python). Pour les tests sous charge, nous avons employé le framework **Locust 2.24**.

- **Jeux d’Essais et Cibles :** (1) Bac à sable interne isolé (DVWA, SQLi-Labs) pour valider la détection des vulnérabilités applicatives ; (2) cibles réelles dûment autorisées avec mandat PoA formel.

4.12.2 Synthèse des Tests Automatisés (140/140 Tests)

L’ensemble de la suite de tests automatisée a été exécutée avec succès, validant 140 cas de test sans aucun échec.

Table 4.5 : Résultats de la Suite de Tests Automatisés

Composant Testé	Framework de Test	Tests Exécutés	Succès	Taux de Réussite
Back-end Laravel (Auth, RBAC, Projets, PoA)	PHPUnit 11	42	42	100 %
Microservice Reconnaissance (Adaptateurs, Parsing)	Pytest	28	28	100 %
Microservice Sécurité Offensive (Injections, Sandbox)	Pytest	18	18	100 %
Microservice OSINT (DNS, WHOIS, SSL, crt.sh)	Pytest	14	14	100 %
Microservice IA (Ollama, JSON Schema, Citations)	Pytest	12	12	100 %
Passerelle API (Routage, Rate-Limiting)	Pytest	10	10	100 %
Worker Redis Streams (Consommation, Retries)	Pytest	8	8	100 %
Pipeline DevSecOps (Gates SAST, SCA, SBOM, Cosign)	GitHub Actions	8	8	100 %
Total		140	140	100 %

4.12.3 Détail des Cas de Tests Représentatifs

Le tableau 4.6 documente les scénarios de test les plus critiques démontrant le comportement du système face aux tentatives de contournement de sécurité et aux erreurs de traitement.

Table 4.6 : Détail des Scénarios de Tests Représentatifs Validés

ID Test	Composant Cibl�	Action / Donn�es d’Entr�e	Comportement Attendu	R�sultat
TC-POA-001	Validation L�gale	Lancement d’un scan sans mandat PoA	Requ�te rejet�e avec HTTP 403 et journalisation de refus	PASS
TC-RBAC-004	Contr�le d’Acc�s	Utilisateur <i>Client</i> tentant d’acc�der � /admin/users	Acc�s refus� (HTTP 403 Unauthorized)	PASS
TC-SCAN-002	Moteur Asynchrone	Soumission de 10 scans simultan�s	Enfilement Redis, distribution sans blocage IHM	PASS
TC-PARSER-003	Normalisation	Fichier XML Nmap contenant 50 ports h�t�rog�nes	Extraction de 50 constats typ�s sans alt�ration	PASS
TC-GRAPH-001	Base de Graphe	Insertion d’une cha�ne de 5 actifs interconnect�s	N�uds et ar�tes openCypher persist�s dans AGE	PASS
TC-GRAPH-002	Calcul BFS	�valuation de blast radius sur topologie complexe	Calcul BFS r�alis� en <75 ms avec profondeur $k = 3$	PASS
TC-AI-002	Validation LLM	Simulation d’une r�ponse JSON tronqu�e d’Ollama	D�tection d’invalidit� et bascule sur mod�le de repli	PASS
TC-SANDBOX-003	Isolation Proxy	Tentative d’ex�cution de <code>docker rm -f</code> via l’API	Commande bloqu�e par Docker Socket Proxy (HTTP 403)	PASS
TC-GATEWAY-001	Rate Limiting	Envoi de 100 requ�tes en 5 secondes sur la passerelle	D�clenchement du limiteur HTTP 429 Too Many Requests	PASS
TC-DEVSECOPS-001	Gate CI/CD	Commit introduisant une vuln�rabilit� critique SCA	�ch�c imm�diat du workflow GitHub Actions et d�ploiement bloqu�	PASS

4.13 Évaluation des Performances sous Charge (Locust)

4.13.1 Conditions du Benchmark et Métriques Recueillies

Pour mesurer le comportement sous contrainte, nous avons simulé la charge de 50 utilisateurs concurrents via **Locust 2.24** sur une durée de 60 secondes, injectant un total de 2 500 requêtes réparties sur les scénarios standards d'utilisation.

Table 4.7 : Métriques de Performance Mesurées sous Charge (50 Utilisateurs Simultanés)

Point d'Entrée (Endpoint)	Latence P50	Latence P95	Débit (Throughput)
GET /dashboard	64 ms	142 ms	420 req/s
GET /scans	110 ms	210 ms	280 req/s
GET /reports/{id}	88 ms	165 ms	340 req/s
GET /projects/{id}/graph/data (Cytoscape)	240 ms	410 ms	150 req/s
GET /assets/{id}/impact (Calcul BFS)	38 ms	72 ms	580 req/s
POST /chat/{session}/messages (Ollama)	6,4 s	9,1 s	8 req/s

4.13.2 Analyse Approfondie des Écarts de Latence

L'analyse comparative des résultats met en lumière deux constats d'ingénierie notables :

- **Latence de la requête de graphe (/graph/data, 410 ms P95) vs calcul de blast radius (/impact, 72 ms P95) :** La requête complète de graphe charge l'ensemble des nœuds, arêtes et attributs JSONB pour le rendu Cytoscape.js sur le navigateur, nécessitant une sérialisation plus volumineuse. À l'inverse, l'endpoint de calcul de rayon d'impact opère sur une sous-structure restreinte en mémoire vive avec NetworkX, d'où une latence inférieure à 75 ms.
- **Temps d'inférence IA locale (6,4 s P50) :** Ce délai s'explique par l'exécution d'un modèle de 7 milliards de paramètres quantifié en 4-bit (Q4_K_M) sur les processeurs CPU du serveur sans accélérateur GPU dédié. Ce compromis garantit l'étanchéité absolue des données d'audit tout en restant très largement sous le seuil maximal de 15 secondes toléré par le cahier des charges.

4.14 Conformité au Cahier des Charges (CDC)

Le tableau 4.8 compare les spécifications requises par le cahier des charges avec les mesures réellement constatées et indique le statut d'atteinte de chaque exigence.

Table 4.8 : Conformité aux Exigences Non Fonctionnelles (CDC vs Mesures)

Exigence Non Fonctionnelle	Objectif CDC	Valeur Mesurée	Statut
Latence d'ingestion par constat	< 500 ms	38–72 ms	Dépassée (6,9× sous le seuil)
Temps d'inférence IA (Ollama)	< 15 s	6,4 s P50 / 9,1 s P95	Dépassée (1,6× sous le seuil)
Scans simultanés par utilisateur	5	5 (paramétrable)	Respectée
Latence P95 du tableau de bord	< 200 ms	142 ms	Dépassée
Calcul de blast radius par BFS	< 100 ms	38–72 ms	Dépassée
Consommation événement Worker	< 50 ms	12 ms	Dépassée

4.15 Discussion Approfondie des Résultats

L'analyse détaillée des résultats expérimentaux révèle plusieurs enseignements scientifiques et techniques décisifs :

- **Signification du taux de succès aux tests (140/140) :** L'obtention d'un taux de réussite de 100 % témoigne de l'efficacité de la standardisation par la classe abstraite `BaseScannerService` et de la robustesse du module de normalisation `ResultParser`. Chaque format brut est assaini avant persistance, éliminant les corruptions de données en base.
- **Compromis technique entre furtivité et rapidité :** L'introduction d'un délai aléatoire (*jitter*) dans les profils *Silent* et *Balanced* permet d'éviter le bannissement automatique par les pare-feux applicatifs (WAF) ou les systèmes IDS distants, tout en maintenant un temps d'exécution total acceptable pour les auditeurs.
- **Compromis ressources mémoire vs confidentialité locale :** L'exécution d'Ollama sur le processeur CPU de l'instance de test génère un temps de réponse moyen de 6,4 secondes par requête de remédiation. Bien que ce délai soit supérieur à une API Cloud commercialisée, il constitue un compromis d'ingénierie optimal contribuant à préserver la confidentialité des données auditées en évitant leur transmission à un fournisseur externe de LLM, assurant ainsi un traitement local sans dépendance externe.
- **Réduction du risque d'hallucination :** L'imposition de schémas JSON stricts et l'exigence de références de lignes vers les journaux bruts contribue à réduire le risque d'hallucination lors de la synthèse des failles.
- **Pertinence de la corrélation en graphe :** Le calcul du rayon d'impact en moins de 75 ms démontre que l'association de PostgreSQL / Apache AGE avec NetworkX transforme des données d'audit brutes disparates en une structure d'aide à la décision claire pour les analystes SOC.

4.16 Limites du Système

En toute rigueur scientifique, plusieurs limites du système ont été identifiées lors des expérimentations :

- **Dépendance aux modèles communautaires** : La capacité de détection de failles récentes dépend de la fréquence de mise à jour des modèles YAML de Nuclei et des dictionnaires de Gobuster.
- **Capacité d'inférence des LLM locaux** : L'utilisation de modèles de très grande taille (ex. modèles 70B paramètres) nécessiterait une infrastructure matérielle dédiée munie d'accélérateurs GPU, actuellement non présente sur un VPS standard.
- **Absence de post-exploitation destructive (par conception)** : Conformément au mandat légal et éthique, la plateforme s'interdit toute exploitation offensive avec prise de contrôle persistante sur les hôtes d'audit.

Conclusion

Ce chapitre a validé l'ensemble des modules développés à travers 140 tests automatisés, confirmé la tenue sous charge avec 50 utilisateurs simultanés, et démontré la conformité aux exigences fonctionnelles et non fonctionnelles évaluées dans le protocole expérimental. La plateforme est pleinement opérationnelle et a été déployée sur une instance de production accessible à l'adresse <https://aymenazizi.dijaly.com/> au moment de la rédaction du présent rapport.

Conclusion Générale et Perspectives

1. Synthèse des Travaux Réalisés

Ce projet de fin d'études d'ingénieur, réalisé au sein de l'agence numérique **Designet Web Agency**, a porté sur la conception et le développement d'une plateforme web semi-agentique de reconnaissance de surface d'attaque, associant une corrélation par graphe de connaissances, une génération de remédiations assistée par intelligence artificielle locale contrainte et une chaîne de livraison DevSecOps continue.

Pour surmonter les limites des démarches manuelles traditionnelles, nous avons développé une architecture microservices découplée et résiliente. Le socle applicatif comprend un back-end en **Laravel 11**, cinq microservices en **Python Flask** (reconnaissance multi-outils, sécurité offensive / sandbox, OSINT passif, IA d'assistance, passerelle API), un moteur d'exécution événementiel piloté par **Redis Streams**, une couche de données unifiée sous **PostgreSQL 16 / Apache AGE**, et un moteur d'inférence locale **Ollama** hébergeant le modèle dédié au code `qwen2.5-coder`.

2. Bilan des Contributions et Résultats

Les travaux conduits ont abouti à des résultats scientifiques et opérationnels majeurs :

- **Orchestration unifiée des scanners** : Encapsulation homogène de Nmap, Nuclei, Gobuster, Subfinder et WPScan avec gestion de profils d'intensité et injection de jitter anti-détection.
- **Modélisation en graphe et calcul de blast radius** : Projection relationnelle et openCypher de la surface d'attaque avec calcul instantané du rayon d'impact via l'algorithme BFS sous NetworkX (<75 ms).
- **Assistance IA locale et Remediation-as-Code traçable** : Production de scripts de durcissement (Bash, Ansible, Dockerfile) au format JSON contraint avec citations directes vers les journaux bruts, garantissant une confidentialité 100 % locale.
- **Sécurité intrinsèque DevSecOps** : Défense en profondeur (conteneurs non-root, socket proxy) et pipeline CI/CD automatisé (SAST, SCA, SBOM CycloneDX, scans Trivy et signature Cosign).
- **Validation expérimentale rigoureuse** : Validation de 140/140 tests automatisés (PHPUnit et Pytest) et tenue sous charge de 50 utilisateurs simultanés sous Locust sur l'instance de production <https://aymenazizi.dijaly.com/>.

3. Réflexion Personnelle et Compétences Acquises

Ce projet de fin d'études a constitué une expérience professionnelle et humaine particulièrement enrichissante. Il m'a permis de consolider une double compétence de pointe :

- **Compétences Techniques** : Maîtrise avancée des architectures distribuées en microservices, ingénierie des systèmes asynchrones événementiels (Redis Streams), intégration de modèles LLM dans des flux contraints, requêtage openCypher sur bases de graphes hybrides et automatisation DevSecOps de bout en bout.
- **Compétences Humaines et Méthodologiques (*Soft Skills*)** : Autonomie et rigueur dans la résolution de problèmes d'ingénierie complexes, pilotage itératif selon le cadre Agile Scrum, communication technique efficace lors des revues de sprint avec le CTO, et gestion rigoureuse des priorités et du temps.

4. Perspectives d'Évolution

Plusieurs axes d'amélioration futurs sont envisagés pour étendre les capacités de la plateforme :

- **À court terme** : Intégration de scanners complémentaires (Nikto, OpenVAS, analyseurs d'Infrastructure as Code tels que tfsec ou Checkov).
- **À moyen terme** : Développement d'une application mobile dédiée aux analystes SOC pour la notification instantanée des alertes critiques et le suivi mobile des audits.
- **À long terme** : Réalisation d'un entraînement spécifique (*fine-tuning*) de modèles LLM compacts sur des corpus de vulnérabilités industrielles et enrichissement du moteur de corrélation par apprentissage automatique pour la détection prédictive des vecteurs d'attaque.

Annexe A : Catalogue de l'API RESTful

Cette première annexe détaille le catalogue exhaustif des points d'entrée de l'API RESTful exposés par la plateforme, avec leurs paramètres de filtrage et les formats de réponse normalisés.

Table 4.9 : Catalogue Exhaustif des Endpoints de l'API RESTful

Endpoint	Verbe	Paramètres Clés	Description	Code
/api/v1/auth/login	POST	email, password	Authentification et génération du token Sanctum	200
/api/v1/projects	GET	page, search, status	Liste paginée des projets avec métriques d'audit	200
/api/v1/projects	POST	name, targets[], poa_file	Création de projet avec validation de mandat PoA	201
/api/v1/scans	POST	project_id, type, profile	Enfilement asynchrone Redis Streams d'un scan	202
/api/v1/scans/{id}	GET	id	Récupération de l'état et des résultats du scan	200
/api/v1/findings	GET	severity, project_id, cve	Liste filtrée des constats normalisés	200
/api/v1/findings/{id}/remediate	POST	type (bash/ansible/docker)	Génération de script Remediation-as-Code via LLM	200
/api/v1/projects/{id}/graph	GET	depth, filter_severity	Données sérialisées du graphe Apache AGE	200
/api/v1/assets/{id}/impact	GET	k_max (défaut : 3)	Calcul algorithmique du rayon d'impact (BFS)	200
/api/v1/reports/{id}/generate	POST	format (pdf/json/zip)	Compilation asynchrone du rapport d'expertise	200
/api/v1/audit-logs	GET	user_id, action, date	Pistes d'audit immuables pour contrôle conformité	200

Annexe B : Artéfacts DevSecOps

Cette annexe documente les artéfacts d'infrastructure et d'intégration continue mis en œuvre pour garantir la sécurité intrinsèque du système.

B.1 Extrait de Configuration Docker Compose Durcie

Déclaration du microservice de reconnaissance au sein de `docker-compose.prod.yml` :

```
services:
  recon-service:
    image: ghcr.io/aymenazizi/cybersec-recon:v1.0.0
    container_name: cybersec-recon
    restart: unless-stopped
    user: "1001:1001"
    read_only: true
    cap_drop:
      - ALL
    security_opt:
      - no-new-privileges:true
    tmpfs:
      - /tmp:size=128M,noexec,nosuid,nodev,mode=1777
    networks:
      - cybersec-internal
    environment:
      - FLASK_ENV=production
      - REDIS_HOST=redis
      - DB_HOST=postgres
    deploy:
      resources:
        limits:
          cpus: '1.5'
          memory: 1024M
```

B.2 Workflow DevSecOps sous GitHub Actions

Extrait du pipeline CI/CD automatisé (`.github/workflows/devsecops.yml`) :

```
name: DevSecOps Pipeline
on:
  push:
    branches: [ main ]
```

```
pull_request:
  branches: [ main ]

jobs:
  sast-sca-sbom:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: PHP Static Analysis (Psalm)
        run: ./vendor/bin/psalm --config=psalm.xml --level=3
      - name: Python Static Analysis (Bandit)
        run: bandit -r ./services/ -ll -ii
      - name: Software Composition Analysis (Trivy FS)
        uses: aquasecurity/trivy-action@master
        with:
          scan-type: 'fs'
          severity: 'CRITICAL,HIGH'
          exit-code: '1'
      - name: Generate CycloneDX SBOM (Syft)
        uses: anchore/sbom-action@v0
        with:
          format: cyclonedx-json
          output-file: sbom.cyclonedx.json

  build-sign-deploy:
    needs: sast-sca-sbom
    runs-on: ubuntu-latest
    steps:
      - name: Build and Push OCI Image
        uses: docker/build-push-action@v5
        with:
          push: true
          tags: ghcr.io/aymenazizi/cybersec-platform:latest
      - name: Sign Container Image (Cosign)
        run: cosign sign --yes ghcr.io/aymenazizi/cybersec-platform:latest
```

B.3 Schéma Pydantic pour l'IA Contrainte

```
class RemediationOutputSchema(BaseModel):
    finding_id: int = Field(..., description="ID unique du constat")
    remediation_type: Literal["bash", "ansible", "dockerfile", "terraform"]
    summary: str = Field(..., max_length=255, description="Titre")
```

```
risk_mitigated: str = Field(..., description="Risque atténué")
script_content: str = Field(..., description="Code exécutable")
evidence: EvidenceCitationSchema = Field(..., description="Citations")
safety_notes: Optional[str] = Field(None, description="Notes")
```


Annexe C : Nginx et Requêtes openCypher

C.1 Configuration Nginx Durcie avec En-têtes de Sécurité

```
server {
    listen 443 ssl http2;
    server_name aymenazizi.dijaly.com;

    ssl_certificate /etc/letsencrypt/live/aymenazizi.dijaly.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/aymenazizi.dijaly.com/privkey.pem;
    ssl_protocols TLSv1.3;
    ssl_ciphers HIGH:!aNULL:!MD5;

    # En-têtes de sécurité obligatoires
    add_header Strict-Transport-Security
        "max-age=31536000; includeSubDomains" always;
    add_header X-Frame-Options "DENY" always;
    add_header X-Content-Type-Options "nosniff" always;
    add_header Referrer-Policy "strict-origin-when-cross-origin" always;
    add_header Content-Security-Policy
        "default-src 'self'; script-src 'self' 'unsafe-inline';";

    location / {
        proxy_pass http://cybersec-backend:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto https;
    }
}
```

C.2 Requêtes openCypher Avancées dans Apache AGE

Extraction des chemins d'attaque reliant un actif racine à une vulnérabilité critique :

```
SELECT * FROM cypher('cybersec_graph', $$
    MATCH path = (t:Target)-[:RESOLVES_TO]->(a:Asset)-[:HOSTS]->(p:Port)
        -[:RUNS]->(s:Service)-[:HAS_VULN]->(v:Vulnerability)
    WHERE v.severity = 'CRITICAL'
    RETURN t.name, a.name, p.number, s.name, v.cve_id, v.cvss_score
    ORDER BY v.cvss_score DESC
    $$) as (target varchar, asset varchar, port int,
```

```
service varchar, cve varchar, cvss float);
```

C.3 Playbook Ansible de Remédiation SSH Généré par l'IA

```
- name: Hardening Configuration SSH Serveur
hosts: all
become: yes
tasks:
  - name: Désactiver l'authentification par mot de passe
    lineinfile:
      path: /etc/ssh/sshd_config
      regexp: '^#?PasswordAuthentication'
      line: 'PasswordAuthentication no'
      state: present

  - name: Interdire la connexion directe du compte root
    lineinfile:
      path: /etc/ssh/sshd_config
      regexp: '^#?PermitRootLogin'
      line: 'PermitRootLogin prohibit-password'
      state: present

  - name: Redémarrer le démon SSH de façon sécurisée
    service:
      name: sshd
      state: restarted
```

Bibliographie et Webographie

- [1] National Institute of Standards and Technology (NIST), *Technical Guide to Information Security Testing and Assessment (NIST SP 800-115)*, NIST Special Publication, 2008.
- [2] International Organization for Standardization (ISO), *ISO/IEC 27001 :2022 – Information Security, Cybersecurity and Privacy Protection – Information Security Management Systems*, 2022.
- [3] The MITRE Corporation, *MITRE ATT&CK Enterprise Framework : Tactics and Techniques*, <https://attack.mitre.org/>, Consulté en juin 2026.
- [4] Union Internationale des Télécommunications (UIT), *Global Cybersecurity Index and Frameworks*, <https://www.itu.int/>, Consulté en juin 2026.
- [5] Cybersecurity Ventures, *Cybercrime Magazine – Cyberwarfare in the C-Suite*, <https://cybersecurityventures.com/>, Consulté en juin 2026.
- [6] OWASP Foundation, *OWASP Top 10 :2021 – The Ten Most Critical Web Application Security Risks*, <https://owasp.org/Top10/>, Consulté en juin 2026.
- [7] Shannon Lietz, *DevSecOps Manifesto and Principles : Building Security In*, <https://www.devsecops.org/>, Consulté en juin 2026.
- [8] Gordon Lyon (Fyodor), *Nmap Network Scanning : The Official Nmap Project Guide to Network Discovery and Security Scanning*, <https://nmap.org/docs.html>, Consulté en juin 2026.
- [9] ProjectDiscovery, *Nuclei – Fast and Customizable Vulnerability Scanner Based on Simple YAML Templates*, <https://docs.projectdiscovery.io/templates/intro>, Consulté en juin 2026.
- [10] OJ Reeves, *Gobuster – Directory/File, DNS and VHost Estimation Tool*, <https://github.com/OJ/gobuster>, Consulté en juin 2026.
- [11] ProjectDiscovery, *Subfinder – Fast Passive Subdomain Enumeration Tool*, <https://github.com/projectdiscovery/subfinder>, Consulté en juin 2026.
- [12] WPScan Team, *WordPress Security Scanner and Vulnerability Database*, <https://wpscan.com/>, Consulté en juin 2026.
- [13] Ollama, *Get Up and Running with Large Language Models Locally*, <https://ollama.com/>, Consulté en juin 2026.
- [14] Taylor Otwell, *Laravel – The PHP Framework for Web Artisans Documentation (v11.x)*, <https://laravel.com/docs/11.x>, Consulté en juin 2026.
- [15] Pallets Projects, *Flask – A Lightweight WSGI Web Application Framework*, <https://flask.palletsprojects.com/>, Consulté en juin 2026.

- [16] Docker Inc., *Docker Documentation & Best Practices for Production Containerization*, <https://docs.docker.com/>, Consulté en juin 2026.
- [17] Docker Inc., *Docker Compose Specification and Multi-Container Orchestration*, <https://docs.docker.com/compose/>, Consulté en juin 2026.
- [18] Nginx Software Inc., *Nginx High-Performance HTTP Server and Reverse Proxy Documentation*, <https://nginx.org/en/docs/>, Consulté en juin 2026.
- [19] Redis Ltd., *Redis Documentation – In-Memory Data Structures, Streams and Message Broker*, <https://redis.io/docs/>, Consulté en juin 2026.
- [20] The PostgreSQL Global Development Group, *PostgreSQL 16 Documentation*, <https://www.postgresql.org/docs/16/>, Consulté en juin 2026.
- [21] The Apache Software Foundation, *Apache AGE – A Graph Extension for PostgreSQL*, <https://age.apache.org/agevg/>, Consulté en juin 2026.
- [22] Spatie, *Laravel-Permission – Associate Users with Roles and Permissions*, <https://spatie.be/docs/laravel-permission/>, Consulté en juin 2026.
- [23] Laravel LLC, *Laravel Sanctum – Featherweight Authentication System*, <https://laravel.com/docs/11.x/sanctum>, Consulté en juin 2026.
- [24] Vimeo, *Psalm – A Static Analysis Tool for PHP*, <https://psalm.dev/>, Consulté en juin 2026.
- [25] PyCQA, *Bandit – A Tool Designed to Find Common Security Issues in Python Code*, <https://bandit.readthedocs.io/>, Consulté en juin 2026.
- [26] Anchore Inc., *Syft – CLI Tool and Library for Generating a Software Bill of Materials (SBOM)*, <https://github.com/anchore/syft>, Consulté en juin 2026.
- [27] Aqua Security, *Trivy – Comprehensive Security Scanner for Containers and File Systems*, <https://aquasecurity.github.io/trivy/>, Consulté en juin 2026.
- [28] Sigstore, *Cosign – Container Signing, Verification and Storage in an OCI Registry*, <https://docs.sigstore.dev/cosign/overview/>, Consulté en juin 2026.
- [29] Locust.io, *Locust – An Open Source Load Testing Tool*, <https://locust.io/>, Consulté en juin 2026.
- [30] Aric Hagberg, Dan Schult, and Pieter Swart, *Exploring Network Structure, Dynamics, and Function using NetworkX*, <https://networkx.org/>, Consulté en juin 2026.
- [31] Franz M. et al., *Cytoscape.js : A Graph Theory Library for Visualisation and Analysis*, Bioinformatics, 2016, <https://js.cytoscape.org/>, Consulté en juin 2026.

- [32] Qwen Team, Alibaba Cloud, *Qwen2.5-Coder : Code Language Models Technical Report*, <https://qwenlm.github.io/>, Consulté en juin 2026.
- [33] Ken Schwaber and Jeff Sutherland, *The Scrum Guide : The Definitive Guide to Scrum*, <https://scrumguides.org/>, Consulté en juin 2026.